



INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Departamento de Ciências e Tecnologia da Informação

**Aprendizagem por Reforço Multiagente e Neuroevolução
no Controlo de Exames de Robôs:
Desempenho, Escalabilidade e Robustez**

Gonçalo Alexandre Pombo Santos

Mestrado em Inteligência Artificial

Orientador:
Doutor Luís Miguel Martins Nunes,
Professor Associado com Agregação,
Iscte-Instituto Universitário de Lisboa

setembro, 2026

[Página intencionalmente deixada em branco.]



TECNOLOGIAS
E ARQUITETURA

Departamento de Ciências e Tecnologia da Informação

**Aprendizagem por Reforço Multiagente e Neuroevolução
no Controlo de Enxames de Robôs:
Desempenho, Escalabilidade e Robustez**

Gonçalo Alexandre Pombo Santos

Mestrado em Inteligência Artificial

Orientador:
Doutor Luís Miguel Martins Nunes,
Professor Associado com Agregação,
Iscte-Instituto Universitário de Lisboa

setembro, 2026

[Página intencionalmente deixada em branco.]

Aos meus pais e a todos que apoiaram este percurso.

[Página intencionalmente deixada em branco.]

Agradecimentos

Ao Professor Doutor Luís Nunes, meu orientador, pelo apoio e pela direção que me deu durante toda a jornada desta tese. Cada reunião ajudou-me a pôr ordem nas ideias, e cada revisão do texto trouxe perguntas que me fizeram voltar aos dados e confirmar o que afirmava antes de o escrever. Foi com essa exigência que o trabalho ganhou forma. Agradeço-lhe também a liberdade para seguir por caminhos que, à partida, nenhum de nós sabia onde iam dar.

Ao Iscte – Instituto Universitário de Lisboa, pelas boas condições de trabalho ao longo do mestrado e, sobretudo, pelo empréstimo dos servidores em que treinei os algoritmos. Os resultados desta dissertação vêm de centenas de treinos, e fazê-los apenas no meu computador teria sido impossível.

À minha família, por estar sempre presente, do primeiro ao último dia deste percurso. Nunca me faltou uma palavra de apoio nem a paciência para os meses em que a tese ocupou quase todo o meu tempo. Aos meus pais, em particular, devo tudo o que me permitiu chegar até aqui.

À minha namorada, a quem este trabalho deve mais do que imagina. Acompanhou de perto cada fase, ouviu-me falar de robôs e de exames mais vezes do que alguma vez quis, e aceitou sem queixas os fins de semana em que o computador levou a melhor. Nos dias em que nada corria bem, foi ela quem me deu ânimo para continuar e quem me lembrou que havia vida além da tese. Obrigado por nunca teres deixado de estar do meu lado.

À TAISCTE, por ter feito parte deste percurso de mestrado. As pessoas que lá conheci e os momentos que vivemos juntos deram a estes anos um equilíbrio de que precisei muito mais do que esperava.

Aos amigos e colegas que fiz pelo caminho, pelas conversas, pela entreaajuda e pela boa companhia, e aos professores que deram o seu melhor para nos ensinar.

A todos, o meu muito obrigado.

This work was partially supported by national funds through Fundação para a Ciência e a Tecnologia, I.P. (FCT), ISTAR-Iscte Pluriannual Projects UID/04466/2025 (<https://doi.org/10.54499/UID/04466/2025>).

[Página intencionalmente deixada em branco.]

Resumo

Esta dissertação compara a aprendizagem por reforço multiagente e a otimização bio-inspirada no controlo descentralizado de enxames de robôs. Num simulador próprio, vinte robôs têm de chegar em conjunto a um ninho. Um controlador evolutivo, uma rede neuronal sobre grafos com atenção, foi comparado com dois algoritmos de aprendizagem por reforço por gradiente (*Proximal Policy Optimization* e *Soft Actor-Critic*) em sete cenários de dificuldade crescente, com 7 treinos independentes por algoritmo e cenário.

Com uma função de aptidão que premeia também a aproximação ao ninho, o controlador evolutivo supera os métodos de gradiente em três cenários, iguala o melhor noutros dois e é o único que funciona sem novo treino com enxames de 10 a 100 robôs; os métodos de gradiente são mais fiáveis em espaço aberto e gastam menos cálculo. O cenário que obriga a afastar-se do ninho para o alcançar só foi resolvido de forma consistente com pressão por comportamentos novos, atenuada após a descoberta: numa replicação com 28 treinos por algoritmo, resolveu-o em 28, contra 15 sem essa pressão e 14 de cada método de gradiente. Um oitavo cenário, que junta as dificuldades dos sete num labirinto de 103×62 m, é resolvido em apenas 4 de 21 treinos.

A hipótese de que a aprendizagem adaptativa supera a otimização estática confirma-se só em parte: escalar depende da representação da política, não do otimizador; nos cenários enganadores, depende do sinal de treino.

PALAVRAS-CHAVE: Robótica de Enxame, Aprendizagem por Reforço Multiagente, Otimização Bio-inspirada, Redes Neurais de Grafos.

[Página intencionalmente deixada em branco.]

Abstract

This dissertation compares multi-agent reinforcement learning and bio-inspired optimization for the decentralized control of robot swarms. In a purpose-built simulator, twenty robots must jointly reach a nest. An evolutionary controller, a graph neural network with attention, was compared with two gradient-based reinforcement learning algorithms (*Proximal Policy Optimization* and *Soft Actor-Critic*) across seven scenarios of increasing difficulty, with 7 independent training runs per algorithm and scenario.

With a fitness function that also rewards approaching the nest, the evolutionary controller outperforms the gradient-based methods in three scenarios, matches the best of them in two others, and is the only one that works without retraining on swarms of 10 to 100 robots; the gradient-based methods are more reliable in open space and need less computation. The scenario that forces the robots to move away from the nest in order to reach it was only solved consistently with pressure towards novel behaviours, attenuated after discovery: in a replication with 28 training runs per algorithm, it solved it in 28, against 15 without that pressure and 14 for each gradient-based method. An eighth scenario, which combines the difficulties of the seven in a 103×62 m maze, is solved in only 4 of 21 runs.

The hypothesis that adaptive learning outperforms static optimization is only partly confirmed: scaling depends on the representation of the policy, not on the optimizer; in deceptive scenarios, it depends on the training signal.

KEYWORDS: Swarm Robotics, Multi-Agent Reinforcement Learning, Bio-inspired Optimization, Graph Neural Networks.

[Página intencionalmente deixada em branco.]

Índice

Agradecimentos	iii
Resumo	v
Abstract	vii
Lista de Figuras	xiii
Lista de Quadros	xv
Lista de Acrónimos	xvii
Capítulo 1. Introdução	1
1.1. Enquadramento e Motivação	1
1.2. Definição do Problema e Justificação da Tese	2
1.3. Objetivos da Tese	3
1.4. Questões de Investigação	4
1.5. Estrutura do Documento	5
Capítulo 2. Fundamentos Teóricos e Tecnologias	7
2.1. Introdução ao Capítulo	7
2.2. Swarm Intelligence (SI) e Robótica de Enxame	7
2.3. Paradigmas de Controlo para Enxames	8
2.4. Abordagem 1: Otimização Bio-inspirada	9
2.4.1. Fundamentos Matemáticos dos Algoritmos Evolutivos	9
2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)	11
2.5.1. Processos de Decisão de Markov (MDPs) e Dec-POMDP	11
2.5.2. Otimização de Políticas: Do Actor-Critic ao PPO e SAC	12
2.5.3. Fundamentos de Graph Neural Networks e Mecanismos de Atenção	14
Capítulo 3. Estado da Arte	17
3.1. Introdução ao Capítulo	17
3.2. Metodologia da Revisão Sistemática	17
3.2.1. Pergunta e Critérios de Elegibilidade	17
3.2.2. Protocolo de Pesquisa e Bases de Dados	18
3.2.3. Processo de Seleção (Fluxo PRISMA)	19
3.2.4. Fronteira entre o Corpo da Revisão e as Fontes de Fundamentação	19
3.2.5. Ameaças à Validade da Revisão	20

3.3.	Caracterização do Corpo de Literatura	21
3.4.	Análise dos Melhores Resultados	22
3.4.1.	Subgrupo A: Otimização Bio-inspirada e Desenho Automático de Enxames	22
3.4.2.	Subgrupo B: MARL, Escalabilidade e Arquiteturas de Grafo	22
3.4.3.	O Estudo Comparativo Isolado	23
3.5.	O Desafio do <i>Deployment Gap</i> em Robótica de Enxame	24
3.5.1.	Fontes do Fosso de Realidade	24
3.5.2.	Agravamento à Escala do Enxame	24
3.5.3.	Estratégias de Mitigação	25
3.5.4.	Posicionamento desta Dissertação	25
3.6.	Lacunas Identificadas e Oportunidade de Melhoria	25
3.7.	Artigos-Chave e Ligação ao Problema	26
3.8.	Conclusão Parcial	27
3.9.	Síntese Comparativa da Literatura	28
Capítulo 4.	Desenvolvimento do Ambiente de Simulação	29
4.1.	Arquitetura de Software e Stack Tecnológica	29
4.1.1.	Ambiente de Simulação e Interação	29
4.1.2.	Implementação dos Algoritmos	30
4.1.3.	Arquitetura Geral do Simulador	30
4.2.	O Ambiente de Simulação	31
4.2.1.	Modelo Físico e Hiperparâmetros	31
4.2.2.	Espaço de Observação	31
4.2.3.	Espaço de Ação e Dinâmica do Passo de Simulação	32
4.2.4.	Cenários de Teste	33
4.3.	Design da Função de Recompensa	36
4.4.	Desafios de Engenharia e Otimização Computacional	38
4.4.1.	Motor de Visualização Interativa (Dashboard)	38
4.4.2.	Otimização Matemática da Física e Detecção de Colisões	39
4.4.3.	Mecanismo de Atenção sobre Vizinhos (GNN)	39
4.4.4.	Ambientes Vetorizados e Paralelização Multiprocesso	40
4.5.	Estrutura do Projeto e Extensibilidade	40
4.5.1.	Organização dos Módulos	40
4.5.2.	Pontos de Extensão	40
4.5.3.	Garantias de Não-Regressão	41
4.5.4.	Reprodutibilidade	42
Capítulo 5.	Arquitetura Algorítmica e Controladores	43
5.1.	Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)	43
5.1.1.	Configuração PPO (<i>On-Policy</i>)	43
5.1.2.	Configuração SAC (<i>Off-Policy</i>)	44

5.1.3. Representação da Vizinhaça e <i>Parameter Sharing</i>	44
5.2. Algoritmo 2: Otimização Bio-inspirada (Benchmark)	45
5.2.1. Função de Aptidão e Operadores de Variação	45
5.2.2. Ciclo de Treino Evolutivo	46
5.2.3. Procura por Novidade (<i>Novelty Search</i>) para Cenários Enganadores (<i>Deceptive</i>)	47
5.3. Plano de Experiências e Tratamento Estatístico	48
Capítulo 6. Resultados Experimentais e Discussão	49
6.1. Metodologia de Avaliação e Notas de Leitura	49
6.2. Desempenho de Tarefa por Cenário (P_{task})	50
6.3. Análise Qualitativa da Navegação: Mapas de Calor	51
6.4. Convergência e Desempenho no Cenário Base (Sandbox)	51
6.5. Desempenho Comparativo por Cenário (Curvas de Aprendizagem)	53
6.6. Fiabilidade e Variância entre Execuções	56
6.7. Visão Global por Algoritmo	58
6.8. Avaliação Determinística (Tarefa Pura)	60
6.9. Análise de Cenários Complexos e Comportamento Emergente	61
6.10. Deceção e Procura por Novidade (<i>Novelty Search</i>)	61
6.11. Composição de Dificuldades: o Mapa Composto	67
6.11.1. Motivação e Questão	67
6.11.2. O Ambiente	67
6.11.3. Protocolo	69
6.11.4. Transferência sem Retreino (F1)	70
6.11.5. Treino Nativo (F2)	71
6.11.6. Discussão	71
6.11.7. Limitações desta Campanha	73
6.12. Análise de Escalabilidade (Zero-Shot Transfer)	76
6.13. Resiliência e Robustez a Falhas de Agentes	77
6.14. Desempenho Computacional do Simulador	78
6.14.1. Custo de Treino e de Execução por Algoritmo	80
6.15. Discussão Global e Validação da Hipótese	81
Capítulo 7. Conclusões e Trabalhos Futuros	85
7.1. Conclusões	85
7.2. Resposta às Questões de Investigação	86
7.3. Limitações do Estudo	89
7.4. Contributos	92
7.5. Trabalhos Futuros	94
Referências Bibliográficas	97
Anexo A. Declaração de Utilização de Ferramentas de Inteligência Artificial	101

Anexo B. Configurações e Hiperparâmetros	103
Anexo C. Estudos Incluídos na Revisão Sistemática	107
Anexo D. Diagrama de Classes e Excertos de Código	115
D.1. Diagrama de Classes	115
D.2. Mecanismo de Atenção sobre Vizinhos (QKV)	115
D.3. Física de Deslizamento em Muros (<i>Wall-Sliding</i>)	115
D.4. Campo Geodésico para o <i>Reward Shaping</i>	115
D.5. Recompensa de Exploração <i>Count-Based</i>	116
Anexo E. Recursos Adicionais	119

Lista de Figuras

Figura 3.1	Fluxograma PRISMA 2020 da revisão conduzida	21
Figura 4.1	Arquitetura do simulador e fluxo agente–ambiente	30
Figura 4.2	Renderizações 3D dos sete cenários de teste	35
Figura 4.3	Vista de topo (planta) dos sete cenários	36
Figura 6.1	Taxa de sucesso (P_{task}) por cenário e algoritmo	52
Figura 6.2	Chegadas por episódio, por cenário e algoritmo	52
Figura 6.3	Potencial de navegação no Muro em U: euclidiano vs. geodésico	53
Figura 6.4	Densidade de ocupação dos robôs no Muro em U (GNN, PPO e SAC)	54
Figura 6.5	Curvas de aprendizagem no cenário Sandbox	55
Figura 6.6	Curvas de aprendizagem no cenário Muro em U	55
Figura 6.7	Curvas de aprendizagem no cenário Gargalo	56
Figura 6.8	Curvas de aprendizagem no cenário Quatro Salas	56
Figura 6.9	Curvas de aprendizagem no cenário Porta Cooperativa	57
Figura 6.10	Curvas de aprendizagem no cenário Percepção Cooperativa	57
Figura 6.11	Curvas de aprendizagem no cenário Porta com Alternativa	58
Figura 6.12	Chegadas por execução: Sandbox e Muro em U	58
Figura 6.13	Chegadas por execução: Gargalo e Quatro Salas	59
Figura 6.14	Chegadas por execução: Porta Cooperativa e Percepção Cooperativa	59
Figura 6.15	Chegadas por execução: Porta com Alternativa	59
Figura 6.16	Desempenho do GNN (evolutivo) nos sete cenários	60
Figura 6.17	Desempenho do PPO nos sete cenários	60
Figura 6.18	Desempenho do SAC nos sete cenários	61
Figura 6.19	Trajetórias: contorno do Muro em U e navegação no Quatro Salas	62
Figura 6.20	Trajetórias: porta cooperativa e perseguição do alvo móvel	62
Figura 6.21	Muro em U: os quatro braços do mega-treino ($n = 28$)	66
Figura 6.22	Planta do mapa composto	68
Figura 6.23	Percursos dos agentes no mapa composto, por controlador	72
Figura 6.24	Eficiência por agente com o aumento do enxame (Sandbox), sem retreino	78

Figura 6.25	Resiliência à perda súbita de 10% dos agentes (R_{robust})	79
Figura D.1	Diagrama de classes do sistema	116

Lista de Quadros

Quadro 3.1	Síntese dos estudos mais relevantes do corpo da revisão	28
Quadro 4.1	Hiperparâmetros da simulação e do treino	31
Quadro 5.1	Plano de testes experimentais	48
Quadro 6.1	Avaliação determinística nos sete cenários	51
Quadro 6.2	Desempenho no cenário Sandbox	53
Quadro 6.3	Treino nativo no mapa composto (F2)	71
Quadro 6.4	Escalabilidade <i>Zero-Shot</i> nos sete cenários	76
Quadro 6.5	O contraste arquitetural no Sandbox	77
Quadro 6.6	Desempenho computacional do simulador	80
Quadro 6.7	Custo computacional por algoritmo	80
Quadro 6.8	Significância das diferenças entre algoritmos	82
Quadro B.1	Configuração do ambiente de simulação e da física	104
Quadro B.2	Hiperparâmetros de treino dos três algoritmos	105
Quadro C.1	Estudos incluídos na revisão sistemática	107

[Página intencionalmente deixada em branco.]

Lista de Acrónimos

AABB: Axis-Aligned Bounding Box
ACO: Ant Colony Optimization
AE: Algoritmos Evolutivos
CTDE: Centralized Training with Decentralized Execution
Dec-POMDP: Decentralized Partially Observable Markov Decision Process
DQN: Deep Q-Network
GAE: Generalized Advantage Estimation
GNN: Graph Neural Network
MADDPG: Multi-Agent Deep Deterministic Policy Gradient
MAMDP: Multi-Agent Markov Decision Process
MAPPO: Multi-Agent Proximal Policy Optimization
MARL: Multi-Agent Reinforcement Learning
MDP: Markov Decision Process
MLP: Multi-Layer Perceptron
NEAT: NeuroEvolution of Augmenting Topologies
PBRs: Potential-Based Reward Shaping
POMDP: Partially Observable Markov Decision Process
PPO: Proximal Policy Optimization
PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PSO: Particle Swarm Optimization
QI: Questão de Investigação
QKV: Query, Key, Value (projeções do mecanismo de atenção)
RL: Reinforcement Learning
RS2C: Robust and Scalable Swarm Control
SAC: Soft Actor-Critic
SI: Swarm Intelligence
SLAM: Simultaneous Localization and Mapping
SLR: Systematic Literature Review
SR: Swarm Robotics

[Página intencionalmente deixada em branco.]

CAPÍTULO 1

Introdução

1.1. Enquadramento e Motivação

A robótica de enxame (*Swarm Robotics*) estuda a coordenação de grandes grupos de robôs simples, sem controlo central, a partir de interações locais [1]. A premissa é que o comportamento útil do grupo não exige complexidade em cada membro: resulta das interações entre eles. Tegmark defende uma ideia próxima, a de que a inteligência é, no essencial, processamento de informação, em larga medida independente do substrato físico [2]. Desde os primeiros trabalhos, no início da década de 1990, a motivação do campo mantém-se: reproduzir a inteligência coletiva observada na natureza, em colónias de formigas ou em cardumes, para obter sistemas robustos, flexíveis e escaláveis [3], [4].

Estes sistemas interessam sobretudo a aplicações em ambientes incertos ou perigosos, como a vigilância de infraestruturas, a exploração espacial ou a resposta a desastres naturais [5]. Nesses cenários, a redundância do enxame é a principal vantagem: a perda de um agente não deve comprometer a missão [6].

A dificuldade está no controlo. Métodos que dependem de uma unidade central ou de um modelo global do sistema escalam mal com o número de agentes (N). Parte da dificuldade é o que se conhece como *complexidade combinatória* (*combinatorial complexity*): o espaço conjunto de estados e ações cresce exponencialmente com o número de agentes, o que torna impraticável programar à mão todas as situações [7]. Hüttenrauch et al. (2019) argumentam que, para escalar, a política de controlo tem de ser independente do número total de agentes [8].

Para desenhar estes controladores descentralizados, a literatura segue sobretudo duas abordagens, que não são as únicas mas são as que este trabalho compara:

- (1) **Otimização Bio-inspirada:** Métodos como o *Particle Swarm Optimization* (PSO) e Algoritmos Evolutivos (AE) focam-se na robustez *offline* [4]. Tratam o desenho do controlador como um problema de otimização estocástica, procurando os pesos ótimos de uma rede neuronal ou parâmetros de regras heurísticas antes da implantação real [9], [10]. Embora robustos, estes métodos tendem a ser estáticos, podendo carecer de adaptabilidade perante mudanças dinâmicas e imprevistas no ambiente [11].
- (2) **Multi-Agent Reinforcement Learning (MARL):** Focado na adaptabilidade *online*, o MARL utiliza o paradigma de aprendizagem sequencial para que os agentes descubram políticas cooperativas através da interação direta com o ambiente [12]. O problema formaliza-se como um *Decentralized Partially Observable Markov Decision Process* (Dec-POMDP), e o que o MARL promete é aprender políticas que reagem ao que cada agente observa, em vez de seguirem um plano fixo [13].

A pertinência desta investigação sustenta-se na escassez de estudos que comparem diretamente a eficiência e a robustez destas duas escolas em cenários operacionais dinâmicos, que a revisão sistemática do Capítulo 3 documenta. Majid et al. (2024) comparam as duas famílias no plano conceptual — as estratégias evolutivas tendem a produzir políticas mais robustas, mas são ainda menos eficientes em amostras do que a aprendizagem por reforço — e apontam a sua hibridização como promissora, sem as confrontar experimentalmente [7]. Esta dissertação compara as duas famílias nas mesmas condições, quanto a desempenho, escalabilidade e robustez.

1.2. Definição do Problema e Justificação da Tese

Falta na literatura uma comparação direta (*benchmarking*) do desempenho de paradigmas concorrentes em cenários de controlo de enxames que sejam, simultaneamente, complexos e dinâmicos — de que é sintoma a própria *suite* de referência do MARL não incluir qualquer *baseline* evolutiva [14]. Na literatura revista no Capítulo 3, embora existam múltiplos *frameworks* de controlo, a maioria dos estudos avalia um único paradigma, sem a validação cruzada necessária para determinar a aplicabilidade de cada método.

O *Multi-Agent Reinforcement Learning* (MARL), embora capaz de aprender comportamentos coletivos complexos, enfrenta dificuldades conhecidas. Entre estas, destaca-se a *não-estacionaridade* do ambiente: como todos os agentes aprendem e alteram as suas políticas simultaneamente, a dinâmica do sistema do ponto de vista de um agente individual muda constantemente, violando a propriedade de Markov necessária para a convergência de algoritmos tradicionais [13]. Além disso, a explosão do espaço de ações conjuntas em enxames de grande escala dificulta a estabilidade do treino e a otimização de funções de recompensa esparsas.

Por outro lado, os métodos evolutivos e bio-inspirados (como o PSO) têm demonstrado uma robustez consolidada, tratando o controlo como a exploração de uma superfície de *fitness* global [4]. Contudo, o controlador que resulta da otimização *offline* fica fixo após a implantação: não continua a adaptar-se perante mudanças estruturais imprevistas no ambiente. A isto soma-se o que Kegeleirs e Birattari (2025) designam por *Deployment Gap* (fosso de implantação) — a ausência de garantia de que um controlador desenvolvido em simulação, ou numa plataforma, mantenha a eficácia quando aplicado noutra —, a par do *reality gap*, que os autores apontam como preocupação central sobretudo na abordagem evolutiva [15].

O problema desta dissertação é, portanto, fazer essa comparação de forma sistemática, em três eixos:

- **Desempenho da Tarefa:** Medir a eficácia na concretização do objetivo coletivo — a taxa de sucesso e o número de chegadas cooperativas ao ninho — e o custo computacional que cada paradigma paga para lá chegar.
- **Escalabilidade (Zero-Shot Transfer):** Analisar como a *performance* se degrada à medida que o número de agentes (N) aumenta para além do cenário de treino. Interessa verificar se a representação por *Graph Neural Networks* (GNNs) — cujo mecanismo de invariância à cardinalidade é caracterizado por Chen et al. (2024) [16], ainda que no domínio das redes sem fios — permite uma escalabilidade superior à das políticas de dimensão de entrada fixa (MLP) usadas pelas *baselines* de gradiente. Note-se que,

neste trabalho, é o controlador *evolutivo* que instancia a arquitetura de grafo: o eixo da escalabilidade opõe, portanto, representações (grafo vs. vetor fixo) e não paradigmas de otimização.

- **Robustez a Falhas:** Testar a resiliência do sistema perante a perda súbita de agentes (falhas de *hardware*) a meio da tarefa. Em ambos os paradigmas a política fica fixa depois do treino, pelo que o que se mede é a tolerância a falhas do comportamento aprendido, e não uma adaptação durante a execução.

O objetivo prático é dar a quem desenha um enxame dados quantitativos para escolher o paradigma de controlo em função dos requisitos da missão e do cómputo disponível.

1.3. Objetivos da Tese

O objetivo geral desta dissertação é *desenvolver um simulador de enxames e comparar, nas mesmas condições, o Multi-Agent Reinforcement Learning (MARL) com a otimização bio-inspirada* (em concreto, a neuroevolução) no controlo descentralizado de enxames. Pretende-se determinar, de forma quantitativa, em que condições cada paradigma é preferível.

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- **Desenvolver um Ambiente de Simulação Abrangente:** Projetar e implementar um simulador em ambiente Python capaz de modelar tarefas fundamentais de enxame, como a navegação cooperativa até um ninho móvel e a perseguição cooperativa (cerco de alvos móveis), em cenários de dificuldade de navegação crescente. O ambiente deve contemplar colisões e deslizamento nas paredes, perseguição do ambiente por sensores de alcance limitado e a simulação de falhas súbitas de agentes para testar a resiliência do sistema [6].
- **Implementar um Framework MARL (RS2C):** Desenvolver o *framework Robust and Scalable Swarm Control* (RS2C), assente em execução descentralizada com partilha de parâmetros, sobre as implementações de referência do PPO e do SAC.
- **Implementar uma Baseline de Otimização Bio-inspirada:** Desenvolver um controlador baseado em *Neuroevolução* (um Algoritmo Evolutivo, AE) para servir de termo de comparação de desempenho [10], [17]. Este controlador é otimizado *offline*, sem gradientes, para encontrar os pesos de uma rede neuronal de grafos com atenção, cuja invariância à permutação permite aplicar um controlador treinado com N agentes a enxames de outra dimensão sem retreino (*Zero-Shot Transfer*) [16].
- **Realizar uma Análise Comparativa e Estatística:** Executar experiências controladas com as métricas de desempenho da tarefa, escalabilidade e robustez a falhas. Os resultados deverão ser validados estatisticamente — testes de hipótese não-paramétricos sobre execuções de treino independentes, complementados por medidas de tamanho de efeito —, garantindo que as conclusões sobre a superioridade de um paradigma sobre o outro são cientificamente sustentadas.

1.4. Questões de Investigação

Decorrentes da definição do problema e dos três eixos de avaliação enunciados (desempenho, escalabilidade e robustez), a investigação organiza-se em *dois níveis*. O primeiro é o da comparação entre paradigmas: quatro questões gerais, que interrogam o que cada família de métodos consegue e em que condições. O segundo desce ao nível dos *mecanismos* que produzem esse desempenho — três questões deliberadamente mais específicas, cada uma isolando um fator que a comparação de primeiro nível agrega: o desenho do sinal de treino, a pressão de exploração e a topologia do ambiente. A *hipótese central* que as atravessa é a de que a *inteligência adaptativa* (MARL) supera a *robustez estática* bio-inspirada no controlo de enxames; é avaliada na Secção 6.15, e todas as questões são retomadas explicitamente nas Conclusões (Secção 7.2).

Nível 1 — Questões gerais: a comparação entre paradigmas.

- QI1. Desempenho de tarefa:** Qual dos paradigmas de controlo descentralizado — MARL por gradiente (PPO/SAC) ou neuroevolução — atinge maior eficácia nas tarefas coletivas avaliadas — navegação cooperativa até ao ninho e perceção cooperativa de alvos móveis —, em cenários de dificuldade de navegação e de coordenação crescente?
- QI2. Escalabilidade (*Zero-Shot Transfer*):** Uma política treinada com $N = 20$ agentes transfere-se, sem retreino, para enxames de dimensão distinta ($N = 10$ a $N = 100$)? Em que medida essa capacidade depende da arquitetura da política (rede de grafos com atenção vs. MLP de dimensão fixa)?
- QI3. Robustez a falhas:** Como se degrada o desempenho coletivo de cada paradigma perante a falha súbita de uma fração dos agentes durante a execução da tarefa?
- QI4. Critério de escolha:** Em que condições operacionais (tipo de cenário, dimensão do enxame, requisitos de fiabilidade) se justifica preferir um paradigma ao outro?

Nível 2 — Questões específicas: os mecanismos por detrás do desempenho. As três questões seguintes decorrem das anteriores e só se tornam formuláveis depois delas: cada uma toma um resultado do primeiro nível e pergunta *de que* é que ele depende. São, por isso, mais estreitas — e é essa estreiteza que as torna testáveis com uma experiência controlada.

- QI5. Desenho da função de *fitness*** (aprofunda a QI1): Em que medida o desempenho do controlador neuroevolutivo depende do desenho da função de *fitness*? Em particular, pode um *shaping* de navegação (*homing*), que premeia a aproximação ao objetivo antes da primeira chegada, desbloquear cenários que uma *fitness* de tarefa pura não resolve?
- QI6. Deceção e procura por novidade** (aprofunda a QI1 e a QI5): Em cenários cujo gradiente de recompensa é enganador (*deceptive*), a hibridização da neuroevolução com procura por novidade (*Novelty Search*) melhora o resultado final face à otimização puramente objetiva?
- QI7. Composição de dificuldades** (aprofunda a QI1 e a QI4): As conclusões obtidas em sete cenários de dificuldade isolada transferem-se para um ambiente que *combina* essas dificuldades a uma escala substancialmente maior, ou a composição faz emergir modos de falha que os cenários isolados não revelam?

1.5. Estrutura do Documento

Além desta Introdução, o documento organiza-se em seis capítulos e cinco anexos:

- O **Capítulo 2** apresenta os fundamentos teóricos dos dois paradigmas comparados — os algoritmos evolutivos, os processos de decisão de Markov e os algoritmos PPO e SAC — e das redes de grafos com atenção em que assenta o controlador neuroevolutivo.
- O **Capítulo 3** reporta a revisão sistemática da literatura: o protocolo seguido, a caracterização dos estudos incluídos, o problema da transferência da simulação para robôs reais e as lacunas a que esta dissertação responde.
- O **Capítulo 4** descreve o simulador desenvolvido — arquitetura de software, modelo físico, observações e ações dos agentes, os sete cenários de teste e a função de recompensa — e a estrutura do projeto.
- O **Capítulo 5** detalha os controladores comparados (o *framework* RS2C, com PPO e SAC, e o controlador neuroevolutivo, com a sua função de aptidão e a procura por novidade) e o plano experimental e estatístico.
- O **Capítulo 6** apresenta e discute os resultados: desempenho por cenário, fiabilidade entre execuções, deceção e procura por novidade, o oitavo cenário composto, escalabilidade, robustez a falhas e custo computacional.
- O **Capítulo 7** responde às questões de investigação e declara as limitações, os contributos e os trabalhos futuros.

Os anexos reúnem a declaração de utilização de ferramentas de inteligência artificial (Anexo A), as configurações e hiperparâmetros (Anexo B), os estudos incluídos na revisão sistemática (Anexo C), o diagrama de classes e os excertos de código mais relevantes (Anexo D) e a localização do repositório com o código e os dados (Anexo E).

[Página intencionalmente deixada em branco.]

CAPÍTULO 2

Fundamentos Teóricos e Tecnologias

2.1. Introdução ao Capítulo

Este capítulo apresenta os conceitos e a notação usados no resto da dissertação: a inteligência e a robótica de enxame, os dois paradigmas de controlo comparados e as redes de grafos com atenção.

Começa-se pelos conceitos de *Swarm Intelligence* (SI) e *Swarm Robotics* (SR), que explicam como comportamentos globais podem emergir de regras locais simples. A descentralização que torna estes sistemas robustos também dificulta o desenho dos controladores: como mostram Rivière et al. (2020), sintetizar políticas que operem apenas sobre o estado relativo dos vizinhos próximos — e que ainda assim aproximem o desempenho de um planeador global — é, em si mesmo, um problema de investigação em aberto [18].

Seguem-se os dois paradigmas de controlo que esta dissertação compara:

- (1) **Otimização Bio-inspirada:** Serão explorados algoritmos como a Otimização por Enxame de Partículas (*Particle Swarm Optimization*, PSO) e os Algoritmos Evolutivos (AE). Estas técnicas tratam o controlo como um problema de busca num espaço de parâmetros global, focando-se na convergência para soluções estáveis e robustas em cenários estáticos.
- (2) **Multi-Agent Reinforcement Learning (MARL):** Será introduzida a formalização matemática do problema através do modelo de Processo de Decisão de Markov Parcialmente Observável Descentralizado (*Dec-POMDP*). Descreve-se como a aprendizagem por reforço aprende políticas por interação com o ambiente, bem como a arquitetura *Centralized Training with Decentralized Execution* (CTDE) e o papel das *Graph Neural Networks* (GNNs) na garantia da escalabilidade do enxame, tal como defendido por Chen et al. (2024).

A transferência destes controladores da simulação para robôs reais — o *Deployment Gap* de Kegeleirs e Birattari (2025) — é tratada no capítulo seguinte (Secção 3.5) [15].

2.2. Swarm Intelligence (SI) e Robótica de Enxame

A Inteligência de Enxame (*Swarm Intelligence*, SI) refere-se ao comportamento coletivo e coordenado de sistemas descentralizados e auto-organizados, sejam eles naturais ou artificiais [3]. Este paradigma baseia-se na observação de sistemas biológicos — tais como colónias de formigas, enxames de abelhas ou cardumes de peixes — onde agentes individuais com capacidades cognitivas limitadas conseguem, através de interações sociais simples, resolver problemas de elevada complexidade que excederiam a capacidade de qualquer indivíduo isolado [4].

A Robótica de Enxame (*Swarm Robotics*, SR) surge como a aplicação direta dos princípios de SI ao domínio de sistemas multi-robô [1]. Ao contrário da robótica cooperativa tradicional, a SR foca-se na coordenação de um grande número de agentes, tipicamente homogêneos e de baixo custo, que operam sem uma infraestrutura de controlo centralizada ou um mapa global do ambiente. O controlo nestes sistemas é inerentemente distribuído, o que confere ao coletivo três propriedades fundamentais identificadas na literatura:

- **Robustez:** A redundância do sistema garante que a missão prossiga mesmo perante a falha de múltiplos agentes [8].
- **Escalabilidade:** O sistema mantém a sua eficiência independentemente da dimensão do enxame (N), uma vez que as interações são predominantemente locais.
- **Flexibilidade:** O enxame consegue adaptar-se a diferentes tarefas e ambientes dinâmicos sem necessidade de reprogramação externa.

O alicerce técnico destes sistemas reside no comportamento emergente. Conforme sistematizado por Bonabeau, Dorigo e Theraulaz (1999) no estudo dos insetos sociais, as ações globais coordenadas emergem de um ciclo de *feedback* contínuo entre regras de interação locais simples e o ambiente [1]. Um mecanismo crítico para esta emergência é a *estigmergia*, um método de coordenação indireta onde os agentes alteram o ambiente (por exemplo, depositando trilhos de feromonas), influenciando as ações subsequentes de outros membros do grupo — o princípio que está na origem da própria Otimização por Colónias de Formigas [3].

No entanto, projetar manualmente estas regras locais para atingir um objetivo global específico é um desafio de engenharia não trivial. Como não existe um “conhecimento global” partilhado, o desenho do controlador deve garantir que a soma das micro-decisões individuais resulte na macro-estabilidade do enxame. É isso que motiva o recurso à aprendizagem automática: em vez de programar as regras à mão, o sistema aprende-as por reforço ou fá-las evoluir por otimização.

2.3. Paradigmas de Controlo para Enxames

O controlo de sistemas multi-robô em larga escala foca-se primordialmente no desenho das chamadas “regras locais” de interação. O desafio fundamental reside na resolução do problema inverso da robótica de enxame: como determinar comportamentos individuais simples que, quando executados em paralelo, resultem numa macro-estabilidade e no cumprimento de um objetivo global. De entre as várias famílias de métodos propostas para o efeito, destacam-se na literatura contemporânea duas abordagens dominantes [7], [15], [19] — que não esgotam o espaço de soluções, mas concentram o essencial do debate atual — e são estas que esta tese se propõe a analisar e comparar:

- (1) **Otimização Bio-inspirada (Offline):** Esta categoria abrange duas subfamílias principais frequentemente utilizadas em robótica:

- **Algoritmos Evolutivos (AE):** Inspirados na seleção natural (Darwin), utilizam operadores como mutação e cruzamento para evoluir uma população de controladores ao longo de gerações.

- **Inteligência de Enxame (SI):** Inspirados em comportamentos sociais (e.g., bandos de pássaros), como o *Particle Swarm Optimization* (PSO), onde a solução emerge do movimento cooperativo de partículas no espaço de busca.

Ambas as abordagens procuram *offline* o melhor conjunto de parâmetros fixos, maximizando uma função de *fitness*. Embora gerem comportamentos robustos, resultam frequentemente em controladores rígidos perante mudanças ambientais não previstas.

- (2) **Aprendizagem por Reforço (Online/Adaptativa):** Em vez de otimizar parâmetros fixos antes da implantação, o agente aprende uma política $\pi(a \mid o)$ através da interação sequencial com o ambiente, guiado por um sinal de recompensa. Na sua extensão multiagente (MARL), cada robô ajusta o seu comportamento em função das observações locais, permitindo em princípio a adaptação a condições não previstas no desenho. O custo desta flexibilidade é uma otimização substancialmente mais difícil: a não-estacionaridade introduzida pela aprendizagem simultânea de todos os agentes, a atribuição de crédito em recompensas coletivas e a sensibilidade ao desenho do sinal de recompensa (Secção 4.3) são desafios centrais que esta tese aborda empiricamente.

É esta distinção que a comparação explora: permite avaliar se o custo computacional e a complexidade de treino da aprendizagem automática são justificados por um ganho real em adaptabilidade face às soluções otimizadas tradicionais.

2.4. Abordagem 1: Otimização Bio-inspirada

Esta abordagem utiliza algoritmos de busca meta-heurística para navegar no vasto espaço de parâmetros θ de um controlador — frequentemente representado por uma rede neuronal (*Neuroevolution*) ou uma máquina de estados — com o intuito de maximizar uma função de desempenho global $J(\theta)$.

2.4.1. Fundamentos Matemáticos dos Algoritmos Evolutivos

O processo evolutivo opera sobre uma *população* $\Pi_g = \{\theta_1, \theta_2, \dots, \theta_K\}$ de K controladores candidatos, denominados *genomas*, onde cada genoma $\theta_k \in \mathbb{R}^{|\theta|}$ é o vetor concatenado de todos os pesos e *biases* de uma rede neuronal. A topologia arquitetural da rede é fixada *a priori*; apenas os seus parâmetros numéricos são sujeitos a otimização — esta abordagem denomina-se *Neuroevolução*. Ao contrário do RL, a Neuroevolução não necessita de gradientes calculáveis, tornando-a aplicável a qualquer função de recompensa não-diferenciável.

O ciclo evolutivo: Um algoritmo evolutivo itera três operações sobre a população: *avaliação* — atribuir a cada genoma um valor de aptidão $J(\theta_k)$, obtido executando o controlador correspondente no ambiente —, *seleção* — reter os genomas de maior aptidão — e *variação* — gerar descendentes por mutação e, nalgumas formulações, por recombinação. A geração $g + 1$ é, assim, uma amostra enviesada para as regiões de $\mathbb{R}^{|\theta|}$ que a geração g mostrou serem promissoras. Como a única informação utilizada é a *ordenação* das aptidões, o método aplica-se a funções não-diferenciáveis, descontínuas ou avaliáveis apenas por simulação — é esta propriedade que o torna atrativo em robótica de enxame, onde o desempenho coletivo raramente admite gradiente analítico [7], [19].

Função de aptidão (*fitness*): A função $J(\theta_k)$ é o único canal por onde o objetivo do projetista entra na otimização, e a literatura mostra que a sua formulação condiciona o resultado tanto quanto o algoritmo de busca. Uma aptidão que meça apenas o objetivo final tende a ser *esparsa* — nenhum genoma da população inicial se distingue dos restantes —, ao passo que uma aptidão enriquecida com termos intermédios (*shaping*) corre o risco de se tornar *enganadora* (*deceptive*), premiando comportamentos que afastam da solução. É esta tensão que motiva alternativas como a procura por novidade, que substitui parcialmente a pressão pelo objetivo por pressão pela diversidade comportamental [20], [21].

Operadores de variação: Em neuroevolução de topologia fixa, o operador canónico é a *mutação gaussiana* (*Gaussian mutation*): cada descendente resulta de perturbar o vetor de pesos do progenitor,

$$\theta^{(\text{filho})} = \theta^{(\text{pai})} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad (2.1)$$

onde $\mathcal{N}(\mu, \Sigma)$ denota a distribuição normal de média μ e covariância Σ , $\mathbf{I}$ é a matriz identidade e σ , o *passo de mutação* (*mutation step size*), fixa a escala da perturbação. Salimans et al. mostram que esta forma de busca, paralelizada por muitos trabalhadores, é competitiva com a aprendizagem por reforço em tarefas de controlo contínuo [10]. Uma variante frequente perturba apenas um subconjunto aleatório das coordenadas, para preservar diversidade em espaços de grande dimensão. Uma família distinta — *NeuroEvolution of Augmenting Topologies* (NEAT) e derivados — evolui também a *topologia* da rede, e não apenas os seus pesos, ao custo de um espaço de busca substancialmente mais complexo [22], [23].

Controlo do passo de mutação: O valor de σ arbitra entre exploração e refinamento: valores elevados amostram regiões distantes do espaço de pesos, valores baixos afinam o ótimo já encontrado. É prática corrente fazê-lo decair ao longo das gerações — por analogia com a *temperagem* (*annealing*) da metalurgia —, tipicamente de forma geométrica e com um patamar mínimo que impede o colapso da diversidade:

$$\sigma_{g+1} = \max(\sigma_{\min}, \sigma_g \cdot \rho), \quad 0 < \rho < 1 \quad (2.2)$$

onde g indexa a geração, ρ é o *fator de decaimento* e $\sigma_{\min}$ o patamar mínimo do passo de mutação.

Seleção e elitismo: A seleção por *truncatura* (*truncation selection*) ordena a população por aptidão e descarta a cauda inferior; o *elitismo* (*elitism*) copia intactos os melhores genomas para a geração seguinte, garantindo que o melhor comportamento descoberto nunca se perde por uma mutação desfavorável — propriedade que torna a melhor aptidão monótona não-decrescente ao longo das gerações. Em robótica de enxame estas escolhas convivem com uma dificuldade adicional: a aptidão é estimada por simulação estocástica, pelo que um genoma pode ser selecionado por ter tido sorte na avaliação e não por ser efetivamente melhor — razão pela qual se avalia cada genoma em vários episódios e se fixam as *seeds* de avaliação [19], [24].

Otimização por Enxame de Partículas: O PSO modela cada controlador candidato como uma “partícula” que navega num espaço de busca multidimensional. A inovação crucial deste método reside na partilha de informação social: cada partícula ajusta a sua velocidade e posição baseada

na sua melhor experiência pessoal (*pBest*) e na melhor experiência coletiva descoberta por todo o enxame (*gBest*). Hassen e Amin (2025) empregam o PSO para a otimização global do percurso, delegando a reação a obstáculos imprevistos num controlador reativo separado (*Dynamic Window Approach*) — arranjo que ilustra a característica estrutural do paradigma: a convergência do enxame produz um controlador *estático*, de elevada qualidade para o problema otimizado, mas cuja adaptação ao imprevisto tem de ser acrescentada por fora [9].

A instanciação concreta destes elementos no controlador desta dissertação — a função de aptidão adotada, a forma da mutação, os valores de σ e a dimensão da elite — é matéria do Capítulo 5 (Secção 5.2), e não deste.

2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)

O MARL aplica a aprendizagem por reforço a sistemas com vários agentes, que aprendem políticas (π) por interação sequencial com o ambiente.

2.5.1. Processos de Decisão de Markov (MDPs) e Dec-POMDP

O Processo de Decisão de Markov: A base matemática da aprendizagem por reforço é o *Processo de Decisão de Markov (MDP)*, definido pela tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- $\mathcal{S}$: espaço de estados do ambiente (contínuo em robótica);
- $\mathcal{A}$: espaço de ações — discreto ou contínuo; em robótica móvel é tipicamente um vetor de comando contínuo e limitado;
- $\mathcal{P}(s' | s, a)$: dinâmica de transição — probabilidade de atingir s' ao executar a em s ;
- $\mathcal{R}(s, a, s') \in \mathbb{R}$: sinal de recompensa escalar;
- $\gamma \in [0, 1)$: fator de desconto temporal, que pondera as recompensas futuras face às imediatas.

A instanciação concreta desta tupla no ambiente desta dissertação — a forma e a escala do vetor de ação, o sinal de recompensa e os valores dos hiperparâmetros — é matéria do Capítulo 4 e da Secção 5.1, e não deste.

Funções de Valor e Equação de Bellman: O objetivo do agente é encontrar uma política $\pi^*(a | s)$ que maximize o *retorno esperado descontado* (*expected discounted return*) $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, em que r_t é a recompensa escalar recebida no instante t . A qualidade de uma política é quantificada por duas funções fundamentais.

A *Função de Valor de Estado* (*State-Value Function*) $V^\pi(s)$ mede o retorno esperado ao seguir π a partir de s :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (2.3)$$

A *Função Q* (ou *Função de Valor de Ação*, *Action-Value Function*) $Q^\pi(s, a)$ mede o retorno esperado ao executar a em s e depois seguir π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (2.4)$$

Estas funções satisfazem a *Equação de Bellman*, que permite a sua estimação recursiva:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} \mathcal{P}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.5)$$

A política ótima verifica a *Equação de Bellman de Optimalidade (Bellman Optimality Equation)*:

$$Q^*(s, a) = \mathbb{E}_{s'} \left[\mathcal{R}(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.6)$$

Extensão para Dec-POMDP em Enxames: Em sistemas multiagente, o estado global $s \in \mathcal{S}$ é inacessível a qualquer robô individual. O problema estende-se para o *Processo de Decisão de Markov Parcialmente Observável Descentralizado (Dec-POMDP)*, definido pela tupla $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R}, \Omega, \{O_i\}_{i \in \mathcal{N}}, \gamma \rangle$, onde $\mathcal{N} = \{1, \dots, N\}$ é o conjunto de agentes, Ω é o espaço de observações conjuntas, e $o_i = O_i(s) \in \mathbb{R}^{111}$ é a observação local e parcial do agente i (o caligráfico $\mathcal{N}$ designa aqui o conjunto de agentes; a distribuição normal escreve-se com os seus parâmetros, $\mathcal{N}(\mu, \Sigma)$) [12], [13], [25]. Cada robô mantém uma política descentralizada $\pi_i(a_i | o_i)$ e o objetivo coletivo é maximizar o retorno conjunto:

$$J(\pi_1, \dots, \pi_N) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, \mathbf{a}_t) \right], \quad \mathbf{a}_t = (a_1^t, \dots, a_N^t) \quad (2.7)$$

2.5.2. Otimização de Políticas: Do Actor-Critic ao PPO e SAC

Arquitetura Actor-Critic: Tanto o PPO como o SAC assentam na arquitetura *Actor-Critic*, onde duas redes são treinadas em conjunto:

- **Ator** (*Actor*, π_θ): mapeia a observação o_i para uma distribuição sobre ações. Em execução descentralizada, *apenas o ator é necessário* a bordo de cada robô.
- **Crítico** (*Critic*, V_ϕ ou Q_ψ): rede auxiliar que estima o retorno esperado, utilizada *exclusivamente durante o treino* para calcular gradientes de atualização do ator.

PPO — Clipped Surrogate Objective: O PPO (*Proximal Policy Optimization*) [26] é um algoritmo *on-policy*: recolhe *rollouts* com a política atual e atualiza os parâmetros através de um objetivo *clipped* que previne passos de gradiente destabilizadores. Seja a *razão de importância (importance ratio)*:

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} \quad (2.8)$$

Antes de enunciar o objetivo, fixam-se os símbolos envolvidos:

- $r_t(\theta)$: razão de importância entre a política atual e a política que recolheu os dados (Equação 2.8), igual a 1 quando ambas atribuem a mesma probabilidade à ação observada;
- $\hat{A}_t$: estimativa da vantagem da ação tomada em t face ao valor esperado do estado — positiva se a ação superou a expectativa do crítico;
- ε : amplitude da região de confiança do *clipping*, que limita o quanto uma única atualização pode afastar a política nova da anterior;
- $\text{clip}(x, a, b)$: truncatura de x ao intervalo $[a, b]$;
- $\mathbb{E}_t[\cdot]$: média empírica sobre o lote de transições recolhidas.

O objetivo PPO clipped é:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right] \quad (2.9)$$

com $\varepsilon = 0,2$. A estimativa da vantagem $\hat{A}_t$ é calculada via *Estimação de Vantagem Generalizada (GAE)* [27]:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.10)$$

onde δ_t é o *erro de diferença temporal (TD error)* no instante t ; r_t é a recompensa recebida nesse instante (sem argumento — não confundir com a razão de importância $r_t(\theta)$ da Equação 2.8); V_ϕ é o crítico, uma rede de parâmetros ϕ que aproxima a função de valor de estado V^π da Equação 2.3; γ é o fator de desconto do MDP (Secção 2.5.1); $\lambda \in [0, 1]$ é o parâmetro do GAE, que arbitra entre enviesamento ($\lambda = 0$ usa um único passo) e variância ($\lambda = 1$ usa o retorno completo); e T é o comprimento do *rollout*. Escreve-se s_t por fidelidade à formulação original; na implementação desta dissertação o crítico recebe a observação local o_t (Secção 7.3).

O objetivo final inclui ainda um erro de regressão do crítico e um bônus de entropia:

$$L^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \mathbb{E}_t \left[(V_\phi(s_t) - V_t^{\text{targ}})^2 \right] + c_2 \mathbb{E}_t [H(\pi_\theta(\cdot | o_t))] \quad (2.11)$$

onde c_1 e c_2 ponderam, respetivamente, o erro de regressão do crítico e o bônus de entropia — $H(\pi_\theta(\cdot | o_t)) = -\mathbb{E}_{a \sim \pi_\theta} [\log \pi_\theta(a | o_t)]$ é a entropia de Shannon da distribuição de ações, que premeia políticas menos deterministas —, e V_t^{targ} é o retorno-alvo estimado para o estado s_t .

SAC — Maximização de Entropia: O SAC (*Soft Actor-Critic*) [28] é um algoritmo *off-policy* que acrescenta ao objetivo de RL o termo de entropia de Shannon $H(\pi) = -\mathbb{E}[\log \pi(a | s)]$, promovendo a *exploração máxima (maximum entropy exploration)* consistente com o desempenho na tarefa:

$$J(\pi) = \mathbb{E}_{\rho_\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \alpha H(\pi(\cdot | s_t)) \right) \right] \quad (2.12)$$

onde ρ_π denota a distribuição dos pares estado-ação visitados ao seguir π e $\alpha > 0$ é o *coeficiente de temperatura (temperature coefficient)*, que controla o equilíbrio entre exploração e recompensa; o seu valor — e a opção de o fixar ou de o adaptar automaticamente — é matéria da Secção 5.1. A *Função de Valor Suave (Soft Value Function)* incorpora diretamente o termo de entropia:

$$V^*(s) = \mathbb{E}_{a \sim \pi} [Q^*(s, a) - \alpha \log \pi(a | s)] \quad (2.13)$$

onde o asterisco denota, como na Equação 2.6, as funções associadas à política ótima. O ator é atualizado minimizando a divergência KL relativamente à distribuição de Boltzmann $\propto \exp(Q/\alpha)$:

$$J_\pi(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}} \left[\mathbb{E}_{a_t \sim \pi_\theta} \left[\alpha \log \pi_\theta(a_t | o_t) - Q_\psi(s_t, a_t) \right] \right] \quad (2.14)$$

onde θ são, como no PPO, os parâmetros do ator, Q_ψ é o crítico de parâmetros ψ e $\mathcal{D}$ é o *replay buffer* descrito a seguir. O SAC mantém dois críticos Q_{ψ_1}, Q_{ψ_2} cujo mínimo é usado no alvo (*double Q-trick*), reduzindo o viés de sobre-estimação. As transições (s_t, a_t, r_t, s_{t+1}) são

armazenadas num *Replay Buffer* de capacidade finita $|\mathcal{D}|$, permitindo a reutilização *off-policy* de experiências passadas.

O Desafio da Não-Estacionaridade: A principal barreira teórica na aplicação de MARL a enxames é a não-estacionaridade do ambiente. Num sistema multiagente, as transições de estado para um agente i dependem das ações de todos os outros agentes. Como todos os robôs estão a aprender e a alterar as suas políticas simultaneamente, a dinâmica do ambiente é instável, o que viola a propriedade fundamental de Markov e dificulta a convergência dos algoritmos tradicionais de agente único.

Arquitetura CTDE e Escalabilidade com GNNs: Para mitigar a instabilidade, a literatura recorre frequentemente à arquitetura de *Treino Centralizado com Execução Descentralizada* (CTDE) [29]. Nesta abordagem, um componente “Crítico” tem acesso ao estado global durante a fase de treino para estabilizar as estimativas de valor, enquanto o “Ator” (controlador do robô) utiliza apenas observações locais durante a fase de execução [16]. Importa notar que a implementação desta tese adota uma variante mais estrita — treino e execução descentralizados, com o crítico a partir da mesma observação local do ator (Secção 7.3) —, pelo que o CTDE é aqui apresentado como contexto da literatura, e não como a arquitetura utilizada.

Para que a política escale com o número de agentes, a literatura recente recorre a *Graph Neural Networks* (GNNs) e a mecanismos de atenção.

2.5.3. Fundamentos de Graph Neural Networks e Mecanismos de Atenção

Representação em Grafo do Enxame: O enxame de N robôs é modelado como um grafo dinâmico $G_t = (\mathcal{V}_t, \mathcal{E}_t)$, onde $\mathcal{V}_t = \{1, \dots, N\}$ é o conjunto de nós (robôs) e $\mathcal{E}_t$ o de arestas. Em vez de um corte rígido por raio de comunicação, adota-se um *grafo completamente ligado* (*fully connected graph*, $\mathcal{E}_t = \{(i, j) : i \neq j\}$) cujas arestas são ponderadas dinamicamente pelos coeficientes de atenção α_{ij} (Equação 2.18). A topologia efetiva é assim aprendida e reflete, a cada passo, a relevância relativa de cada vizinho para a decisão do agente — arestas irrelevantes recebem peso próximo de zero, emulando o efeito de um corte de proximidade sem o impor *a priori*.

Propagação de Mensagens (*Message Passing*): O paradigma geral das GNNs é o framework MPNN (*Message Passing Neural Network*). A representação $\mathbf{h}_i^{(l)}$ do nó i na camada l é atualizada por:

$$\mathbf{h}_i^{(l+1)} = \text{UPDATE}\left(\mathbf{h}_i^{(l)}, \text{AGG}\left(\left\{\mathbf{m}_{j \rightarrow i}^{(l)} : j \in \mathcal{N}(i)\right\}\right)\right) \quad (2.15)$$

onde $\mathcal{N}(i) = \{j : (j, i) \in \mathcal{E}_t\}$ é o conjunto de vizinhos do nó i , $\mathbf{m}_{j \rightarrow i}^{(l)} = \text{MSG}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)})$ é a mensagem do vizinho j , $\text{AGG}(\cdot)$ é uma função invariante à permutação (soma, média ou máximo) e $\text{UPDATE}(\cdot)$ é a função treinável que combina a representação atual do nó com a agregação das mensagens. Esta invariância é fundamental: a operação é independente da ordem dos vizinhos, garantindo que o enxame é tratado como um conjunto não-ordenado.

Mecanismo de Atenção Escalada: Entre as formulações de atenção sobre grafos distinguem-se a formulação aditiva original das *Graph Attention Networks* [30] e a *atenção de produto interno escalado* (*scaled dot-product attention*) [31], popularizada pelos *Transformers*. É esta segunda

que se formaliza em seguida, por ser a adotada no controlador desta dissertação (Secção 5.2). Para o agente i , codificam-se as suas características locais $\mathbf{h}_i^{env} = f_{enc}(\mathbf{o}_i^{env}) \in \mathbb{R}^d$ e as dos $N - 1$ vizinhos $\mathbf{h}_j^{nbr} = f_{nbr}(\mathbf{n}_{j,i}) \in \mathbb{R}^d$, onde:

- $\mathbf{o}_i^{env}$: parte *não-relacional* da observação do agente i — tudo o que descreve o ambiente e o objetivo (bússola de *homing*, telemetria de proximidade e sub-objetivo do cenário) e não os companheiros de enxame;
- $\mathbf{n}_{j,i}$: descritor do vizinho j tal como observado por i (direção relativa, distância normalizada e sinal de comunicação), sempre expresso no referencial egocêntrico de i ;
- f_{enc} e f_{nbr} : codificadores treináveis (perceptrões multicamada com parâmetros distintos) que projetam observações de dimensões diferentes num mesmo espaço latente;
- d : dimensão desse espaço latente, comum aos dois codificadores — é o que permite tratar o agente e os vizinhos como nós do mesmo grafo.

A instanciação concreta destes vetores no simulador desta dissertação (dimensões e conteúdo exatos de $\mathbf{o}_i^{env}$ e $\mathbf{n}_{j,i}$) é dada na Secção 4.2.2. Os embeddings formam a matriz de contexto:

$$\mathbf{H} = [\mathbf{h}_i^{env}; \mathbf{h}_1^{nbr}; \dots; \mathbf{h}_{N-1}^{nbr}] \in \mathbb{R}^{N \times d} \quad (2.16)$$

As projeções de consulta, chave e valor são calculadas pelas matrizes treináveis $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V \in \mathbb{R}^{d \times d}$:

$$\mathbf{Q} = \mathbf{h}_i^{env} \mathbf{W}^Q \in \mathbb{R}^{1 \times d}, \quad \mathbf{K} = \mathbf{H} \mathbf{W}^K \in \mathbb{R}^{N \times d}, \quad \mathbf{V} = \mathbf{H} \mathbf{W}^V \in \mathbb{R}^{N \times d} \quad (2.17)$$

Os *coeficientes de atenção* (*attention coefficients*) α_{ij} medem a importância que o agente i atribui ao vizinho j :

$$\alpha_i = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{1 \times N} \quad (2.18)$$

A divisão por $\sqrt{d}$ previne a saturação do *softmax* em espaços de alta dimensão. A *agregação de mensagens* (*message pooling*) é:

$$\mathbf{m}_i = \alpha_i \mathbf{V} = \sum_{j=0}^{N-1} \alpha_{ij} \mathbf{h}_j \mathbf{W}^V \in \mathbb{R}^d \quad (2.19)$$

onde $\mathbf{h}_j$ é a linha j de $\mathbf{H}$ — $\mathbf{h}_0 = \mathbf{h}_i^{env}$ e $\mathbf{h}_j = \mathbf{h}_j^{nbr}$ para $j \geq 1$ —, pelo que o próprio agente também entra na agregação. A representação final concatena o vetor ego com a agregação de mensagens e gera a ação contínua:

$$\mathbf{z}_i = [\mathbf{h}_i^{env}; \mathbf{m}_i] \in \mathbb{R}^{2d}, \quad \mathbf{a}_i = f_{actor}(\mathbf{z}_i) \in [-1, 1]^3 \quad (2.20)$$

onde f_{actor} é um perceptrão multicamada com ativação tanh à saída, que limita cada componente da ação ao intervalo $[-1, 1]$ (Secção 4.2.3).

Ao modelar o enxame como um grafo dinâmico $G = (\mathcal{V}, \mathcal{E})$, onde os nós são robôs e as arestas são as interações entre eles, é possível aprender políticas invariantes ao número de agentes. A razão é estrutural: como Chen et al. (2024) demonstram — em redes multi-salto sem fios, e não em robótica, mas o argumento é o mesmo — o número de parâmetros treináveis

de uma política sobre grafo é *independente do número de agentes*, pelo que a política aprendida se aplica a grafos maiores sem retreino [16]. Em robótica, Zhang et al. (2022) confirmam empiricamente esta invariância na exploração multi-robô [32]. É esta propriedade — o *Zero-Shot Transfer* — que esta dissertação testa em enxames de $N = 10$ a $N = 100$, confrontando-a com as limitações de rigidez das abordagens de dimensão fixa.

CAPÍTULO 3

Estado da Arte

3.1. Introdução ao Capítulo

Este capítulo apresenta uma Revisão Sistemática da Literatura (SLR) sobre os paradigmas de controlo de enxames descritos no capítulo anterior, conduzida segundo o protocolo PRISMA para ser reproduzível. Mais do que resumir o que existe, a revisão procura saber se o *Multi-Agent Reinforcement Learning* (MARL) e a otimização bio-inspirada (PSO, algoritmos evolutivos) têm sido comparados entre si no controlo descentralizado de enxames.

A motivação é a impressão, comum na área, de que as duas comunidades trabalham em silos: a literatura de MARL foca-se predominantemente na estabilidade do treino sob condições de não-estacionaridade, enquanto a literatura de otimização prioriza a robustez de controladores estáticos. Esta divisão tem dificultado a realização de confrontos diretos que validem qual o paradigma mais resiliente perante o chamado “Deployment Gap” (fosso de implantação). A falta de um referencial comum de desempenho impede, assim, uma compreensão clara de quando a adaptabilidade *online* compensa a complexidade computacional do treino em larga escala.

A revisão procura, por isso, as lacunas de *benchmarking* a que esta dissertação responde. Serão examinadas as tendências de escalabilidade por via de *Graph Neural Networks* (GNNs), o desenho automático de controladores de enxame do lado bio-inspirado, e as abordagens de hibridização — tanto as que acoplam MARL a pesquisa heurística, como o *Large Neighborhood Search* no planeamento de trajetórias multiagente [33], quanto as que combinam efetivamente aprendizagem por reforço com evolução colaborativa de populações [34]. O capítulo termina com as lacunas encontradas e com a forma como a metodologia do Capítulo 4 lhes responde.

3.2. Metodologia da Revisão Sistemática

Para garantir o rigor científico, a reprodutibilidade e a transparência no processo de seleção da informação, adotou-se uma metodologia baseada no protocolo PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*, versão 2020). O protocolo foi definido antes de executar qualquer pesquisa e está integralmente disponível no repositório da dissertação (docs/PROTOCOLO_SLR.md, Anexo E), a par do registo completo da triagem; qualquer leitor pode, com esses ficheiros, reproduzir as pesquisas e reconstruir cada uma das decisões de inclusão e exclusão aqui reportadas.

3.2.1. Pergunta e Critérios de Elegibilidade

A revisão responde à pergunta que motiva a dissertação: *como se comparam, no controlo descentralizado de enxames robóticos, os paradigmas de MARL e de otimização bio-inspirada*

quanto a desempenho de tarefa, escalabilidade e robustez — e que lacunas de benchmarking comparativo persistem?

Foram incluídos estudos que, cumulativamente: (I1) abordam o controlo, navegação ou coordenação de *sistemas multi-robô* (dois ou mais agentes, excluindo o robô isolado); (I2) aplicam *MARL/RL profundo* ou *otimização bio-inspirada* (PSO, ACO, algoritmos evolutivos, neuroevolução); (I3) apresentam *validação empírica quantitativa*; (I4) foram publicados entre 2020 e 2026 com revisão por pares; e (I5) têm texto integral acessível. Os critérios de exclusão, aplicados com registo do motivo em cada registo, foram: robô único (E1), ausência de validação experimental — incluindo revisões e artigos conceptuais (E2), ausência de dados quantitativos (E3), falta de revisão por pares ou integridade da publicação em causa (E4), duplicação (E5), método fora do âmbito — multi-robô mas controlado por teoria de jogos, *Dynamic Window Approach* ou campos potenciais clássicos, isto é, nem MARL nem bio-inspirado (E6), e tarefa fora do âmbito — alocação e escalonamento industrial, SLAM, comunicações ou manipulação (E7), uma vez que a revisão incide sobre *navegação, exploração, foraging, formação e coordenação de movimento*.

3.2.2. Protocolo de Pesquisa e Bases de Dados

As pesquisas foram executadas a 13 de julho de 2026 sobre o *Scopus* e o *IEEE Xplore*, com acesso institucional. A ACM Digital Library foi dispensada por decisão explícita: o *Scopus* indexa a maior parte das suas atas relevantes para esta pergunta e a sobreposição observada entre as duas bases utilizadas já era substancial (203 duplicados em 883 registos, $\approx 23\%$), pelo que o retorno marginal de uma terceira base seria reduzido. O Google Scholar não foi usado como fonte primária por não ser reproduzível — os seus resultados variam entre utilizadores e sessões — nem exportável em bloco.

A *string* de pesquisa exige o conceito de enxame no *título* e o paradigma de controlo em *título*, *resumo* ou *palavras-chave*:

```
TITLE ( "swarm robot*"OR "robot swarm*"OR "multi-robot")
AND TITLE-ABS-KEY ( "reinforcement learning"OR "MARL"OR "neuroevolution"
                    OR "evolutionary algorithm"OR "particle swarm"
                    OR "bio-inspired optimization")
AND TITLE-ABS-KEY ( control OR navigation OR coordination OR foraging
                    OR "path planning") AND PUBYEAR > 2019
```

A calibração desta *string* é ela própria um resultado metodológico que importa reportar. Numa primeira versão, o conceito de enxame era procurado também no *resumo* e nas *palavras-chave*, o que devolveu 7 628 registos: a expressão “*multi-agent system*” num *resumo* captura toda a literatura de sistemas multiagente sem componente robótica. Restringir o conceito de enxame ao *título* — decisão de âmbito, e não de conveniência — concentra a pesquisa nos trabalhos cujo *objeto de estudo* é o coletivo de robôs, devolvendo 456 registos no *Scopus*. Ambas as contagens ficam registadas no repositório, pela mesma razão que se regista tudo o resto: uma revisão só é sistemática se as suas escolhas forem auditáveis.

3.2.3. Processo de Seleção (Fluxo PRISMA)

As duas pesquisas devolveram 883 *registos* (456 no Scopus, 427 no IEEE Xplore). A deduplicação automática por DOI e por título normalizado removeu 203 *duplicados*, restando 680 *registos únicos*.

Na *triagem* por título e resumo foram excluídos 201 *registos*. O motivo dominante foi, com igual peso (75 *registos* cada), o robô único (E1) e o método fora do âmbito (E6). O primeiro é um artefacto instrutivo da própria *string*: a expressão *particle swarm* designa um *método* de otimização, pelo que a pesquisa recolhe abundante literatura de PSO aplicado a um único manipulador ou a um robô móvel isolado — trabalhos legítimos, mas sem enxame. Excluíram-se ainda 42 revisões e artigos conceptuais (E2), quatro *registos* por integridade ou ausência de revisão por pares (E4) — entre os quais um artigo formalmente retirado pelo IEEE por violação dos princípios de publicação —, quatro duplicados que a detecção automática não apanhara (E5), por terem DOI numa base e não na outra, e um *registo* sem dados quantitativos (E3).

Os restantes 479 *estudos* passaram à fase de *elegibilidade*. Este número, incomportável para leitura integral no âmbito de uma dissertação individual, é em si mesmo um dado sobre a maturidade do campo. Aplicaram-se então dois critérios adicionais, ambos registados no protocolo: a exclusão de tarefas fora do âmbito da pergunta (E7: alocação e escalonamento, SLAM, comunicações, manipulação, plataformas de simulação), que retirou 79 *estudos*; e a restrição do corpo a trabalhos que tratem efetivamente de *enxames* e estejam publicados em *venues de referência* — ou que, independentemente do *venue*, constituam o *núcleo* da pergunta desta revisão, isto é, que apliquem *novelty search*, neuroevolução ou *foraging* evolutivo a *enxames*, ou que comparem *diretamente* os dois paradigmas (E8, 342 *estudos* excluídos).

A lista de *venues* de referência cobre deliberadamente *ambos* os campos em confronto: robótica (IEEE T-RO, RA-L, ICRA, IROS, *Autonomous Robots*), MARL e IA (AAMAS, IEEE T-Cybernetics, TNNLS) e computação evolutiva e inteligência de enxame (GECCO, IEEE CEC, *Swarm Intelligence*, *Swarm and Evolutionary Computation*). Esta simetria não é acessória: uma lista restrita aos *venues* de robótica e MARL — a tentação natural de quem parte da literatura de aprendizagem — teria excluído sistematicamente a produção de qualidade do paradigma bio-inspirado, enviesando a revisão *contra* um dos dois objetos que ela existe para comparar.

O processo fixou 58 *estudos* para análise qualitativa e extração de dados (Figura 3.1; a lista completa consta do Anexo C).

3.2.4. Fronteira entre o Corpo da Revisão e as Fontes de Fundamentação

Nem todas as referências citadas nesta dissertação provêm do corpo da revisão, e a distinção deve ser explícita para que o leitor não procure reconciliar o *n* do fluxograma com o número de entradas na bibliografia. Distinguem-se três categorias. O *corpo da revisão* (os 58 *estudos* do Anexo C) responde à pergunta da Secção 3.2 e é o único que sustenta as contagens aqui reportadas. As *fontes primárias dos métodos* utilizados — PPO [26], SAC [28], neuroevolução [10], [17], redes de atenção sobre grafo [30], procura por novidade [20], Dec-POMDP [25] — descrevem as ferramentas empregues e são citadas nos Capítulos 2 e 5, não por resultarem desta pesquisa. Finalmente, as *revisões e artigos de enquadramento* [7], [14], [15], excluídos

do corpo por não apresentarem validação experimental própria (critério E2), são usados para situar o problema — e um deles, precisamente por catalogar apenas *baselines* de aprendizagem, serve de evidência do isolamento metodológico que aqui se documenta [14].

Um caso merece nota, por ser instrutivo quanto aos limites da própria *string* de pesquisa. O trabalho de Chen et al. (2024), que fundamenta a escolha arquitetural desta tese, não integra o corpo por não ser robótico: caracteriza a invariância à dimensão do grafo em *redes multi-salto sem fios* [16]. É citado pelo mecanismo que estabelece — o número de parâmetros de uma política sobre grafo não depende do número de agentes — e não por um resultado em enxames; a evidência robótica equivalente vem de um trabalho recuperado pela pesquisa mas excluído na elegibilidade (E8) [32]. Do mesmo modo, a pesquisa não recupera trabalhos cujo título use *swarm navigation* sem o termo *robot* [35], uma consequência assumida da decisão de exigir o conceito de enxame no título.

3.2.5. Ameaças à Validade da Revisão

Duas limitações devem ser declaradas. Primeiro, a revisão foi conduzida por um *único revisor*, sem dupla triagem independente, o que não permite calcular concordância entre avaliadores — uma restrição inerente a uma dissertação individual; em contrapartida, o registro integral das decisões de triagem, com o motivo de cada exclusão, está publicado no repositório e permite a auditoria de qualquer uma delas. Segundo, a restrição a duas bases, a publicações em inglês e ao recorte temporal 2020–2026 introduz um possível viés de cobertura, deliberado quanto ao recorte (o foco é o estado da arte posterior à adoção de GNNs) mas assumido quanto às bases.

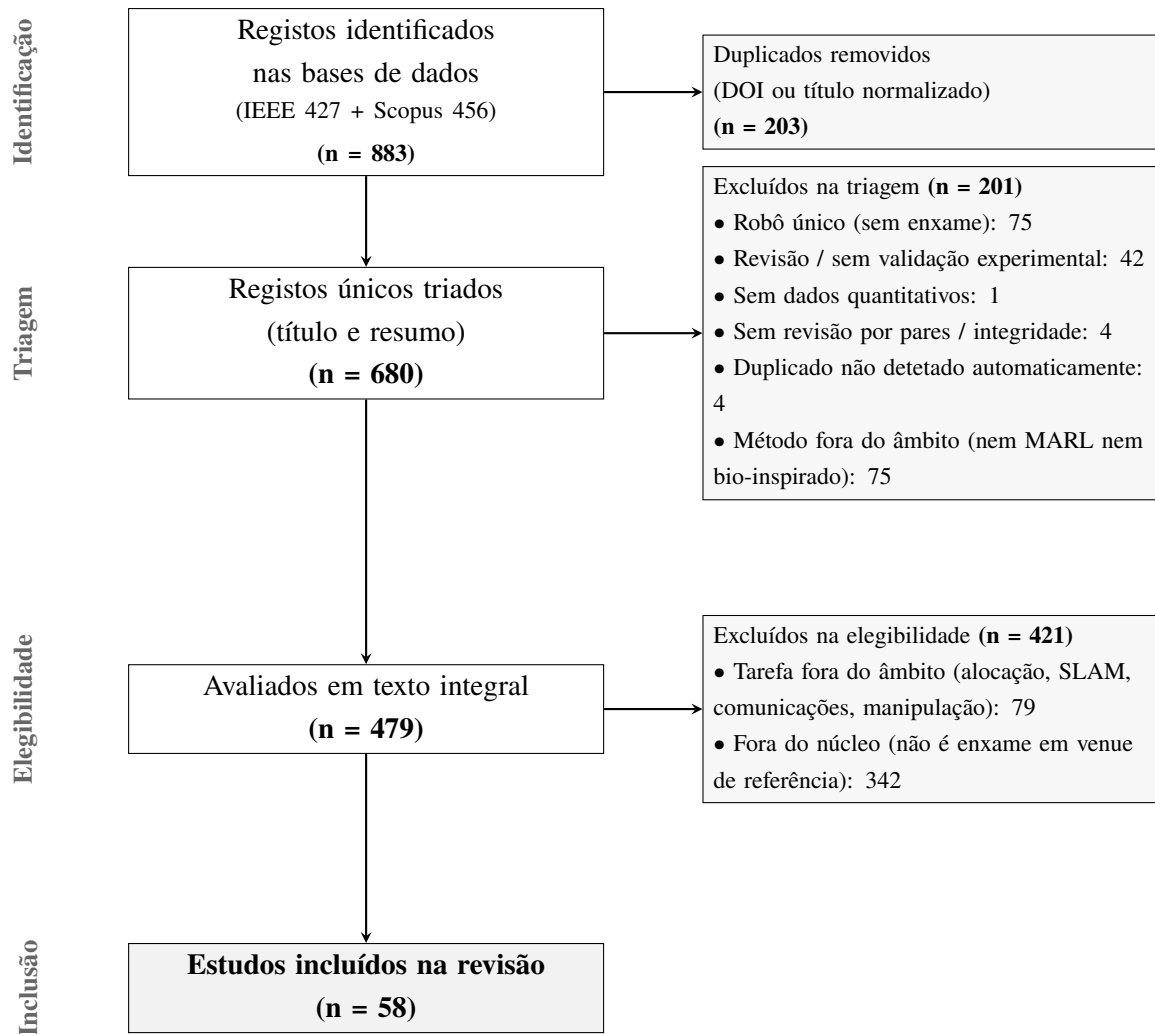


FIGURA 3.1. Fluxograma PRISMA 2020 da revisão conduzida. Os números são gerados automaticamente a partir do registo de triagem (docs/slir/screening.csv no repositório, Anexo E), onde cada decisão está associada ao respetivo critério de exclusão.

3.3. Caracterização do Corpo de Literatura

Antes de analisar os subgrupos, importa caracterizar quantitativamente os 58 estudos incluídos, pois a sua distribuição é, por si só, o argumento central deste capítulo.

Classificando cada estudo pelo paradigma de controlo que emprega, obtém-se um equilíbrio quase perfeito: *21 estudos assentam exclusivamente em MARL* (aprendizagem por reforço profunda, nas suas variantes MADDPG, MAPPO, PPO, DQN ou actor-critic), *23 assentam exclusivamente em otimização bio-inspirada* (PSO, colónias de formigas, algoritmos evolutivos, neuroevolução) e apenas *4 combinam explicitamente os dois* numa arquitetura híbrida; os restantes recorrem a formas de aprendizagem que não se enquadram em nenhuma das duas famílias (aprendizagem por imitação, por demonstração, ou modelos de campo médio). Quanto aos eixos de avaliação, *30 dos 58* abordam a escalabilidade ou a generalização a dimensões de enxame não vistas, e *14* testam explicitamente robustez a falhas, ruído ou ataques.

O dado decisivo, porém, é outro. Dos 58 estudos, *apenas dois* colocam os dois paradigmas em confronto direto — e um deles fá-lo por hibridização (combinando colónias de formigas com *Q-learning*), não por comparação. Existe, portanto, na literatura recolhida, *um único estudo que compara empiricamente MARL com otimização bio-inspirada na navegação de enxames* [36], e fá-lo numa única tarefa, sem avaliar a transferência para dimensões de enxame não vistas (*Zero-Shot*) nem a robustez a falhas.

Esta assimetria — dois campos igualmente produtivos (21 contra 23 estudos), que quase nunca se medem um contra o outro — é a lacuna que a presente dissertação ocupa, e deixa de ser aqui uma afirmação de princípio para passar a ser uma contagem verificável no registo de triagem publicado (Anexo C).

3.4. Análise dos Melhores Resultados

3.4.1. Subgrupo A: Otimização Bio-inspirada e Desenho Automático de Enxames

O trabalho mais consequente deste subgrupo provém da linha de *desenho automático* de enxames desenvolvida no IRIDIA. Hasselmann, Ligtot e Birattari (2023) geram, por *novelty search*, um repertório de comportamentos que é depois montado modularmente em controladores de enxame, mostrando que a procura por novidade — ao abandonar o objetivo explícito — produz repertórios comportamentais que um objetivo único não descobriria [21]. A mesma equipa demonstra que o desenho automático *off-line* transfere para plataformas físicas distintas daquela para que foi otimizado [37] e, num desenvolvimento notável, que missões compostas podem ser especificadas por *demonstração* em vez de por função-objetivo, recorrendo a aprendizagem por reforço inversa [38]. A relevância direta para esta dissertação é considerável: confirma que a procura por novidade é um mecanismo estabelecido no desenho de enxames (Secção 6.10), e que a dificuldade de *especificar a função de aptidão* — que aqui se revelou determinante (Q15) — é reconhecida na literatura ao ponto de motivar métodos que a dispensam.

Na tarefa de *foraging* — próxima da desta dissertação, que dela retém a ida cooperativa ao ninho mas não a recolha de objetos —, o subgrupo é ativo e recente: aplicam-se *NEAT* para evoluir estratégias adaptativas de recolha em ambientes com obstáculos [22], [23] e otimização multiobjetivo para arbitrar entre recolha e consumo energético [39]. Uma linha complementar — a *aprendizagem social* — trata a evolução como um processo *online* e distribuído, em que os controladores se propagam entre robôs durante a própria missão, sob restrições realistas de largura de banda [19], [24].

3.4.2. Subgrupo B: MARL, Escalabilidade e Arquiteturas de Grafo

No subgrupo de aprendizagem, a preocupação dominante é a mesma que motiva a escolha arquitetural desta tese: como manter uma política válida quando a dimensão do enxame muda. As respostas convergem para duas famílias. A primeira substitui a política de entrada fixa por *redes neuronais de grafos*: Zhang et al. (2022) propõem GNNs hierárquicas por saltos para exploração multi-robô, obtendo políticas que operam sobre vizinhanças de dimensão variável [32]. O mecanismo subjacente é geral e foi caracterizado com clareza fora da robótica, em redes multi-salto sem fios, por Chen et al. (2024): numa política sobre grafo, o número de parâmetros

treináveis não depende do número de agentes, pelo que a política transfere para grafos maiores sem retreino [16] — é esta a propriedade que o controlador de grafo desta dissertação explora, e que a avaliação *Zero-Shot* do Capítulo 6 testa em enxames de $N = 10$ a $N = 100$. A segunda recorre a aproximações de *campo médio*, em que o agente reage à distribuição estatística dos vizinhos em vez de a cada um individualmente, tornando o custo de coordenação independente de N [40]. Li et al. (2024) atacam o problema pelo lado da tarefa, mantendo a formação de padrões coerente perante *variação da escala do enxame* [41].

Mais próximo da formulação adotada nesta dissertação, Yigit et al. (2026) propõem um sistema de navegação de enxames com execução descentralizada, modelado como um *Multi-Agent Markov Decision Process* (MAMDP) e treinado com *PPO* sob *parameter sharing* — uma política única, partilhada por todos os agentes, que decidem a partir da observação local e sem comunicação explícita [35]. Os autores reportam convergência rápida e elevadas taxas de sucesso numa arena bidimensional com obstáculos circulares imóveis, recorrendo a uma função de recompensa multicomponente (progresso em direção ao alvo, penalização de colisões, bónus de sucesso coletivo e amortecimento de velocidade junto ao objetivo) da mesma família da aqui empregue. O trabalho valida, assim, o paradigma PPO descentralizado adotado nesta dissertação como *baseline*, e as suas limitações delimitam com precisão o espaço que esta tese ocupa.

Três dessas limitações são pertinentes. Primeira, a *representação da vizinhança*: a observação de cada agente resume-se a oito componentes — a sua posição, a do objetivo, e as posições relativas do vizinho *mais próximo* e do obstáculo *mais próximo*. O coletivo é, pois, percecionado através de um único vizinho, o que impede o agente de captar a estrutura do enxame (densidade, distribuição, formação) e contrasta com a atenção sobre todos os vizinhos empregue no controlador de grafo aqui proposto. Segunda, a *escalabilidade não é avaliada*: a dimensão do enxame é mantida fixa, não havendo qualquer teste de transferência para N não visto em treino. Terceira, a *avaliação* confronta a política aprendida apenas com políticas aleatórias, sem *baseline* de outro paradigma, sem repetição em execuções independentes e sem inferência estatística. Os próprios autores remetem para trabalho futuro a inclusão de obstáculos dinâmicos, agentes heterogêneos e limitações de comunicação — aspetos que a metodologia desta dissertação contempla.

3.4.3. O Estudo Comparativo Isolado

Merece registo autónomo o único estudo do corpo que confronta diretamente os dois paradigmas na navegação de enxames [36]. É significativo que provenha do mesmo grupo (Universidade de Miskolc) que havia aplicado RL profundo à navegação de enxames [42]: a comparação surge, portanto, de investigadores que chegaram à otimização bio-inspirada *a partir* da aprendizagem, e não de uma tradição comparativa estabelecida. A sua existência isolada — um estudo em 58, restrito a uma tarefa de navegação (ainda que avaliada em ambientes de complexidade crescente) e sem transferência para dimensões de enxame não vistas, sem robustez a falhas e sem significância estatística — confirma, mais do que refuta, o diagnóstico de isolamento metodológico entre as duas comunidades.

3.5. O Desafio do *Deployment Gap* em Robótica de Enxame

Uma política de controlo que convirja para taxas de sucesso próximas de 100% em simulação não oferece, por si só, qualquer garantia de desempenho num robô físico. O fosso entre os dois domínios — o *reality gap* — é apontado por Kegeleirs e Birattari (2025) como uma preocupação central, sobretudo na abordagem evolutiva, e alargado por estes autores ao *Deployment Gap*, que abrange também a transferência entre plataformas físicas [15]. A causa é estrutural: o treino por reforço otimiza a política para a distribuição exata de transições $\mathcal{P}(s' | s, a)$ do simulador, pelo que qualquer discrepância sistemática entre essa distribuição e a do mundo real é explorada como se fosse uma regularidade legítima do ambiente, conduzindo a comportamentos que falham — por vezes catastroficamente — na plataforma física.

3.5.1. Fontes do Fosso de Realidade

As discrepâncias que alimentam o *Deployment Gap* podem agrupar-se em três classes:

- **Dinâmica de atuação e física não modelada:** um simulador cinemático — como o adotado nesta dissertação, em que a ação é um deslocamento egocêntrico — ignora a inércia, a aceleração finita dos motores, a fricção assimétrica entre rodas, a folga mecânica e o escorregamento. Num chassis *differential-drive* real (e-puck, Khepera IV), um comando de velocidade não se traduz instantaneamente na deslocação pretendida, e pequenos enviesamentos de calibração entre motores acumulam-se em desvios de trajetória que a política treinada nunca observou.
- **Ruído e enviesamento sensorial:** a observação simulada (bússola de *homing*, LiDAR, posição relativa de vizinhos) é exata ou corrompida apenas por ruído idealizado. Os sensores reais introduzem ruído não-gaussiano, latência, ângulos mortos, reflexões espúrias no LiDAR e erros de odometria que derivam ao longo do tempo. Uma política que dependa de leituras precisas da direção ou da distância degrada-se à medida que estas se tornam ruidosas.
- **Comunicação e coordenação:** o sinal de recrutamento entre vizinhos é, no simulador, transmitido de forma instantânea e fiável. Em hardware real, a comunicação sofre de latência, perda de pacotes, alcance limitado e colisões de mensagens — e a perceção da posição dos vizinhos depende ela própria de sensores imperfeitos.

3.5.2. Agravamento à Escala do Enxame

O *Deployment Gap* é particularmente severo em sistemas multiagente. Ao contrário de um único robô, num enxame os pequenos erros de cada agente *compõem-se e interagem*: um desvio de trajetória individual altera a configuração espacial percebida pelos vizinhos, que ajustam o seu comportamento, propagando a perturbação pelo coletivo. Comportamentos coordenados que emergem de um equilíbrio delicado em simulação (a sincronização de três agentes na porta cooperativa, o cerco a 360° do alvo móvel) são precisamente os mais frágeis perante este efeito de amplificação, pois dependem de uma coordenação fina que o ruído real tende a dissolver.

3.5.3. Estratégias de Mitigação

A literatura propõe várias famílias de técnicas para estreitar o fosso. A *aleatorização de domínio* (*domain randomization*) é a mais difundida: durante o treino, randomizam-se sistematicamente os parâmetros físicos e sensoriais do simulador (massas, fricções, ganhos de motor, níveis de ruído), forçando a política a ser robusta a uma *distribuição* de dinâmicas em vez de a uma única, de modo que o mundo real surja como apenas mais uma amostra dessa distribuição [43]. Complementarmente, a *identificação de sistema* (*system identification*) calibra os parâmetros do simulador a partir de medições reais, reduzindo o enviesamento à partida; e o *ajuste fino* (*fine-tuning*) no robô físico, ou em simuladores de fidelidade crescente, adapta a política às condições efetivas de operação.

3.5.4. Posicionamento desta Dissertação

Embora a validação *sim-to-real* permaneça fora do âmbito experimental deste trabalho (Secção 7.3), várias opções de desenho foram tomadas no sentido de *antecipar* o fosso, e não de o ignorar. A percepção do ambiente é mantida *local e parcial* (LiDAR limitado a 8 m, sem mapa dos obstáculos), à semelhança das plataformas de exame, ainda que com um alcance superior ao dos sensores de proximidade de robôs como o e-puck; introduz-se *observabilidade parcial genuína* ao nível dos obstáculos (detetáveis apenas localmente); a robustez é explicitamente testada através da *injeção de falhas* de 10% dos agentes a meio do episódio (Secção 6.13); e o ambiente inclui *perturbações dinâmicas* (obstáculos e alvo móveis). A elevada resiliência observada perante falhas sugere que a partilha de parâmetros confere ao exame uma redundância funcional favorável à transferência. A validação empírica em hardware, recorrendo às técnicas de *domain randomization* e *system identification* acima descritas, constitui a extensão natural deste trabalho (Secção 7.5).

3.6. Lacunas Identificadas e Oportunidade de Melhoria

A revisão permite substituir por evidência quantificada aquilo que, na literatura, é habitualmente afirmado por intuição: a principal barreira ao avanço da robótica de exame não é a escassez de algoritmos — os 58 estudos incluídos, produzidos em sete anos, atestam o contrário — mas a *ausência de rigor comparativo entre paradigmas*. As duas comunidades operam em silos metodológicos quase estanques, e a Secção 3.3 mede a espessura da parede que os separa: 21 estudos de MARL, 23 de otimização bio-inspirada, *um único* a compará-los.

O silo manifesta-se de forma simétrica em cada campo. A comunidade de aprendizagem afere os seus algoritmos contra outros algoritmos de aprendizagem — a própria *suite* de referência da área contempla exclusivamente métodos de MARL, sem qualquer *baseline* evolutiva [14] — privilegiando a estabilidade do treino e a superação da não-estacionaridade. A comunidade de otimização, por seu turno, mede-se em otimalidade de percurso e eficiência energética, tipicamente em mapas conhecidos, sem confrontar a rigidez dos seus controladores com a adaptabilidade que a aprendizagem promete. Nenhuma das duas responde à pergunta que um engenheiro de exames efetivamente enfrenta: *qual dos paradigmas escolher, para este cenário, com este orçamento de cómputo e esta variabilidade de dimensão do exame?*

Desta fragmentação decorrem três consequências que esta dissertação procura corrigir:

- **Ausência de *benchmarking* cruzado sob protocolo idêntico.** Sem cenários, métricas, orçamentos de treino e protocolos de avaliação comuns, os resultados publicados pelos dois campos são incomensuráveis. Esta tese avalia os três controladores nos mesmos sete cenários, com o mesmo número de execuções independentes, avaliação determinística emparelhada e inferência não-paramétrica ao nível da execução.
- **Escalabilidade afirmada, raramente medida.** Ainda que 30 dos 58 estudos invoquem a escalabilidade, a avaliação por transferência *Zero-Shot* para dimensões de enxame não vistas em treino permanece rara; as soluções que a asseguram fazem-no pela *representação* — grafos [16], [32] ou campo médio [40] — e não pelo algoritmo de otimização. É esta separação entre representação e otimizador que a QI2 examina — embora, como a Secção 7.3 declara, a arquitetura de grafo só exista aqui no controlador evolutivo, pelo que os dois fatores não ficam isolados experimentalmente.
- **O desenho do sinal de treino como variável esquecida.** A literatura de desenho automático de enxames reconhece a dificuldade de especificar a função-objetivo, ao ponto de desenvolver métodos que a contornam — procura por novidade [21] ou aprendizagem por demonstração [38]. Mas o efeito do *desenho da aptidão* sobre o sucesso ou o colapso do paradigma evolutivo não é, tipicamente, tratado como objeto experimental. Esta tese trata-o (QI5), e conclui que dele depende a diferença entre um controlador que colapsa e um que iguala os métodos de gradiente.

A oportunidade que se abre é, assim, dupla: fornecer o *benchmark* cruzado que falta, e fazê-lo sobre uma arquitetura — o grafo dinâmico $G = (V, E)$ com atenção [16], [30] — que confere invariância à permutação e escalabilidade, permitindo comparar não apenas algoritmos, mas os *eixos* em que cada paradigma é preferível.

3.7. Artigos-Chave e Ligação ao Problema

Seis trabalhos delimitam com precisão o espaço que esta dissertação ocupa — três provenientes dos 58 estudos incluídos (Iskandar; Hasselmann; Kegeleirs) e três de enquadramento complementar, externos ao corpus e assinalados como tal: Zhang, identificado pela pesquisa mas excluído na fase de elegibilidade (E8 — fora do núcleo da revisão); Chen, doutro domínio de aplicação; e Yigit, não recuperado pelas cadeias de pesquisa (o título não contém nenhum dos termos de enxame robótico exigidos no título). A lacuna quantificada adiante (“um em 58”) refere-se exclusivamente ao corpus da revisão:

- (1) **Iskandar et al. (2024)** — o *único* estudo do corpo a comparar diretamente aprendizagem por reforço com PSO na navegação de enxames [36]. Confirma que a comparação é possível e pertinente, mas fá-la numa só tarefa (em ambientes de complexidade crescente), sem transferência para dimensões de enxame não vistas, sem robustez a falhas e sem tratamento estatístico: exatamente as três dimensões que esta tese acrescenta.
- (2) **Hasselmann, Ligot e Birattari (2023)** — gera repertórios comportamentais por *novelty search* para o desenho automático de enxames [21]. Estabelece a procura por

novidade como mecanismo legítimo no desenho de enxames, fundamentando a hibridização testada na QI6.

- (3) **Zhang et al. (2022)** — GNNs hierárquicas para exploração multi-robô, operando sobre vizinhanças de dimensão variável [32]; é a evidência *robótica* de que a invariância à dimensão do enxame é uma propriedade da *representação*, e não do otimizador (QI2). (Recuperado pela pesquisa mas excluído na elegibilidade — E8, tarefa de exploração fora do núcleo da revisão; é citado pelo mecanismo de representação.)
- (4) **Chen et al. (2024)** — estabelece o mecanismo que sustenta essa invariância: numa política sobre grafo, o número de parâmetros treináveis é *independente do número de agentes*, pelo que a política aprendida se aplica diretamente a grafos maiores sem retreino [16]. O trabalho é conduzido em redes multi-salto sem fios, e não em robótica — é citado aqui pelo mecanismo, não pelo domínio.
- (5) **Kegeleirs et al. (2024)** — mede a *transferibilidade* do desenho automático *off-line* de enxames, da simulação para o real e entre plataformas distintas [37]; delimita o *Deployment Gap* que esta dissertação antecipa no desenho mas não valida empiricamente (Secção 7.3).
- (6) **Yigit et al. (2026)** — PPO descentralizado com *parameter sharing* em navegação de enxame [35], da mesma família da *baseline* aqui adotada (identificado fora da pesquisa sistemática, cujas cadeias exigem termos de enxame robótico no título); a sua avaliação — contra políticas aleatórias, com vizinhança reduzida ao agente mais próximo, sem variação da dimensão do enxame e sem inferência estatística — ilustra o déficit de rigor comparativo que motiva este trabalho.

3.8. Conclusão Parcial

A revisão sistemática confirma que os dois paradigmas em confronto estão, ambos, em desenvolvimento ativo: 21 dos 58 estudos incluídos assentam em MARL e 23 em otimização bio-inspirada. O enquadramento complementar mostra, por sua vez, que a escalabilidade a dimensões de enxame não vistas é hoje resolvida pela *representação* — grafos com atenção [16], [32] ou aproximações de campo médio [40] — e que o paradigma evolutivo dispõe de instrumentos maduros para tarefas decetivas, da procura por novidade [21] à aprendizagem social distribuída [19].

O que a revisão *não* encontra é um confronto rigoroso entre os dois. Um único estudo em 58 compara empiricamente as duas famílias na navegação de enxames [36], e fá-lo sem transferência para dimensões de enxame não vistas, sem testes de robustez a falhas e sem inferência estatística. É este vazio — medido, e não apenas alegado — que o presente trabalho preenche: um *benchmark* cruzado sob protocolo idêntico, que permite determinar quando o custo computacional da aprendizagem compensa a robustez da otimização, e que mostra, no processo, que a fronteira entre os dois paradigmas depende menos do algoritmo escolhido do que do desenho do sinal que o guia (Capítulo 6).

3.9. Síntese Comparativa da Literatura

O Quadro 3.1 sintetiza os estudos — do corpo da revisão e do seu enquadramento complementar, com as entradas externas assinaladas — que melhor delimitam a lacuna, organizados pelo paradigma que empregam.

QUADRO 3.1. Síntese dos estudos mais relevantes do corpo da revisão ($n = 58$) e do seu enquadramento complementar (entradas externas ao corpus assinaladas), com as lacunas que motivam esta dissertação.

Referência	Paradigma	Contributo principal	Limitação / lacuna
Iskandar et al. (2024) [36]	Comparativo (RL vs. PSO)	Único estudo do corpo a confrontar os dois paradigmas na navegação de enxames.	Uma tarefa (vários ambientes); sem <i>Zero-Shot</i> , sem testes de falha, sem inferência estatística.
Hasselmann et al. (2023) [21]	Bio-inspirado (<i>novelty search</i>)	Repertórios de comportamentos gerados por procura por novidade para desenho automático de enxames.	Não confronta o desempenho com controladores treinados por gradiente.
Ordáz-Rivas et al. (2026) [39]	Bio-inspirado (multiobjetivo)	Otimização multiobjetivo de <i>foraging</i> com enxames (recolha vs. energia).	Sem <i>baseline</i> de aprendizagem; sem cenários decetivos.
Bredeche & Fontbonne (2022) [19]	Bio-inspirado (evolução <i>online</i>)	Aprendizagem social distribuída: os controladores propagam-se entre robôs durante a missão.	Foco no mecanismo evolutivo; sem comparação com MARL.
Chen et al. (2024) [16]	MARL (GNN) (<i>fora do corpus: redes sem fios</i>)	Estabelece o mecanismo da invariância: nº de parâmetros independente do nº de agentes $\Rightarrow$ transferência sem retreino.	Domínio não robótico (redes multi-salto); sem obstáculos físicos nem <i>baseline</i> evolutiva.
Zhang et al. (2022) [32]	MARL (GNN) (<i>fora do corpus: excluído na elegibilidade, E8</i>)	GNNs hierárquicas para exploração com vizinhanças de dimensão variável.	Avaliado só contra métodos de aprendizagem; escalabilidade não isolada do otimizador.
Tang et al. (2023) [40]	MARL (campo médio)	Custo de coordenação independente de N por aproximação de campo médio.	Sem comparação com otimização bio-inspirada; sem testes de robustez a falhas.
Kegeleirs et al. (2024) [37]	Bio-inspirado (<i>sim-to-real</i>)	Quantifica a transferibilidade do desenho automático entre simulação e plataformas reais.	Não avalia desempenho de tarefa face a MARL.
Yigit et al. (2026) [35]	MARL (PPO) (<i>fora do corpus: não recuperado pelas cadeias</i>)	PPO descentralizado com <i>parameter sharing</i> em navegação de enxame; valida a <i>baseline</i> aqui adotada.	Vizinhança reduzida ao agente mais próximo; N fixo (escalabilidade não avaliada); compara-se apenas com políticas aleatórias.

A leitura do quadro expõe o padrão: cada estudo é sólido *dentro* do seu paradigma e mede-se contra os seus pares — os trabalhos de MARL comparam-se com MARL, os evolutivos com evolutivos. A coluna das limitações repete, com variações, a mesma frase: *não confronta o outro paradigma*. É precisamente esse confronto, sob protocolo comum e nas três dimensões que a literatura trata isoladamente (desempenho de tarefa, escalabilidade e robustez), que constitui o objeto do capítulo seguinte.

CAPÍTULO 4

Desenvolvimento do Ambiente de Simulação

A totalidade do sistema descrito neste capítulo — simulador, cenários, implementações dos três algoritmos, *scripts* de avaliação e de geração de figuras — está disponível em código aberto no repositório do projeto, <https://github.com/Pombo2001/swarm-robotics-tese>, com o procedimento de instalação e de reprodução dos resultados documentado no `README.md` (Anexo E). Cada decisão de implementação discutida nas secções seguintes é, assim, verificável no código correspondente.

O simulador inspirou-se, no plano conceptual, no JBotEvolver [44], a plataforma de robótica evolutiva desenvolvida no Iscte que integra num mesmo ambiente a simulação de enxames e a evolução dos respetivos controladores. Dele se retomou a ideia de uma plataforma única em que o ambiente, o controlador e o algoritmo de otimização são componentes separáveis, configurados por ficheiro e avaliados sob o mesmo protocolo. Não se reutilizou código: o JBotEvolver é implementado em Java e orientado para a neuroevolução, ao passo que a comparação que esta dissertação faz exigia acesso direto às implementações de referência do PPO e do SAC da *Stable-Baselines3* e à computação tensorial do PyTorch, ambas em Python — razão pela qual o simulador foi escrito de raiz nessa linguagem.

4.1. Arquitetura de Software e Stack Tecnológica

O sistema foi implementado com bibliotecas de uso corrente em aprendizagem automática. A arquitetura é modular: cada componente (ambiente, controlador, algoritmo de treino) pode ser substituído sem alterar os restantes.

4.1.1. Ambiente de Simulação e Interação

O desenvolvimento foi centralizado na linguagem *Python 3.x*, escolhida pelas bibliotecas de aprendizagem automática disponíveis. A interface do simulador segue o estilo da *API parallel do PettingZoo* — um padrão aberto para ambientes de agentes múltiplos —, devolvendo a cada passo dicionários de observações, recompensas e sinais de terminação indexados por agente. Esta convenção abstrai a física interna e facilita a troca entre algoritmos sem reescrever o código base do ambiente [14].

A execução é descentralizada: cada agente decide a partir da sua própria observação, sem qualquer controlador central. Essa observação é local e parcial quanto ao *ambiente* — os obstáculos só são detetados pelo LiDAR, de alcance limitado —, mas não quanto ao *enxame*: inclui a direção e a distância de todos os outros agentes, sem limite de alcance (Secção 4.2.2). Em ambos os casos os sensores simulados são idealizados (Secção 3.5).

4.1.2. Implementação dos Algoritmos

A construção das redes neurais e o treino dos agentes MARL apoiaram-se nas seguintes ferramentas:

- **PyTorch:** Utilizado para a implementação da *Graph Neural Network* (GNN) do algoritmo evolutivo, pela facilidade com que permite escrever e depurar operações tensoriais próprias.
- **Stable-Baselines3:** Biblioteca de referência para a implementação dos algoritmos PPO e SAC, fornecendo implementações estáveis e bem validadas pela comunidade científica, com suporte a ambientes vetorizados paralelos.
- **Paralelização Multiprocesso (CPU):** A aceleração do treino assenta na execução paralela de múltiplos ambientes de simulação em processos independentes — `SubprocVecEnv` para o PPO e o SAC e `multiprocessing.Pool` para a avaliação da população evolutiva —, distribuindo a carga pelos vários núcleos de CPU dos servidores de cálculo. O simulador é implementado em Python/NumPy e executa em CPU; as redes envolvidas (a MLP [256, 256] da *Stable-Baselines3* e a *GNNAgent3D* de $\approx 8k$ parâmetros) são suficientemente compactas para não exigirem GPU dedicada, sendo o *throughput* de treino dominado pela simulação física do ambiente e não pela passagem pelas redes.

4.1.3. Arquitetura Geral do Simulador

A Figura 4.1 ilustra o fluxo de dados em ciclo fechado do simulador, onde cada agente iterativamente recolhe observações e executa ações sob a validação de um motor de física contínuo. A organização estática do código — as classes e as relações entre elas — está no diagrama de classes da Figura D.1, no Anexo D.



FIGURA 4.1. Diagrama da arquitetura do simulador e fluxo de interação agente-ambiente.

4.2. O Ambiente de Simulação

O simulador foi desenvolvido de raiz em *Python*. É um simulador cinemático: modela as restrições sensoriais dos agentes, as colisões e as perturbações dinâmicas do ambiente, mas não a dinâmica dos motores (Secção 3.5). Todos os algoritmos são avaliados no mesmo ambiente, garantindo a paridade da comparação.

4.2.1. Modelo Físico e Hiperparâmetros

A arena de simulação é um domínio esférico de raio $r_{arena} = 15$ m (a sua leitura tridimensional está detalhada na Secção 4.2.4), onde $N = 20$ agentes homogêneos operam em simultâneo. O Quadro 4.1 sintetiza os hiperparâmetros físicos e arquiteturais aplicados.

QUADRO 4.1. Hiperparâmetros utilizados na simulação e no treino dos modelos.

Categoria	Hiperparâmetro	Valor
Ambiente	Raio da Arena (r_{arena})	15,0 m
	Número de Agentes (N)	20
	Número de Obstáculos Dinâmicos	100
	Raio dos Obstáculos (r_{obs})	0,2 m
	Velocidade dos Obstáculos (v_{obs})	0,02 m/s
	Raio do Ninho (r_{nest})	1,5 m
	Velocidade do Ninho (v_{nest})	0,015 m/s
	Limite do Temporizador de Fome ($hunger$)	250 passos (sem geodésico)
	Agentes Exigidos por Chegada (k_{min})	3 ag. (1 nos labirintos)
	Alcance do LiDAR	8,0 m
	Fator de Progresso (PBRs)	10,0
	Resolução do Campo Geodésico	0,4 m
PPO	Taxa de Aprendizagem (lr)	10^{-4}
	Passos por Rollout (n_{steps})	64 (por CPU)
	Batch Size	512
	Número de Épocas (n_{epochs})	4
	Arquitetura da Rede	MLP [256, 256]
	Número de CPUs Paralelos	16
SAC	Buffer de Replay	500 000 transições
	Coeficiente de Entropia (α)	0,1
	Taxa de Aprendizagem (lr)	10^{-4}
	Arquitetura da Rede	MLP [256, 256]
Evolutivo (AE)	Tamanho da População (K)	30 genomas
	Taxa de Mutação (p_{mut})	0,1
	Desvio Padrão Inicial (σ)	0,1 ($\times 0,999/\text{ger.}$; $\sigma_{\min} = 0,03$)
	Episódios de Avaliação por Genoma (E)	4
	Dimensão Oculta da Rede (h)	32 ($\approx 8k$ pesos)

4.2.2. Espaço de Observação

As observações de cada agente são parciais quanto ao ambiente — os obstáculos só são vistos pelo LiDAR — e expressas num referencial *egocêntrico* (relativo à orientação atual do próprio

robô). O vetor de observação tem dimensão $d_{obs} = 16 + (N - 1) \times 5 = 111$ (para $N = 20$ agentes) e está estruturado em quatro componentes:

$$\mathbf{o}_i = [\mathbf{v}_{nest,i}, d_{nest,i}, \mathbf{l}_i, \mathbf{v}_{porta,i}, d_{porta,i}, \mathbf{n}_{1,i}, \dots, \mathbf{n}_{19,i}] \in \mathbb{R}^{111} \quad (4.1)$$

- (1) **Direção e Distância ao Ninho (4 valores):** Projeção do vetor direcional unitário do ninho nos eixos egocêntricos Frontal (F), Direito (R) e Superior (U) do robô, mais a distância normalizada ($d/2r_{arena}$). Esta informação de *homing* está sempre disponível, mesmo quando existem muros entre o agente e o ninho. Este pressuposto é padrão em robótica de enxame — assume-se um sentido global de orientação para a base (análogo à bússola magnética dos pombos, à orientação solar das formigas ou ao GPS de drones), mas *nunca* um mapa prévio dos obstáculos. A parcialidade da observação é assim mantida ao nível dos obstáculos (detetáveis apenas localmente pelo LiDAR), e não ao nível do objetivo. Esta opção corrige uma limitação de uma versão anterior, em que a direção era anulada na ausência de linha de visão, o que cegava os agentes em todos os cenários com paredes e os impedia de aprender a contornar.
- (2) **LiDAR Horizontal (8 valores, vetor $\mathbf{l}_i$):** Oito raios medidos em plano horizontal, uniformemente distribuídos a 360° , com alcance máximo de 8 m. O valor devolvido é $1 - (d_{hit}/d_{max})$, onde 1,0 significa impacto iminente. O alcance foi dimensionado para que o contorno dos grandes obstáculos dos labirintos (p. ex. as pernas do Muro em U, a ≈ 8 m da rota direta) seja detetável a tempo de iniciar o desvio, preservando ainda assim a observabilidade parcial: o agente não vê o mapa completo nem a direção geodésica.
- (3) **Direção e Distância à Porta/Entrada (4 valores, $\mathbf{v}_{porta,i}$ e $d_{porta,i}$):** Direção egocêntrica e distância normalizada à porta-objetivo do cenário, preenchidas *apenas* nos cenários com uma porta física definida (Porta Cooperativa e Porta Cooperativa com Alternativa); nos restantes são nulas. A porta é um sub-objetivo explícito do cenário — tal como o ninho —, pelo que a indicar não constitui revelar o trajeto. Distingue-se assim de fornecer *waypoints* das passagens dos labirintos, o que equivaleria a um planeador externo e quebraria a observabilidade parcial.
- (4) **Vetor de Vizinhos ($19 \times 5 = 95$ valores, vetores $\mathbf{n}_{j,i}$):** Para os restantes 19 agentes, envia-se o vetor de direção local (3 valores), distância normalizada (1 valor) e um sinal binário de comunicação (1 valor). Este último é ativado (1,0) quando o vizinho atinge o ninho e se encontra a repousar, servindo como sinal de recrutamento. Não há limite de alcance nem oclusão pelas paredes: cada agente recebe a posição relativa de todos os outros, onde quer que estejam — uma simplificação idealizada, comum aos três controladores (Secção 7.3).

4.2.3. Espaço de Ação e Dinâmica do Passo de Simulação

Cada agente emite, a cada passo, uma ação contínua $\mathbf{a}_i = (a_F, a_R, a_U) \in [-1, 1]^3$ interpretada como um *deslocamento no referencial egocêntrico* do robô. O vetor é convertido para o referencial global através da base ortonormal $\{F, R, U\}$ (Frontal, Direito, Superior) e escalado

por um passo máximo $\delta_{max} = 0,2$ m:

$$\Delta \mathbf{p}_i = 0,2 \cdot (a_F \mathbf{F}_i + a_R \mathbf{R}_i + a_U \mathbf{U}_i) \quad (4.2)$$

A orientação $\mathbf{F}_i$ é atualizada para a direção do movimento sempre que $\|\Delta \mathbf{p}_i\| > 0,02$, garantindo coerência entre a pose visual e a trajetória.

O passo de simulação (step) processa o enxame por uma ordem determinística que assegura a consistência física:

- (1) **Perturbações dinâmicas:** movimento dos obstáculos e do alvo móvel; injeção de falhas de agentes (Secção 6.13) quando aplicável.
- (2) **Aplicação da ação com *wall-sliding*:** cada deslocamento é projetado ortogonalmente à normal de qualquer muro intersetado, permitindo deslizamento em vez de paragem abrupta.
- (3) **Resolução de colisões:** reposicionamento por penetração contra obstáculos (distância euclidiana), muros (AABB) e fronteira circular da arena.
- (4) **Separação física:** repulsão geométrica de pares de agentes sobrepostos (a penalização *anti-clustering* associada está desativada na formulação final da recompensa, Secção 4.3; a resolução física mantém-se sempre ativa).
- (5) **Lógica de tarefa:** verificação de ocupação do ninho (k_{min} agentes, consoante o cenário), cerco do alvo móvel (360°) ou abertura da porta cooperativa.
- (6) **Atribuição de recompensa e terminação:** cálculo dos termos de $\mathcal{R}$ (Secção 4.3) e verificação do horizonte temporal ($t \geq t_{max}$).

4.2.4. Cenários de Teste

A tarefa é a mesma em todos os cenários, com uma exceção, e não inclui objetos para apanhar: o objetivo é levar agentes até ao ninho. Conta-se uma *chegada* sempre que o número de agentes exigido pelo cenário (k_{min} : três nos cenários cooperativos, um nos labirintos de navegação pura; Secção 4.3) se encontra em simultâneo dentro do ninho. Esses agentes recebem a recompensa de tarefa e reaparecem na zona de partida, pelo que a chegada se repete ao longo do episódio, e o número de *chegadas por episódio* é a métrica de tarefa desta dissertação; no Sandbox, o próprio ninho muda de posição após cada chegada. A exceção é a Perceção Cooperativa, que não tem ninho: aí conta como chegada o cerco do alvo móvel por três agentes, após o qual o alvo reaparece noutro ponto.

Para garantir uma avaliação exaustiva e progressiva das capacidades de controlo descentralizado, foram desenvolvidos sete cenários de dificuldade crescente (Figuras 4.2 e 4.3), descritos um a um a seguir às figuras.

Duas notas sobre a leitura da Figura 4.2, ambas consequência de o domínio ser tridimensional. Primeira, o deslocamento dos agentes tem componente vertical — a ação inclui Δz — e o domínio é uma esfera, não um disco: por isso os obstáculos aparecem nas suas cotas reais, distribuídos pelo volume, e não assentes num plano. Medido em 400 passos de uma política PPO no Sandbox, o afastamento vertical mediano ao plano de referência é de 2,35 m e o máximo de 11,8 m. Segunda, cada parede está desenhada duas vezes de propósito: sólida a 2 m, altura

arbitrária que serve apenas para a planta se ler, e translúcida com a altura que ocupa de facto no simulador — de -15 a $+15$ m, ou seja, cortando o domínio de lado a lado. É essa altura que a torna estanque, e a distinção não é cosmética: quando o mapa composto da Secção 6.11 passou a ter 60 m de raio sem que a altura das paredes acompanhasse, sobravam 45 m de espaço livre por cima de cada parede e as políticas atravessavam o labirinto por cima, sem o resolver. A altura passou a ser derivada do raio da arena, e as figuras mostram-na.

- (1) **Arena Aberta (Sandbox):** Cenário base sem barreiras fixas. Contém os agentes, obstáculos dinâmicos e um ninho móvel. Serve para aferir a cooperação no ninho sem a dificuldade adicional de navegar entre paredes.
- (2) **Beco Sem Saída (Muro em U):** Um grande obstáculo em forma de “U” bloqueia a linha direta ao ninho. A parede superior tem 14 m e as pernas laterais têm 10 m. Os agentes começam do lado de fora da “taça” e têm de explorar lateralmente as aberturas de 7 m. Avalia a exploração prolongada em detrimento da navegação gananciosa.
- (3) **Gargalo (Porta Estreita):** A arena é cortada a meio por uma parede contínua, deixando apenas uma passagem de 2,5 m de largura. Obriga o enxame a alinhar-se e a gerir o fluxo, e penaliza o congestionamento na passagem.
- (4) **Quatro Salas (Labirinto):** Duas grandes paredes cruzam a arena vertical e horizontalmente, criando quatro salas isoladas ligadas apenas por aberturas de 2,5 m de largura. Os agentes começam no extremo oposto ao ninho, exigindo navegação estruturada de longo alcance, sem que nenhum dos controladores disponha de memória.
- (5) **Porta Cooperativa:** Similar ao Gargalo, mas a passagem de 3 m de largura encontra-se bloqueada por um portão físico. A porta só abre (deslizando para fora do caminho, com realce visual no *dashboard*) quando no mínimo 3 agentes ocupam a zona de pressão em simultâneo. Por exigir uma sincronização mais demorada, este cenário dispõe de um horizonte temporal alargado de 800 passos (face aos 500 dos restantes), dando margem para que o enxame coordene a abertura e ainda alcance o ninho. Foca-se em comportamentos de sincronização sem comunicação explícita.
- (6) **Percepção Cooperativa (Alvo Móvel):** A noção de “ninho físico” desaparece. O alvo de interesse viaja livremente pelo mapa ao dobro da velocidade normal (0,03 m/s). Só é considerado capturado quando 3 agentes se aproximam a menos de 4 m de distância e se distribuem num raio de 360° em redor do mesmo, testando manobras de cerco e perseguição.
- (7) **Porta Cooperativa com Alternativa (Bypass):** Variante da Porta Cooperativa desenhada como problema de *recompensa enganadora (deceptive)*. A barreira horizontal mantém a porta central de 3 m (que continua a exigir 3 agentes na zona de pressão), mas o segmento direito da barreira termina antes da fronteira da arena, deixando uma abertura permanentemente livre de 4 m junto à parede leste — um percurso alternativo que dispensa cooperação. Uma parede defletora, colocada mais a norte no mesmo lado, obriga quem usa a alternativa a afastar-se temporariamente do ninho, tornando o percurso alternativo cerca de duas vezes mais longo do que a passagem pela porta. A

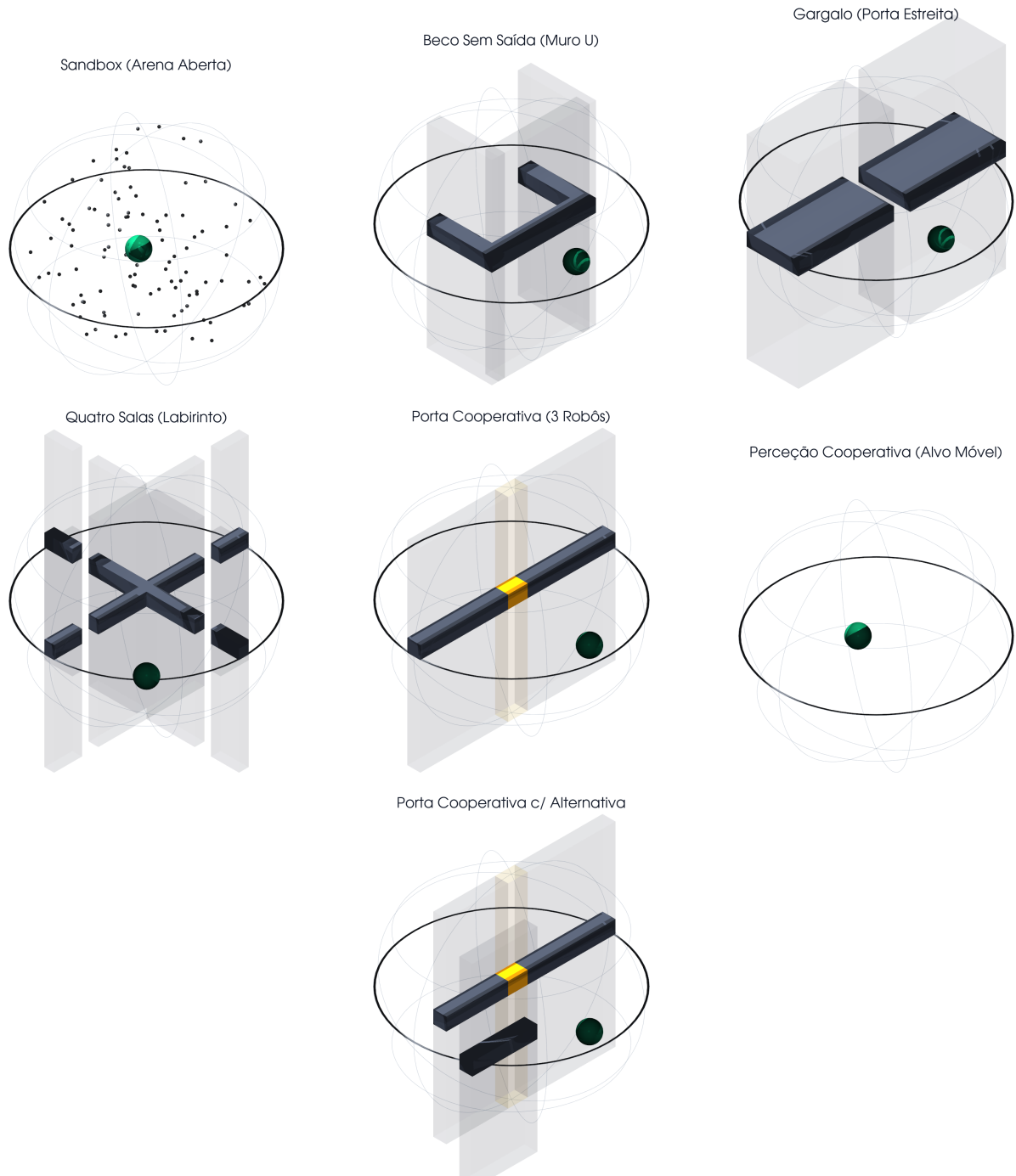


FIGURA 4.2. Renderizações 3D dos sete cenários de teste, geradas a partir da geometria real do simulador (`scripts/render_maps.py`). Em cima, da esquerda para a direita: Sandbox, Beco Sem Saída (Muro em U) e Gargalo; ao centro: Quatro Salas, Porta Cooperativa (com a porta destacada a âmbar) e Percepção Cooperativa; em baixo: Porta Cooperativa com Alternativa (*bypass*). A esfera verde assinala o ninho (ou o alvo móvel, na Percepção Cooperativa) e as esferas cinzentas os obstáculos. A malha de círculos é a fronteira do domínio, que é uma esfera de raio 15 m; o anel escuro marca a circunferência de referência $z = 0$. Cada parede aparece duas vezes: sólida, com altura ilustrativa de 2 m, e em cinzento translúcido com o volume real. A leitura da figura está detalhada no texto desta secção.

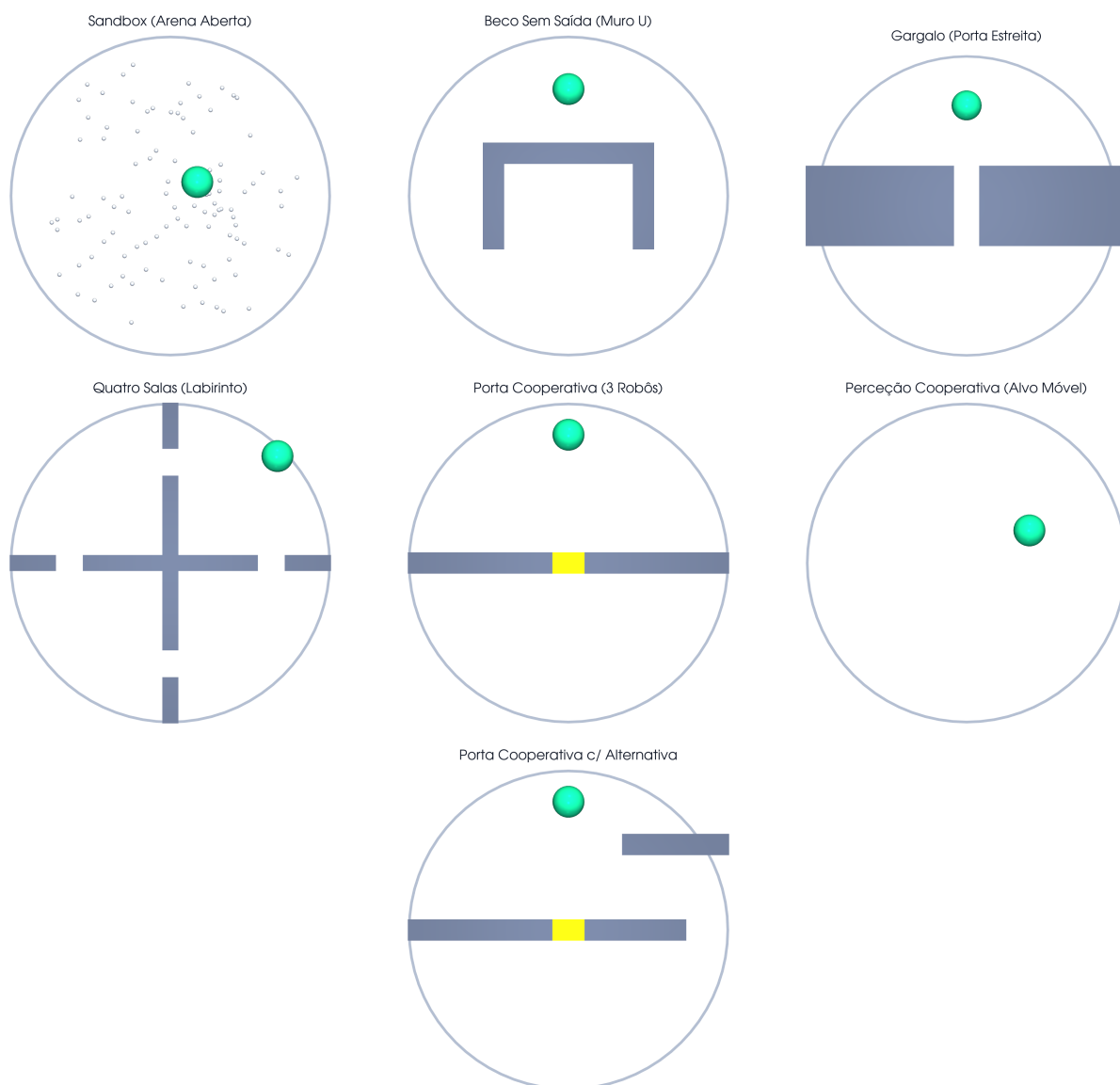


FIGURA 4.3. Vista de topo (planta) dos sete cenários, evidenciando a geometria das paredes e a posição relativa do ninho. As dimensões (comprimentos das paredes e largura das passagens) estão no texto desta secção; as plantas são esquemáticas e não têm escala impressa. Mesma ordem da Figura 4.2: em cima, Sandbox, Muro em U e Gargalo; ao centro, Quatro Salas, Porta Cooperativa e Percepção Cooperativa; em baixo, Porta Cooperativa com Alternativa.

rota mais curta atrai, assim, o enxame para a porta (que exige coordenação), enquanto a descoberta da alternativa exige contrariar o gradiente de aproximação ao objetivo — a propriedade que faz deste o cenário de referência para a QI6 (procura por novidade, Secção 1.4). Pelo horizonte de exploração acrescido, o limite temporal é alargado para 1000 passos.

4.3. Design da Função de Recompensa

A função de recompensa é o instrumento de desenho experimental mais sensível do sistema: as primeiras campanhas de treino acumularam termos secundários (penalizações de dispersão, de colisão entre agentes e de contacto com obstáculos) que, na prática, diluíam o sinal da tarefa

e abriam portas a comportamentos degenerados. A formulação final adotou deliberadamente uma *recompensa simples*, com o número mínimo de termos e dois focos — *explorar o mapa* e *encontrar o ninho* — em que a recompensa de tarefa domina claramente todos os sinais de *shaping*. As penalizações secundárias foram removidas *apenas do sinal de treino*: a resolução física correspondente mantém-se no motor (os agentes continuam a não se sobrepor nem a atravessar paredes, Secção 4.4.2), pelo que a simplificação não altera a dinâmica do mundo, apenas o que é ensinado.

$$\mathcal{R}(s, \mathbf{a}) = R_{\text{tarefa}} + R_{\text{progresso}} + R_{\text{exploração}} - R_{\text{controlo}} \quad (4.3)$$

Os valores exatos utilizados na campanha experimental final são os seguintes:

- **Tarefa (R_{tarefa}):** Recompensa dominante de +300 por chegada, atribuída (e *repetível* ao longo do episódio) a cada agente que participa numa chegada bem sucedida. O número de agentes exigidos em simultâneo no ninho ($k_{\min}$) é definido *por cenário*: $k_{\min} = 3$ nos cenários em que a cooperação é o objeto de estudo (Sandbox, Porta Cooperativa, Perceção Cooperativa e *Bypass*), mas $k_{\min} = 1$ nos labirintos de navegação pura (Muro em U, Gargalo e Quatro Salas) — exigir três agentes simultâneos nesses cenários transformava um problema de navegação numa cooperação artificial, registando zero chegadas mesmo quando um ou dois agentes alcançavam o ninho. Nos cenários com porta cooperativa acresce um bónus único de +100 a cada agente que participa na abertura da porta.
- **Progresso ($R_{\text{progresso}}$):** Baseado no método *Potential-Based Reward Shaping* (PBRs) [45], atribui $10,0 \cdot (\Phi_{t-1} - \Phi_t)$, onde Φ é o *potencial de navegação* até ao ninho. Nos cenários com paredes (Muro em U, Gargalo, Quatro Salas e as duas Portas Cooperativas) este potencial é a *distância geodésica* — o comprimento do caminho mais curto que contorna os obstáculos, calculado por uma pesquisa de custo uniforme (Dijkstra, 8-conexa) sobre uma grelha de discretização (0,4 m, com as paredes dilatadas pelo raio do robô) — e não a distância euclidiana. Esta escolha elimina na raiz um mínimo local clássico: com a distância euclidiana, contornar uma parede *afasta* o agente do ninho e é penalizado, prendendo o enxame contra os obstáculos; com a distância geodésica, qualquer passo ao longo do caminho viável é recompensado. Em rigor, a garantia de invariância de Ng et al. exige o termo $\gamma\Phi_t$ em vez de Φ_t ; como aqui o fator $\gamma = 0,99$ é omitido, o termo difere do PBRs por uma penalização $10,0 (1 - \gamma) \Phi_t$, proporcional à distância ao ninho e alinhada com a tarefa, pelo que a política ótima se preserva apenas de forma aproximada [45]. Decisivamente, o campo geodésico é um *sinal de treino* (a geometria é conhecida pelo simulador apenas durante o treino) e *não* uma observação: em execução o agente dispõe somente da bússola de *homing* euclidiana e do LiDAR local (Secção 4.2.2), preservando a observabilidade parcial. Distingue-se assim, claramente, de fornecer ao agente um mapa dos obstáculos ou *waypoints* de um planeador A^* .

- **Exploração ($R_{\text{exploração}}$):** Bónus de exploração *count-based* que atribui +0,5 na primeira vez que um agente visita cada célula de uma grelha de discretização espacial (2×2 m), em linha com a necessidade de mecanismos densos de exploração identificada por Xiao et al. (2026) [46]. O valor é *pequeno de propósito*: incentiva a cobertura do mapa em regiões onde o gradiente do potencial é pouco informativo, mas é dominado por larga margem pela recompensa de tarefa (+300, repetível). Iterações anteriores com um bónus maior demonstraram o risco desta calibração: a política aprendia a *vaguear* para colher exploração em vez de convergir para a chegada — um caso de *reward hacking* que motivou a simplificação da recompensa aqui descrita, e cujo análogo evolutivo (*fitness exploitation*) se analisa na Secção 6.15.
- **Controlo Energético (R_{controlo}):** Subtração de 0,05 a cada passo de simulação por agente não inativo. Pressiona temporalmente o enxame para chegar ao ninho o mais depressa possível; nos cenários de objetivo fixo é a única fonte de pressão temporal.
- **Fome (cenários sem campo geodésico):** Caso o temporizador de *hunger* de um agente ultrapasse 250 passos, o agente “morre”, sofre um revés de $-50,0$ e a sua posição é restabelecida no ponto de partida. Esta reposição está *ativa apenas nos cenários sem campo geodésico* (Sandbox e Perceção Cooperativa); nos labirintos, em que o potencial de progresso é geodésico, foi desativada, pois a combinação do teletransporte com a re-sincronização do potencial tornava o termo de progresso *farmável* (aproximar-se, ser repostado longe sem cobrança do recuo, repetir), induzindo políticas degeneradas que acumulavam recompensa de *shaping* sem nunca cumprir a tarefa.
- **Penalizações secundárias (desativadas na formulação final):** As versões preliminares incluíam penalizações de *anti-clustering* (proximidade entre agentes), de colisão agente-agente e de contacto com obstáculos. Na formulação final, os respetivos pesos foram colocados a zero: a separação física e o *push-out* continuam a ser *resolvidos* pelo motor (Secção 4.4.2), mas deixaram de gerar sinal de recompensa, eliminando três termos cuja calibração relativa baralhava o sinal da tarefa. Definiu-se como salvaguarda experimental que, caso surgissem comportamentos degenerados (agregação excessiva ou agentes “colados” às paredes), os termos seriam reintroduzidos gradualmente — o que não se revelou necessário.

4.4. Desafios de Engenharia e Otimização Computacional

O treino por reforço exige milhões de passos de simulação, pelo que o custo de cada passo decide o que é possível treinar. Esta secção descreve as opções de engenharia do ambiente (`swarm_env_3d.py`) e dos visualizadores 3D que mantêm esse custo baixo.

4.4.1. Motor de Visualização Interativa (Dashboard)

Para se poder inspecionar o comportamento do enxame sem abrandar o treino, a simulação está separada da renderização gráfica. A inspeção do comportamento é feita por um visualizador 3D implementado com o motor *Ursina* (assente em Panda3D), que projeta a telemetria do motor contínuo interno num espaço tridimensional navegável, com representação do ninho, obstáculos

móveis, muros e indicadores de proximidade. O controlo experimental — lançamento de treinos, seleção de cenários e geração de gráficos — é centralizado num *dashboard* interativo construído em *NiceGUI* (aplicação *web* local). Esta separação permite que o simulador opere num modo *headless* (sem sobrecarga gráfica) durante sessões de treino intensivo — processando milhares de transições por segundo —, enquanto oferece um modo interativo para demonstração e análise qualitativa das políticas aprendidas.

4.4.2. Otimização Matemática da Física e Detecção de Colisões

Cada passo de simulação tem de levar poucos milissegundos. Com $N = 20$ agentes e uma centena de obstáculos móveis, o custo da deteção de colisões cresce com o número de pares a testar, pelo que as operações são vetorizadas.

- **Deteção Geométrica de Colisões:** O motor resolve as colisões por testes geométricos diretos — distância Euclidiana para os obstáculos esféricos e caixas envolventes alinhadas aos eixos (*Axis-Aligned Bounding Boxes*, AABB) para os muros retangulares —, recorrendo a operações vetoriais *NumPy* no cálculo de distâncias e profundidades de penetração. Um passo dedicado de *separação física* repele geometricamente qualquer par de agentes sobrepostos, assegurando a ausência de interpenetrações não-físicas no enxame.
- **Wall-Sliding Cinemático:** Quando um robô entra em contacto com uma parede retangular, o motor não o imobiliza mecanicamente (o que penalizaria irreversivelmente a exploração do algoritmo MARL). Em vez disso, remove a componente do deslocamento segundo a normal da superfície de colisão, e o agente desliza ao longo da parede (*wall-sliding*). É uma simplificação cinemática: um chassis *differential-drive* real não desliza lateralmente desta forma.

4.4.3. Mecanismo de Atenção sobre Vizinhos (GNN)

O extrator de relações entre robôs foi implementado de raiz em *PyTorch puro* (sem recurso a bibliotecas de grafos como o *PyTorch Geometric*), através de um mecanismo de *atenção de produto interno escalado* sobre o conjunto de vizinhos, conforme formalizado nas Equações 2.17–2.19.

- **Representação Densa da Vizinhança:** A cada passo, cada agente i observa os restantes $N - 1$ agentes através de um vetor de *features* de dimensão fixa (direção egocêntrica, distância normalizada e sinal de comunicação por vizinho). Diferentemente de uma abordagem baseada em arestas esparsas com raio de comunicação fixo, todos os vizinhos são apresentados ao módulo de atenção, sendo a *relevância de cada um aprendida* pelos coeficientes α_{ij} — a rede atribui dinamicamente pesos próximos de zero aos vizinhos irrelevantes, dispensando um corte geométrico explícito.
- **Dimensão Fixa e *Parameter Sharing*:** Com $N = 20$, a observação tem dimensão fixa ($\mathbb{R}^{111}$), pelo que a integração em *batches* de treino é direta, sem necessidade de *padding* dinâmico ou conversão para listas de índices. Já as projeções *Query-Key-Value* e o *softmax* operam sobre o número de vizinhos presentes, pelo que a mesma

rede processa enxames de qualquer dimensão — viabilizando o *Zero-Shot Transfer* para $N \in \{10, 50, 100\}$ sem retreino.

4.4.4. Ambientes Vetorizados e Paralelização Multiprocesso

A recolha de experiência (PPO e SAC) e a avaliação da população evolutiva correm em paralelo:

- **Vectorized Environments:** O ambiente é envolvido pelo *wrapper* `SubprocVecEnv` da *Stable-Baselines3*, que corre 16 cópias do simulador em processos de CPU separados.
- **Mecanismo de Guilhotina (Early Stopping):** No treino evolutivo, um episódio cuja recompensa acumulada ao fim de 150 passos fique abaixo de um limiar é interrompido, e o genoma é penalizado (Secção 5.2.2). Evita-se assim gastar tempo de CPU a avaliar até ao fim genomas que a mutação tornou claramente disfuncionais.

4.5. Estrutura do Projeto e Extensibilidade

Um dos objetivos declarados desta dissertação é entregar uma *implementação de referência* reutilizável (Secção 7.4). Essa ambição obriga a tratar a estrutura do software como resultado, e não como detalhe de implementação: importa dizer, com precisão, o que custa a um terceiro estender este trabalho — e o que *não* foi desenhado para ser barato.

4.5.1. Organização dos Módulos

O repositório separa quatro responsabilidades, com dependências num só sentido (a simulação não conhece o treino, o treino não conhece a análise):

- `src/` ($\approx 2\,700$ linhas) — o núcleo. Contém o ambiente (`environment/swarm_env_3d.py`: física, LiDAR, geodésicas, recompensa), os controladores (`agents/`: a rede `GNNAgent3D` e uma interface `Agent.get_action`, hoje só usada pelo controlador aleatório de referência), os treinadores (`training/`: um módulo por paradigma) e o *registo canónico de cenários* (`scenarios.py`).
- `scripts/` ($\approx 25\,600$ linhas) — a orquestração e a análise: `run_experiments.py` (campanhas), a bateria de avaliação (`eval_*.py`), os testes estatísticos, a geração de figuras e os verificadores que confrontam os números desta dissertação com os dados.
- `dashboard/` ($\approx 8\,300$ linhas) — observabilidade: acompanhamento do treino, curvas ao vivo, matriz de resultados e ligação ao servidor remoto.
- `tests/` ($\approx 4\,800$ linhas) — garantias de não-regressão, discutidas adiante.

4.5.2. Pontos de Extensão

Acrescentar um algoritmo é barato. Um novo controlador exige (i) uma classe que produza a ação a partir da observação (uma rede PyTorch, como a `GNNAgent3D`, ou uma política da *Stable-Baselines3*), (ii) um módulo de treino em `src/training/` e (iii) *uma entrada* no dicionário `ALGORITHMS` de `run_experiments.py`. A partir daí, o algoritmo é automaticamente incluído nas campanhas, na avaliação determinística, nos testes de significância e nas figuras — porque toda a cadeia a jusante itera sobre esse registo, e não sobre nomes escritos à mão. A comparação MADDPG ou QMIX que a Secção 7.5 propõe é, neste sentido, um exercício de dias e não de semanas.

Acrescentar uma métrica ou um teste é barato. As métricas de avaliação são calculadas num único ponto (`scripts/eval_suite.py`) e propagam-se por escrita em `results/evaluation/eval_summary.csv`, do qual dependem, por leitura, a análise estatística, os gráficos e o painel. Uma métrica nova torna-se visível em toda a cadeia sem alterar nenhum consumidor.

Acrescentar um cenário é barato à superfície e caro no fundo. Os *metadados* de cenário — lista canónica, rótulos longos e curtos, classificação como labirinto — residem num único ficheiro (`src/scenarios.py`), do qual todos os consumidores importam. Esta centralização não é acidental: resultou da correção de um defeito real, em que o sétimo cenário foi treinado mas nunca avaliado, por a lista de cenários estar duplicada entre os módulos de treino e de avaliação. Com a fonte única, um cenário novo aparece automaticamente nas campanhas, nos quadros e nas figuras.

A *geometria*, porém, não goza da mesma disciplina: a construção de paredes, as posições de *spawn* e as regras específicas de cada cenário estão distribuídas por mais de uma dúzia de blocos condicionais (`if classic_scenario == ...`) dentro do ambiente. Acrescentar um cenário implica, portanto, tocar em vários pontos de um ficheiro extenso — um custo que cresce linearmente com o número de cenários e que constitui a dívida técnica mais visível deste projeto. A correção natural é um *registo de cenários* análogo ao dos algoritmos: cada cenário passaria a ser uma classe com uma interface fixa (`build_obstacles`, `spawn_positions`, `required_to_eat`), registada por decorador, e esses condicionais colapsariam numa única invocação polimórfica. A refatoração é direta e está identificada; não foi executada porque as campanhas experimentais já tinham começado e qualquer alteração ao ambiente invalidaria a comparabilidade dos resultados já obtidos — precisamente o tipo de compromisso entre pureza arquitetural e integridade experimental que uma dissertação empírica impõe.

4.5.3. Garantias de Não-Regressão

A extensibilidade só é real se o sistema puder ser alterado sem medo. Para esse efeito, as otimizações de desempenho mais arriscadas foram acompanhadas de *testes de equivalência bit-a-bit* contra a implementação de referência: a vetorização do LiDAR (adotada por multiplicar o *throughput* por ≈ 19) é confrontada com o ciclo explícito original em 8 000 cenas aleatórias — incluindo casos-limite como raios paralelos aos eixos e ausência de obstáculos —, exigindo-se igualdade *exata*; a vetorização das observações egocêntricas ($\approx 2,6\times$ no passo de simulação) é verificada do mesmo modo. A mecânica da porta cooperativa — o comportamento mais subtil do simulador — e o *cache* de elites do treinador evolutivo têm testes dedicados, este último assegurando que a otimização não altera a trajetória de treino. Estas verificações são o que permite afirmar que os ganhos de desempenho não alteraram a semântica do ambiente, e o que dará a um terceiro a confiança para modificar o núcleo sem invalidar silenciosamente os resultados publicados.

4.5.4. Reprodutibilidade

A configuração completa — física, recompensa, hiperparâmetros dos três algoritmos — reside num único ficheiro declarativo (`configs/foraging.yaml`), sem constantes dispersas pelo código. As sementes de avaliação são fixas, tornando a avaliação determinística e emparelhada entre algoritmos. Cada campanha regista, por execução, a curva de treino e o modelo campeão, e o repositório inclui ainda o protocolo e o registo integral da revisão sistemática (Capítulo 3). Um terceiro que clone o repositório e reavalie os modelos arquivados obtém, com as mesmas sementes e na mesma máquina, os mesmos números (noutra máquina, a vírgula flutuante desloca-os em cerca de uma chegada por episódio, sem mudar nenhuma contagem de convergência nem nenhuma comparação do GNN com os métodos de gradiente); repetir o *treino* com `run_experiments.py` e a configuração publicada reproduz o protocolo, mas não bit a bit, porque o orçamento é fixado em minutos e o número de passos de treino depende da máquina. A reavaliação exata é o critério mínimo para que uma comparação entre paradigmas possa ser contestada, e portanto levada a sério.

Arquitetura Algorítmica e Controladores

5.1. Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)

A primeira abordagem investigada fundamenta-se no *framework Robust and Scalable Swarm Control (RS2C)*. Este adota um esquema de *execução descentralizada com partilha de parâmetros (parameter sharing)* — em que todos os agentes partilham uma única política que atua sobre a observação individual de cada agente —, formulando a coordenação do enxame como um Processo de Decisão de Markov Parcialmente Observável Descentralizado (Dec-POMDP), em regime cooperativo:

$$\langle N, S, \{\mathcal{A}_i\}_{i \in N}, \mathcal{P}, \mathcal{R}, \Omega, \{O_i\}_{i \in N}, \gamma \rangle \quad (5.1)$$

cujos elementos foram definidos na Secção 2.5.1: N é o conjunto dos N agentes, S o espaço de estados, $\mathcal{A}_i$ o espaço de ações do agente i (Secção 4.2.3), $\mathcal{P}$ a dinâmica de transição, $\mathcal{R}$ a recompensa (Secção 4.3), Ω o espaço de observações conjuntas, O_i a observação local do agente i (instanciada no vetor $\mathbf{o}_i \in \mathbb{R}^{111}$ da Secção 4.2.2) e γ o fator de desconto.

Sob o RS2C investigam-se dois algoritmos de aprendizagem por reforço profundo padrão: *PPO (on-policy)* e *SAC (off-policy)*. Em ambos, os N agentes partilham uma única política (*parameter sharing*) que recebe a observação local de dimensão fixa $\mathbf{o}_i \in \mathbb{R}^{111}$ — a qual já codifica, em formato *dense*, as *features* de toda a vizinhança (Secção 4.2.2). Na implementação atual, esta política partilhada é uma rede totalmente ligada (MLP [256, 256], da *Stable-Baselines3*); o *mecanismo de atenção sobre grafo* formalizado no Capítulo 2 é instanciado no controlador neuroevolutivo (Secção 5.2), sendo a base da invariância à permutação e da capacidade de *Zero-Shot Transfer* aí avaliadas. A integração direta do módulo de atenção numa política treinável por gradiente constitui uma extensão natural deixada para trabalho futuro (Secção 7.5).

5.1.1. Configuração PPO (On-Policy)

No modo PPO, os pesos da política MLP partilhada são otimizados através de gradientes calculados sobre *rollouts* de experiência recolhidos com a política atual. O *pipeline* de treino opera da seguinte forma:

- (1) **Recolha de Experiência:** Em cada iteração, $N_{arenas} = 16$ cópias paralelas do simulador (*SubprocVecEnv*) geram transições durante $n_{steps} = 64$ passos cada. Como os $N = 20$ agentes de cada arena são tratados como ambientes independentes (*parameter sharing*), o *rollout buffer* acumula $16 \times 20 \times 64 = 20\,480$ transições por iteração.
- (2) **Cálculo da Vantagem:** As vantagens $\hat{A}_t$ são estimadas via GAE (Equação 2.10) com $\gamma = 0,99$ e $\lambda = 0,95$ sobre o *rollout* completo. Regista-se que $n_{steps} = 64$ é um horizonte curto para os padrões do PPO — o valor por omissão da biblioteca é 2048

—, o que significa que a vantagem é estimada sobre janelas de 64 passos dentro de episódios de 500; o que a paralelização recupera é o *tamanho* do *batch* (as 20 480 transições por iteração), não o alcance temporal de cada estimativa.

- (3) **Atualização:** O buffer é amostrado em *mini-batches* de 512 transições e o gradiente de L^{PPO} é calculado durante $n_{\text{epochs}} = 4$ passagens completas.
- (4) **Parameter Sharing:** Os $N = 20$ robôs partilham o mesmo conjunto de parâmetros θ . Cada transição (o_i^t, a_i^t, r_t) de qualquer agente contribui para o gradiente, aumentando efetivamente o tamanho do *batch* por fator N .

5.1.2. Configuração SAC (*Off-Policy*)

O SAC opera em modo *off-policy*: as experiências acumulam-se assincronamente num *Replay Buffer* de capacidade $|\mathcal{D}| = 500\,000$ transições. Para $N = 20$ agentes, cada passo de simulação contribui 20 transições, preenchendo o buffer rapidamente. O SAC mantém dois críticos independentes Q_{ψ_1} e Q_{ψ_2} (*double Q-trick*) e a cada passo do ambiente uma mini-batch de 256 transições é amostrada aleatoriamente para atualizar os parâmetros. O coeficiente de temperatura $\alpha = 0,1$ é mantido constante, sem adaptação automática. Esta é uma escolha de configuração, e deve ser lida como tal: a formulação de referência do SAC ajusta α por otimização dual contra uma entropia-alvo, e fixá-lo remove ao algoritmo o mecanismo com que ele próprio regula a exploração ao longo do treino. Não se sabe, com estes dados, qual teria sido o efeito de o deixar adaptativo — mas a possibilidade é consistente com o sub-treino que a verificação de convergência documenta em quatro das células do SAC (Secção 6.1), e é mais uma razão para ler os seus valores como *o desempenho atingível nesta configuração e neste orçamento*, e não como um limite do algoritmo.

5.1.3. Representação da Vizinhança e *Parameter Sharing*

A cada passo de simulação, o vetor de observação de cada agente é construído em `swarm_env_3d.py`:

- (1) **Vizinhança Completa:** As *features* de todos os restantes $N - 1$ agentes (direção egocêntrica, distância normalizada e sinal de comunicação) são incluídas no vetor de observação $\mathbf{o}_i$. Não é aplicado um corte por raio de comunicação fixo: no PPO e no SAC é a própria MLP que aprende a ponderar as entradas de cada vizinho; no controlador evolutivo, essa ponderação cabe ao mecanismo de atenção (coeficientes α_{ij}).
- (2) **Representação Densa de Dimensão Fixa:** O vetor $\mathbf{o}_i \in \mathbb{R}^{111}$ incorpora os $N - 1$ vizinhos em formato *dense*, preservando uma dimensão de entrada constante e dispensando estruturas esparsas (*edge-index*) ou *padding* dinâmico.
- (3) **Dimensão de Entrada Fixa (PPO/SAC):** A política MLP partilhada recebe um vetor de dimensão fixa $\mathbb{R}^{111}$, definida para $N = 20$ agentes. Por construção, esta abordagem não é invariante ao tamanho do exame: alterar N modifica a dimensão da observação $(16 + (N - 1) \times 5)$ e inviabiliza a aplicação direta da rede. A invariância à permutação e o *Zero-Shot Transfer* para $N \in \{10, 50, 100\}$ são propriedades do mecanismo de

atenção (Equações 2.17 e 2.18), cujas operações QKV e *softmax* se definem sobre o número de vizinhos presentes; são, por isso, avaliados no controlador neuroevolutivo (Secção 5.2).

5.2. Algoritmo 2: Otimização Bio-inspirada (Benchmark)

A *baseline* adotada assenta num processo de *Neuroevolução* [17], utilizando um Algoritmo Evolutivo clássico com elitismo (preservação dos 20% melhores) e mutações Gaussianas independentes nos pesos ($p_{mut} = 0,1$, com atenuação *sigma decay*). É neste controlador que se instancia a *rede de grafos com atenção* (GNNAgent3D) formalizada no Capítulo 2: é precisamente esta arquitetura, invariante ao número de agentes, que habilita o *Zero-Shot Transfer*. A otimização evolutiva *offline* ajusta os seus pesos sobre 30 genomas candidatos, sem recurso a gradientes nem a um sinal de recompensa denso por passo.

5.2.1. Função de Aptidão e Operadores de Variação

Os elementos genéricos de um algoritmo evolutivo foram enunciados na Secção 2.4.1; descreve-se aqui a forma que cada um assume nesta dissertação — que é onde reside a contribuição, dado que a QI5 investiga precisamente o efeito destas escolhas.

Função de *Fitness*: O critério de qualidade de cada genoma, $J(\theta_k)$, foi deliberadamente desenhado para que a *tarefa* domine a seleção, relegando o *shaping* para o papel de gradiente de desempate. A formulação final combina as chegadas com um *shaping* de *navegação terminal* (*homing*):

$$J(\theta_k) = \underbrace{\bar{f}_k \cdot \lambda_{task}}_{\text{tarefa}} + \underbrace{\beta \cdot \bar{h}_k}_{\text{shaping de homing}} \quad (5.2)$$

onde $\bar{f}_k = \frac{1}{E} \sum_{e=1}^E f_{k,e}$ é o número médio de chegadas (a tarefa pura) nos E episódios de avaliação do genoma, com $f_{k,e}$ o número obtido no episódio e , e $\bar{h}_k \in [0, 1]$ é o *homing*: a fração média de aproximação ao ninho *no fim do episódio*, medida por agente como $\text{clip}((\Phi_0 - \Phi_T)/\Phi_0, 0, 1)$, em que clip trunca ao intervalo $[0, 1]$ e Φ é o potencial de navegação até ao ninho da Secção 4.3 (geodésico nos cenários com paredes), avaliado no primeiro e no último instante do episódio. O peso $\lambda_{task} = 10000$ e a amplitude $\beta = 5000$ garantem que uma única chegada vale o dobro do *shaping* máximo: assim que um genoma chega ao ninho, a tarefa domina a seleção.

A escolha do *homing* terminal como *shaping* — em vez do retorno acumulado $\bar{R}_k$ — é a decisão de desenho central desta função, e resultou de uma iteração experimental documentada no Capítulo 6. Numa primeira formulação, o desempate era o retorno acumulado comprimido por uma $\tanh$; contudo, o retorno acumulado é *farmável*: os termos de progresso e exploração rendem a cada passo, pelo que genomas que vagueiam indefinidamente pelo mapa maximizam-no sem nunca entrar no ninho (observaram-se retornos brutos $\approx 88\,000$ com zero chegadas) — parar no ninho, a pré-condição da tarefa, corta precisamente o rendimento por passo. O *homing* elimina esta patologia por construção: depende apenas dos estados *extremos* do episódio (posição inicial vs. final) e não do caminho percorrido, pelo que vaguear não o aumenta; a única forma de o maximizar é *terminar* junto ao ninho. A média sobre $E = 4$ episódios com *seeds* fixas reduz a variância estocástica da avaliação, estabilizando o sinal de seleção entre gerações.

Mutação Gaussiana *Element-wise*: Em vez de perturbar a totalidade dos $|\theta|$ pesos em simultâneo — o que degradaria irrecuperavelmente a diversidade populacional em redes de grande dimensão —, implementa-se uma variante *esparsa* da mutação gaussiana da Equação 2.1, controlada por uma máscara binária $\mathbf{m} \in \{0, 1\}^{|\theta|}$:

$$\theta_j^{(\text{filho})} = \theta_j^{(\text{pai})} + m_j \cdot \varepsilon_j, \quad m_j \sim \text{Bernoulli}(p_{\text{mut}}), \quad \varepsilon_j \sim \mathcal{N}(0, \sigma^2) \quad (5.3)$$

com $p_{\text{mut}} = 0,1$ e σ o passo de mutação corrente (Equação 2.2). Apenas 10% dos pesos são perturbados por operação, preservando a estrutura do controlador pai enquanto introduz variações localizadas. É a mutação gaussiana do algoritmo genético de Such et al. (2017), aqui restrita a uma fração dos pesos; distingue-se das estratégias evolutivas de Salimans et al. (2017), que combinam todas as perturbações numa estimativa do gradiente em vez de selecionar entre elas [10], [17].

Parametrização do *Sigma Decay*: O passo de mutação segue a temperagem geométrica da Equação 2.2, com $\sigma_0 = 0,1$, $\rho = 0,999$ e $\sigma_{\min} = 0,03$. Este decaimento assegura que nas fases iniciais o algoritmo explora amplamente a superfície $J(\theta)$, convergindo progressivamente para refinamento local do melhor ótimo encontrado. O patamar mínimo $\sigma_{\min}$ é deliberadamente conservador: deixar σ colapsar para valores residuais fazia o campeão sobre-ajustar-se às *seeds* de avaliação (*overfitting* de seleção), degradando a generalização medida na avaliação determinística.

Elitismo e Seleção por Ordenação: Após avaliar todos os K genomas, estes são ordenados por $J(\theta_k)$ em ordem decrescente. Os $n_e = \max(3, \lfloor 0,2 \cdot K \rfloor)$ melhores genomas (elites) são preservados intactos na nova geração, garantindo que o melhor comportamento descoberto nunca se perde por mutações desfavoráveis. Os restantes $K - n_e$ genomas são gerados como mutações de cópias aleatórias dos elites. Para $K = 30$, preservam-se $n_e = 6$ genomas elite por geração.

5.2.2. Ciclo de Treino Evolutivo

O treino evolutivo segue o ciclo *avaliação* $\rightarrow$ *seleção* $\rightarrow$ *mutação*, implementado em `evo_trainer_3d.py`:

- (1) **Avaliação Paralela:** Os $K = 30$ genomas são avaliados em paralelo, com um processo por genoma, via `multiprocessing.Pool` (o número de processos é o mínimo entre o tamanho da população e o número de núcleos disponíveis; nos servidores de cálculo utilizados, com 64 núcleos, os 30 genomas são avaliados simultaneamente). Cada processo executa $E = 4$ episódios independentes (com *seeds* fixas ao longo da evolução) e devolve $J(\theta_k)$ (Equação 5.2).
- (2) **Mecanismo de Guilhotina:** Se a recompensa acumulada após 150 passos for inferior ao limiar de -200 , o episódio é terminado prematuramente e o genoma recebe uma penalidade adicional de -1000 . Este mecanismo evita desperdiçar ciclos de CPU em genomas claramente disfuncionais.
- (3) **SeleçãoELITista:** Os $n_e = 6$ melhores genomas são copiados intactos. Os restantes 24 são gerados por mutação *element-wise* (Equação 5.3) de cópias aleatórias dos elites.

- (4) **Temperagem de σ :** A cada geração, σ é atualizado pela Equação 2.2, com decaimento de 0,1% por geração e patamar mínimo $\sigma_{\min} = 0,03$.
- (5) **Cr terio de Paragem:** O treino prossegue at  esgotar o or amento temporal configurado (195 minutos por execu  o na campanha final), garantindo dura  o determin stica e compar vel com os algoritmos MARL.

O ciclo   *geracional*: a popula  o s    substituída depois de avaliados todos os genomas. Uma variante n o-geracional (*steady-state*), em que cada genoma avaliado substitui de imediato um membro da popula  o sem esperar pelos restantes, *n o foi implementada nem testada* nesta disserta  o: todas as campanhas reportadas usam o ciclo descrito acima. N o se sabe, por isso, se essa variante aproveitaria melhor o paralelismo — a guilhotina termina cedo a avalia  o de alguns genomas, e numa gera  o os processos que acabam primeiro ficam   espera do mais lento — nem que efeito teria na din mica de sele  o. Fica como trabalho futuro (Sec  o 7.5).

5.2.3. Procura por Novidade (*Novelty Search*) para Cen rios Enganadores (*Deceptive*)

O *shaping* de *homing* da Equa  o 5.2 pressup e que descer o potencial de navega  o conduz   solu  o. O cen rio Porta Cooperativa com Alternativa (Figura 4.3) viola deliberadamente este pressuposto: o campo geod sico trata a porta como transpon vel, pelo que o gradiente de *homing* conduz o enxame diretamente   porta *fechada* — um  timo local potencial para a evolu  o objetiva, dado que o percurso alternativo exige, primeiro, *subir* o potencial. Para este tipo de paisagem de recompensa enganadora (*deceptive*), implementou-se a *procura por novidade* (*Novelty Search*) de Lehman e Stanley [20], que substitui parcialmente a press o pelo objetivo por uma press o pela *diversidade comportamental*:

- **Caracteriza  o comportamental (BC):** cada genoma   resumido pelo vetor $\mathbf{b}(\theta_k) \in \mathbb{R}^2$ — a posi  o final (x, y) do centroide do enxame, m dia dos E epis dios de avalia  o. Este descritor captura *para onde* a pol tica transporta o enxame, permitindo que genomas que exploram regi es novas do mapa (por exemplo, o percurso alternativo) se distingam mesmo sem qualquer chegada.
- **Novidade:** $v(\theta_k)$   a dist ncia euclidiana m dia de $\mathbf{b}(\theta_k)$ aos seus $k_{nn} = 10$ vizinhos mais pr ximos no conjunto formado pela popula  o atual e por um *arquivo* de comportamentos passados (os 3 de maior novidade de cada gera  o; capacidade m xima de 1000, gerida em regime FIFO).
- **Sele  o mista:** os genomas s o ordenados pelo *score* $s(\theta_k) = (1-w) \cdot \hat{J}(\theta_k) + w \cdot \hat{v}(\theta_k)$, onde $\hat{\cdot}$ denota normaliza  o *min-max* intra-gera  o e $w = 0,5$ pondera objetivo e novidade em partes iguais. Com $w = 0$ o mecanismo desliga-se e o algoritmo reduz-se *exatamente* ao ciclo evolutivo base — a implementa  o   *config-driven*, permitindo compara  es controladas.
- **Salvaguarda do objetivo:** a novidade influencia apenas a *sele  o* dos progenitores; o campe o gravado e reportado   sempre o melhor genoma segundo o *objetivo* $J(\theta_k)$. Garante-se assim que o modelo final resolve a tarefa, e n o apenas que   comportamentalmente “diferente”.

O impacto desta extensão é avaliado no Capítulo 6 em três frentes: uma comparação emparelhada preliminar no cenário *deceptive*; duas campanhas multi-execução com *orçamento igualado* ao da campanha final (7 execuções de 195 minutos por braço) nos dois cenários onde a otimização objetiva se mostrou vulnerável à decepção — a Porta com Alternativa (decepção por desenho) e o Muro em U (decepção espacial, comportamento bimodal); e uma campanha final *pré-registada* de *dosagem adaptativa* do peso (w decai geometricamente após a descoberta sustentada, regressando a seleção ao objetivo puro), corrida nos sete cenários — respondendo à QI6 (Secção 1.4).

5.3. Plano de Experiências e Tratamento Estatístico

O *pipeline* experimental é segmentado em cinco avaliações prioritárias (Quadro 5.1) por forma a assegurar validação e significância dos resultados reportados na tese.

QUADRO 5.1. Plano de testes experimentais e métricas associadas.

Prior.	Experiência	Duração	Objetivo Científico	Métrica
1	Treino longo (Sandbox)	24h	Convergência máxima dos 3 modelos no cenário base.	P_{task} (Treino)
2	Avaliação Determinística (eval_all.py)	5 min	Desempenho sem exploração (20 episódios).	P_{task} (Eval)
3	Benchmark Estatístico (execuções múltiplas)	3–7 dias	Validação de hipóteses estatísticas ($p < 0,05$).	Mann-Whitney U / δ de Cliff
4	Robustez a Falhas (Remover 10% agentes)	1h + Treino	Medir resiliência à perda súbita de robôs a meio do episódio.	R_{robust}
5	Escalabilidade Zero-Shot ($N \in \{10, 50, 100\}$)	3h	Testar generalização do grafo dinâmico sem re-treino.	S_{scale}

As métricas da última coluna definem-se no Capítulo 6, junto dos resultados que medem: P_{task} é a taxa de sucesso — a fração de episódios com pelo menos uma chegada (Secção 6.2) —, R_{robust} é a retenção de chegadas quando 10% dos agentes falham a meio do episódio (Secção 6.13) e S_{scale} é a eficiência por agente ao variar a dimensão do enxame sem retreino (Secção 6.12).

As diferenças entre algoritmos são avaliadas por um protocolo *não-paramétrico* — as distribuições de chegadas por episódio violam sistematicamente a normalidade (massas em zero, bimodalidade), inviabilizando o teste t clássico como teste principal. A unidade estatística é a execução de treino: cada execução é resumida pela média das chegadas nos seus episódios de avaliação determinística (*seeds* comuns aos algoritmos comparados), e as execuções de dois algoritmos comparam-se com o teste de *Mann-Whitney U*. Cada comparação reporta ainda o δ de *Cliff* como tamanho de efeito não-paramétrico (em $[-1, 1]$, onde $|\delta| \geq 0,474$ se convencionou como efeito grande), com o limiar convencional $p < 0,05$. Todo o protocolo está automatizado em `scripts/statistical_tests.py`, que gera os quadros e figuras do Capítulo 6 diretamente dos CSV de avaliação.

CAPÍTULO 6

Resultados Experimentais e Discussão

Este capítulo apresenta e analisa os resultados da campanha experimental final. Confronta-se o paradigma de *Multi-Agent Reinforcement Learning* (PPO e SAC) com o de otimização bioinspirada (controlador neuroevolutivo com atenção sobre grafo), com tratamento estatístico, para responder às questões de investigação (Secção 1.4).

6.1. Metodologia de Avaliação e Notas de Leitura

Os resultados reportados provêm de uma *campanha de treino contínua de sete dias* nos servidores de cálculo, executada com a configuração final do sistema (recompensa simplificada da Secção 4.3, *fitness* de *homing* da Eq. 5.2, LiDAR de 8 m). Antes da análise, importa fixar as convenções de leitura:

- **Execuções independentes:** Cada combinação (algoritmo, cenário) foi treinada em 7 *execuções independentes* com sementes distintas, nos 7 *cenários* do protocolo ($3 \times 7 \times 7 = 147$ treinos). As curvas reportam a média entre execuções e a banda sombreada o desvio padrão.
- **Orçamentos de treino por paradigma:** cada execução do controlador evolutivo dispôs de 195 minutos (população de 30 genomas avaliada em paralelo), e cada execução do PPO/SAC de 48 minutos (16 ambientes vetorizados). Os orçamentos foram calibrados, com base nas campanhas exploratórias, para levar cada paradigma ao seu planalto de convergência — e, na maioria das células, é o que se observa. Uma verificação *a posteriori* sobre as curvas (comparação emparelhada, por execução, entre a média do último quinto do orçamento e a do quinto anterior; `scripts/verificar_planalto.py`) mostra, porém, que 7 das 21 combinações ainda subiam de forma significativa no fim — e duas delas de forma marcada: o SAC no Gargalo (+112%) e na Porta com Alternativa (+105%), seguidos do GNN no Muro em U (+33%) e do PPO no Quatro Salas (+27%). A comparação mede, portanto, o *desempenho atingível dentro do orçamento concedido*, e não o desempenho assintótico de cada paradigma nem a eficiência amostral a cómputo igual; nas quatro células assinaladas, os valores do algoritmo em causa devem ser lidos como *limite inferior*. A implicação mais forte é para o SAC nos gargalos físicos: a fragilidade que a Secção 6.6 lhe atribui é indistinguível, com os dados retidos, de sub-treino — os históricos da campanha final guardaram a recompensa episódica mas não a métrica de tarefa por passo, pelo que não é possível decidir se o crescimento final corresponde a mais chegadas ou apenas a mais *shaping*. A assimetria de cómputo (favorável ao evolutivo) é retomada na discussão (Secção 6.15).

- **Avaliação determinística por execução:** cada um dos 7 modelos finais de cada célula foi avaliado em 20 *episódios determinísticos emparelhados* (*seeds* comuns aos três algoritmos), num total de 140 episódios por combinação (algoritmo, cenário). A *unidade estatística é a execução*: os testes entre algoritmos usam as médias por execução ($n = 7$ por grupo, Mann-Whitney U + δ de Cliff), evitando inflacionar n com episódios correlacionados do mesmo modelo.
- **Métrica de tarefa pura (P_{task}):** Para além da recompensa total (que inclui *shaping*), reportam-se a taxa de sucesso (fração de episódios com pelo menos uma chegada) e as chegadas por episódio, obtidas em avaliação determinística, por serem as medidas não-enviesadas e *comparáveis* do cumprimento da missão.
- **Escalas não-comparáveis entre paradigmas:** O GNN usa *fitness* evolutiva e o PPO/SAC recompensa episódica; comparações de valor absoluto entre curvas de treino de paradigmas distintos devem ser evitadas, privilegiando as métricas de tarefa da avaliação determinística. O eixo das abcissas das curvas é normalizado para [0%, 100%] do orçamento de cada algoritmo. As curvas medem, portanto, o progresso de cada algoritmo *no seu próprio critério*, e servem para ler convergência e estabilidade dentro de cada um — não para comparar paradigmas entre si. A medida externa aos algoritmos, igual para os três e coerente com a métrica de desempenho (as chegadas por episódio em avaliação determinística), só existe nesta campanha *no fim* do treino: os históricos guardaram o critério interno de cada algoritmo e não essa medida ao longo do treino, e não foram arquivados modelos intermédios que permitissem reconstruí-la (Secção 7.3). A comparação entre paradigmas faz-se, por isso, sobre os modelos finais (Secção 6.2).

6.2. Desempenho de Tarefa por Cenário (P_{task})

A métrica central de comparação é o desempenho de *tarefa* em avaliação determinística: a taxa de sucesso (P_{task} , fração de episódios com pelo menos uma chegada) e o número de chegadas por episódio. Ao contrário da recompensa de treino — que inclui *shaping* e mistura escalas entre paradigmas (*fitness* evolutiva vs. recompensa episódica) —, estas métricas são diretamente comparáveis entre os três algoritmos e medem o cumprimento efetivo da missão. Os valores reportados agregam os 140 *episódios determinísticos emparelhados* de cada combinação (7 execuções $\times$ 20 episódios, *seeds* comuns aos três algoritmos).

O Quadro 6.1 e as Figuras 6.1–6.2 mostram um panorama maioritariamente resolvido, mas com fragilidades concentradas — e diferentes — por algoritmo: 15 das 21 combinações algoritmo–cenário atingem 100% de sucesso em todas as execuções e episódios; as seis restantes correspondem a células em que uma ou mais execuções de treino falham por completo, pelo que a taxa média reportada esconde uma distribuição tudo-ou-nada entre execuções (analisada na Secção 6.6). A Porta Cooperativa e o cenário *deceptive* com passagem alternativa estão resolvidos pelos três algoritmos. O PPO é o mais consistente: 100% de sucesso em seis dos sete cenários, com a menor variância entre execuções (desvios padrão de 0,4 a 4,0 chegadas/ep fora do Muro em U). O *controlador evolutivo* (GNN) resolve os quatro cenários de gargalo —

Gargalo, Quatro Salas e as duas Portas Cooperativas — em *todas* as execuções, igualando o PPO no Gargalo (121,4 chegadas/ep em média, empate estatístico) e destacando-se no Quatro Salas (59,8, cerca de $1,8\times$ o melhor método de gradiente) e na Porta Cooperativa (69,8, acima de ambos), mas exibe execuções degeneradas precisamente nos cenários *sem* paredes (Sandbox: 5/7 execuções funcionais; Percepção Cooperativa: 6/7). O SAC mantém 100% nos cenários cooperativos e no Quatro Salas, mas é o mais frágil nos gargalos físicos: no Gargalo apenas 5/7 execuções convergem (com magnitude muito dispersa, $41,4 \pm 36,8$) e no Muro em U apenas 2/7.

QUADRO 6.1. Avaliação determinística (7 execuções $\times$ 20 episódios emparelhados): taxa de sucesso (P_{task} , sobre os 140 episódios) e chegadas por episódio (média $\pm$ desvio padrão entre execuções). A negrito, a melhor média de chegadas de cada cenário.

Cenário	GNN		PPO		SAC	
	Sucesso	Chegadas	Sucesso	Chegadas	Sucesso	Chegadas
Sandbox	85,7%	$38,3 \pm 31,0$	100%	$71,5 \pm 1,0$	100%	$69,2 \pm 1,9$
Muro em U	42,9%	$24,5 \pm 32,6$	70,7%	$39,6 \pm 36,7$	33,6%	$9,0 \pm 15,1$
Gargalo	100%	$121,4 \pm 19,9$	100%	$123,2 \pm 1,2$	72,1%	$41,4 \pm 36,8$
Quatro Salas	100%	$59,8 \pm 13,2$	100%	$33,6 \pm 3,8$	100%	$31,8 \pm 3,3$
Porta Cooperativa	100%	$69,8 \pm 0,9$	100%	$67,1 \pm 3,7$	100%	$62,1 \pm 2,5$
Percepção Cooperativa	91,4%	$19,0 \pm 8,7$	100%	$15,3 \pm 0,4$	100%	$16,1 \pm 0,8$
Porta com Alternativa	100%	$86,7 \pm 2,0$	100%	$85,3 \pm 4,0$	100%	$68,6 \pm 3,4$

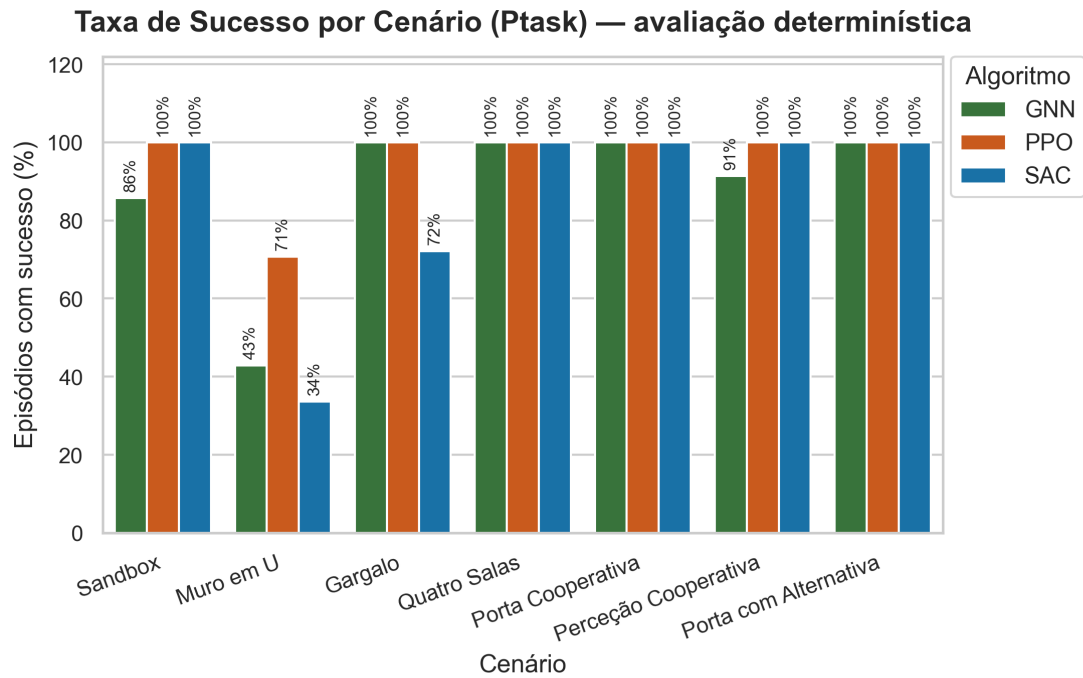
O Muro em U é o único cenário que nenhum algoritmo resolve de forma fiável: o desempenho é *bimodal* nos três paradigmas — cada execução ou aprende o desvio ao beco e atinge 100% de sucesso, ou não o aprende de todo (GNN 3/7 execuções, PPO 4/7, SAC 2/7) — e nenhuma diferença entre algoritmos é estatisticamente significativa (Secção 6.15). A fronteira de dificuldade da campanha final já não separa, portanto, os cenários de gargalo dos de espaço aberto (essa fronteira foi eliminada pela *fitness* de *homing* e pela recompensa simplificada, como analisado adiante): o que resta é a *armadilha do desvio comprido* do Muro em U, transversal aos dois paradigmas.

6.3. Análise Qualitativa da Navegação: Mapas de Calor

Para fundamentar a opção pelo potencial geodésico no termo de progresso e diagnosticar o comportamento de navegação, recorre-se a dois tipos de mapa de calor. A Figura 6.3 contrasta o potencial euclidiano com o geodésico no Muro em U: as curvas de nível euclidianas atravessam a parede (atraindo o agente para o interior do beco), enquanto as geodésicas a contornam, atribuindo gradiente positivo ao desvio correto. A Figura 6.4 mostra a densidade de ocupação dos robôs — zonas brilhantes coladas a uma parede revelam agentes presos, ao passo que um fluxo que contorna o obstáculo indica navegação resolvida.

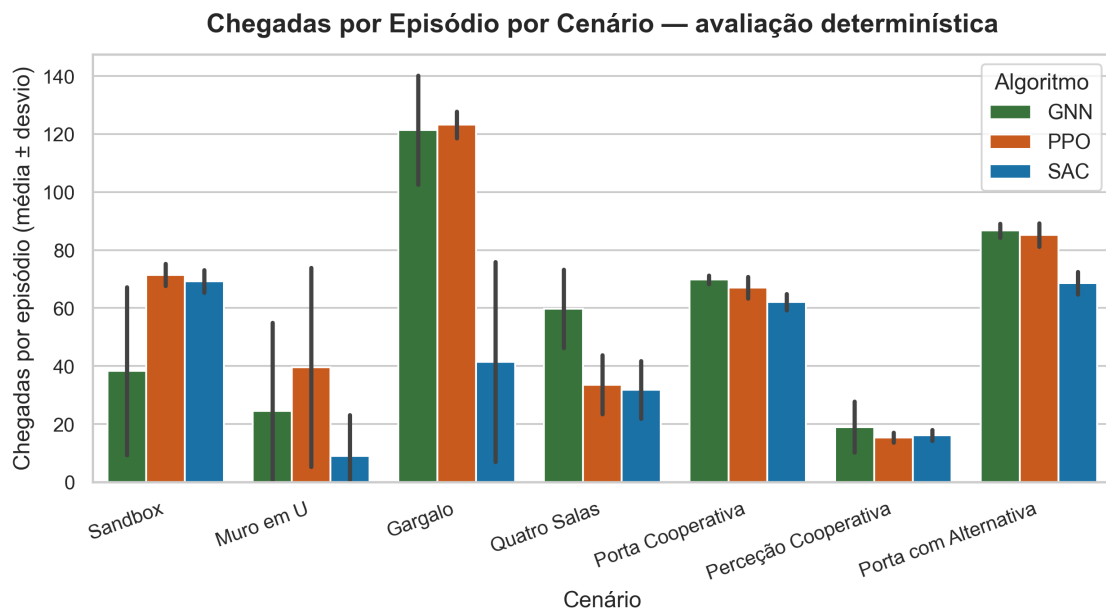
6.4. Convergência e Desempenho no Cenário Base (Sandbox)

O cenário Sandbox — arena aberta, sem paredes interiores — isola a cooperação no ninho da componente de navegação por obstáculos, servindo de referência de otimalidade. A Figura 6.5 mostra que os dois métodos de gradiente convergem para políticas funcionais e *repetíveis*: as



Sucesso = pelo menos 1 chegada ao ninho no episódio. Métrica de tarefa, comparável entre algoritmos (ao contrário do reward de treino).

FIGURA 6.1. Taxa de sucesso (P_{task}) por cenário, em avaliação determinística (7 execuções $\times$ 20 episódios emparelhados por algoritmo).



Média de chegadas ao ninho (food_collected) por episódio; barras de erro = desvio padrão entre episódios. Mede magnitude do desempenho, não só sucesso/falha.

FIGURA 6.2. Chegadas por episódio, por cenário. Barra = média sobre os 140 episódios de avaliação (7 execuções $\times$ 20); barra de erro = desvio padrão *entre episódios*, que nos cenários de convergência bimodal (Muro em U, Sandbox-GNN, Gargalo-SAC) chega a exceder a própria média — a dispersão que lhe está por trás é a dispersão entre execuções da Secção 6.6. Mede a magnitude do desempenho, para além do sucesso binário.

Potencial de navegação — Beco Sem Saída (Muro U)
As linhas de nível mostram para onde o progress reward 'puxa' o robô

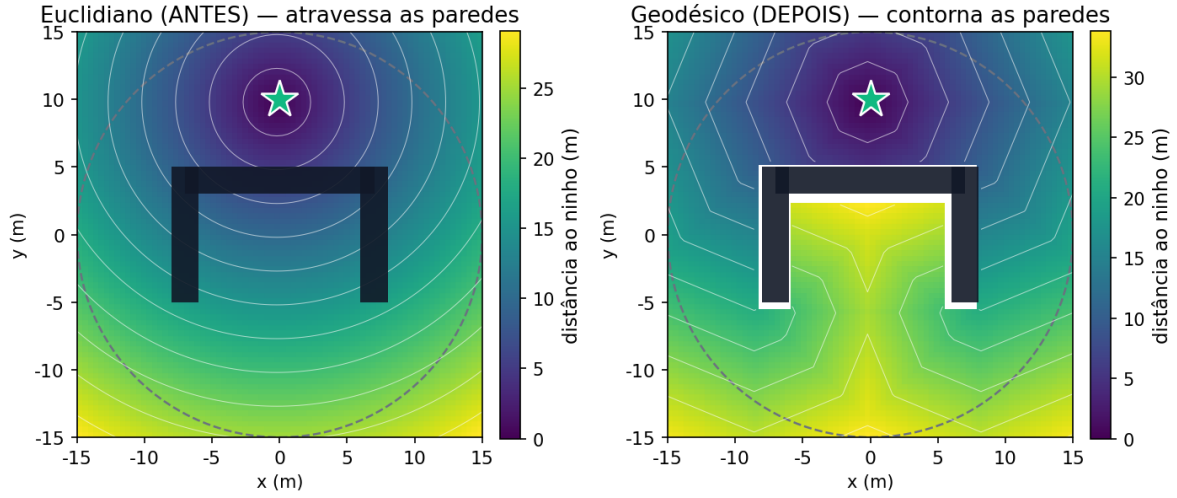


FIGURA 6.3. Potencial de navegação no Muro em U: euclidiano (esquerda) vs. geodésico (direita). As curvas de nível indicam a direção para a qual o termo de progresso atrai o agente; só o geodésico contorna a barra do U.

bandas de desvio padrão entre as 7 execuções são estreitas, e o desempenho final de tarefa é praticamente idêntico (PPO $71,5 \pm 1,0$ chegadas/ep; SAC $69,2 \pm 1,9$), com o SAC (*off-policy*) a subir mais cedo, beneficiando da reutilização de experiência do *replay buffer*. O GNN evolutivo apresenta aqui o seu comportamento mais irregular de toda a campanha: quatro das sete execuções convergem para políticas competitivas (61,6 a 64,7 chegadas/ep), duas degeneram por completo (< 1 chegada/ep) e uma fica num regime intermédio (15,8 chegadas/ep, com sucesso em todos os episódios mas uma fração da magnitude dos melhores), arrastando a média para $38,3 \pm 31,0$. É um resultado à primeira vista paradoxal — o cenário mais simples é o menos fiável para o evolutivo — que se explica pela ausência de gradiente geodésico informativo em campo aberto: sem paredes que canalizem o *homing*, a paisagem de *fitness* tem múltiplos ótimos locais de vaguear, e a fuga a esses ótimos depende da sorte da inicialização (Secção 6.15).

O Quadro 6.2 sintetiza o desempenho de avaliação neste cenário.

QUADRO 6.2. Desempenho no cenário Sandbox (avaliação determinística; chegadas/ep em média $\pm$ desvio padrão entre as 7 execuções; P_{task} sobre os 140 episódios).

Algoritmo	Chegadas/ep	P_{task}	Execuções funcionais
GNN (Evolutivo)	$38,3 \pm 31,0$	85,7%	5/7
PPO	$71,5 \pm 1,0$	100%	7/7
SAC	$69,2 \pm 1,9$	100%	7/7

“Execuções funcionais” = execuções com 100% de sucesso nos 20 episódios de avaliação.

6.5. Desempenho Comparativo por Cenário (Curvas de Aprendizagem)

As curvas de aprendizagem por cenário (Figuras 6.6–6.11) detalham a dinâmica de treino subjacente aos resultados de tarefa, agora com bandas de variância reais (7 execuções). Importa

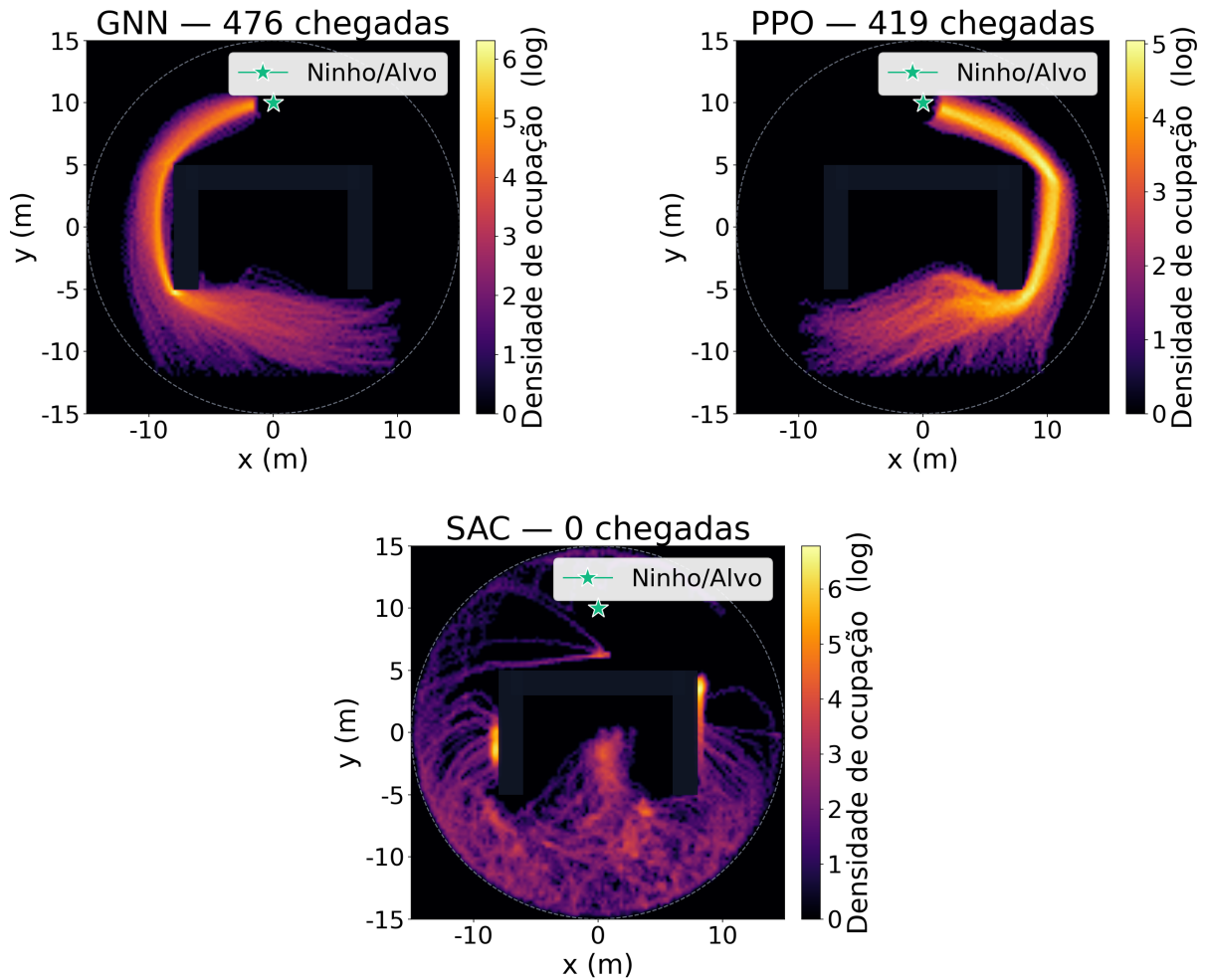


FIGURA 6.4. Densidade de ocupação dos robôs no Muro em U (GNN e PPO em cima, SAC em baixo; 6 episódios por painel, escala logarítmica). O GNN contorna a barra do U pela esquerda e o PPO pela direita — em ambos, um fluxo contínuo até ao ninho, com 476 e 419 chegadas. O SAC não resolve o cenário: a ocupação concentra-se *colada* às faces interiores das paredes verticais, o padrão de agentes presos, e nenhum dos seis episódios tem uma chegada. A escala de cor é normalizada por painel, pelo que as cores comparam a distribuição *dentro* de cada algoritmo, não a magnitude entre eles.

uma ressalva de leitura: os eixos verticais não são comparáveis entre paradigmas — o GNN é seleccionado por *fitness* (dominada pelo termo das chegadas, $\times 10^4$), ao passo que PPO e SAC reportam recompensa episódica com *shaping* —, pelo que cada painel deve ser lido como a trajetória *interna* de cada algoritmo, e não como uma corrida no mesmo eixo.

Nos cenários de navegação por gargalos (*Gargalo*, Figura 6.7; *Quatro Salas*, Figura 6.8; *Porta Cooperativa*, Figura 6.9) o padrão da campanha final é o oposto do observado nas campanhas exploratórias: a *fitness* do GNN sobe de forma sustentada e traduz-se em chegadas na avaliação — a assinatura do antigo *fitness exploitation* (subida de *shaping* sem tarefa) desapareceu com a *fitness* de *homing*. Entre os métodos de gradiente, o PPO converge de forma quase determinística (bandas estreitas), enquanto o SAC exhibe bandas largas no Gargalo, refletindo as execuções que não convergem. No *Muro em U* (Figura 6.6), as bandas largas dos

Curvas de Aprendizagem — Sandbox (Arena Aberta)

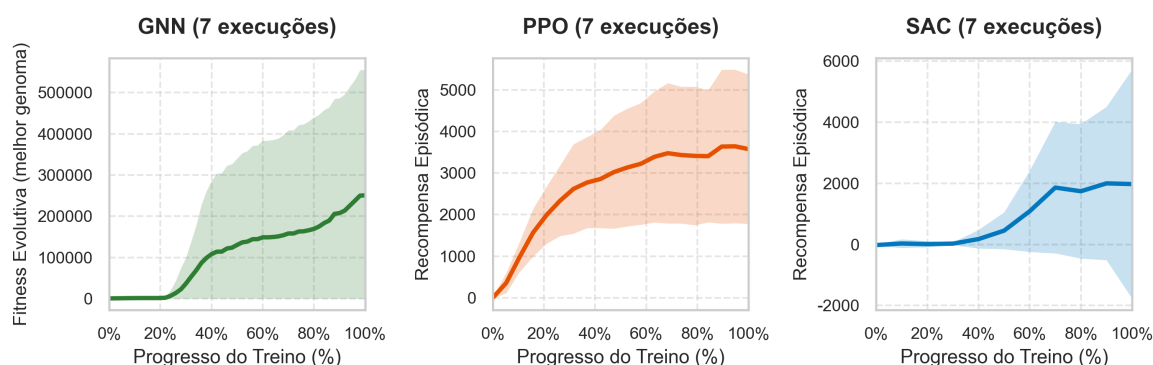


Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.5. Curvas de aprendizagem dos três algoritmos no cenário Sandbox. O eixo das abcissas está normalizado (0–100% do orçamento de treino); linha = média de 7 execuções, banda = ± 1 desvio padrão. Cada execução é interpolada numa grelha comum de progresso, porque as execuções não registam nos mesmos passos; os painéis são separados porque as métricas não são comparáveis entre paradigmas (*fitness* evolutiva no GNN, recompensa episódica no PPO e no SAC) — só o eixo das abcissas o é. Não há, nestas curvas, uma medida de progresso externa aos algoritmos e comum aos três; a comparação entre paradigmas faz-se sobre os modelos finais (Secção 6.1). Vale igualmente para as curvas dos restantes cenários.

três algoritmos são o retrato da bimodalidade já referida: a média das curvas mistura execuções resolvidas e falhadas. Na *Percepção Cooperativa* (Figura 6.10) os três algoritmos progridem, com o GNN a atingir a maior magnitude média. Finalmente, na *Porta Cooperativa com Alternativa* (Figura 6.11) — o cenário *deceptive* introduzido para a QI6 — os três algoritmos aprendem o desvio pelo corredor alternativo em todas as execuções, com o GNN e o PPO em vantagem clara de magnitude sobre o SAC (86,7 e 85,3 vs. 68,6 chegadas/ep).

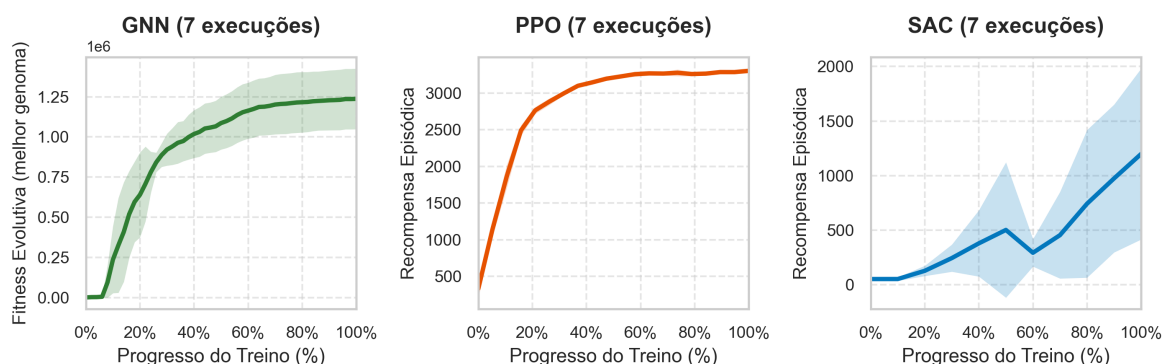
Curvas de Aprendizagem — Muro em U (Beco Sem Saída)



Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.6. Curvas de aprendizagem no cenário Beco Sem Saída (Muro em U).

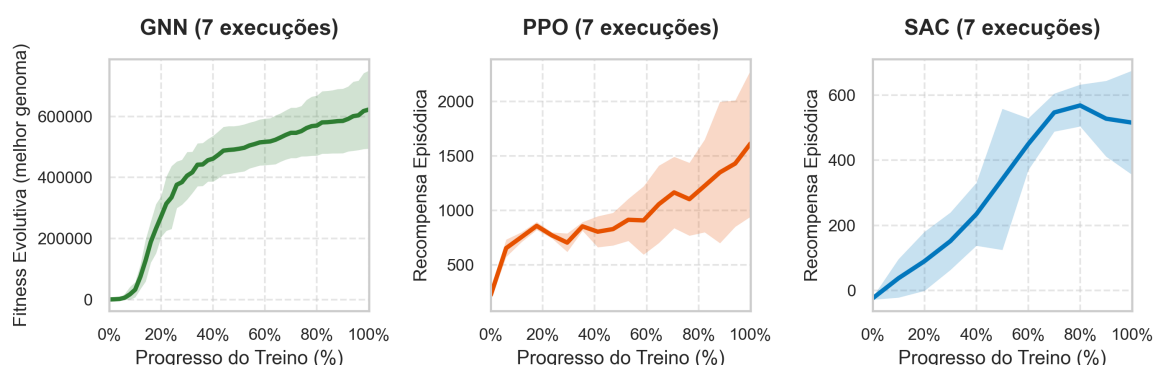
Curvas de Aprendizagem — Gargalo (Porta Estreita)



Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.7. Curvas de aprendizagem no cenário Gargalo.

Curvas de Aprendizagem — Quatro Salas (Labirinto)



Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

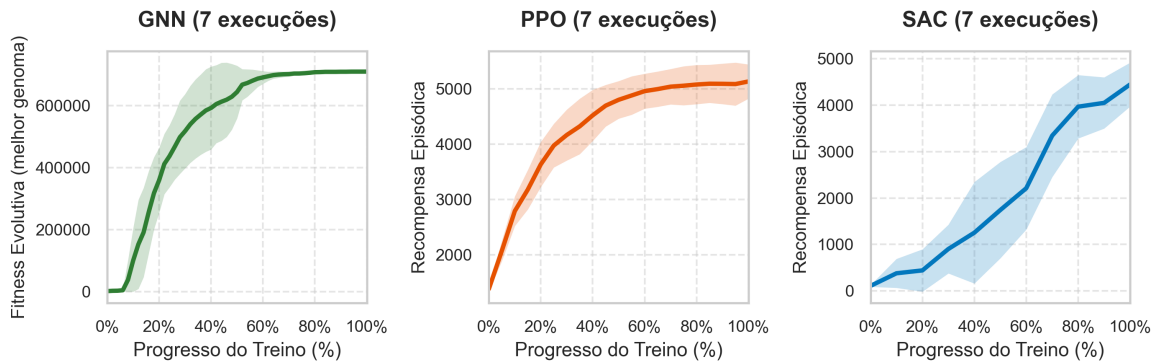
FIGURA 6.8. Curvas de aprendizagem no cenário Quatro Salas (Labirinto).

6.6. Fiabilidade e Variância entre Execuções

A análise da fiabilidade de treino requer múltiplas execuções independentes por configuração, de modo a distinguir o desempenho sistemático do ruído de inicialização. As Figuras 6.12–6.15 mostram, para cada cenário, *um ponto por execução* — as chegadas por episódio dessa execução (média dos 20 episódios de avaliação; 7 pontos por algoritmo) — com a média reportada na tese marcada a traço vertical. A métrica é de tarefa, na mesma unidade para os três algoritmos, ao contrário dos *scores* de treino. Com $n = 7$ mostram-se as execuções um a um em vez de um diagrama de caixa: nestes dados a distribuição é frequentemente *bimodal* — execuções que resolvem o cenário e execuções que ficam a zero, sem nada pelo meio — e uma caixa a cobrir todo esse intervalo sugeriria densidade onde não existe uma única execução.

A leitura por cenário quantifica o que as secções anteriores anteciparam. Nos cenários *cooperativos* (Porta Cooperativa, Porta com Alternativa) e no *Quatro Salas*, as sete execuções

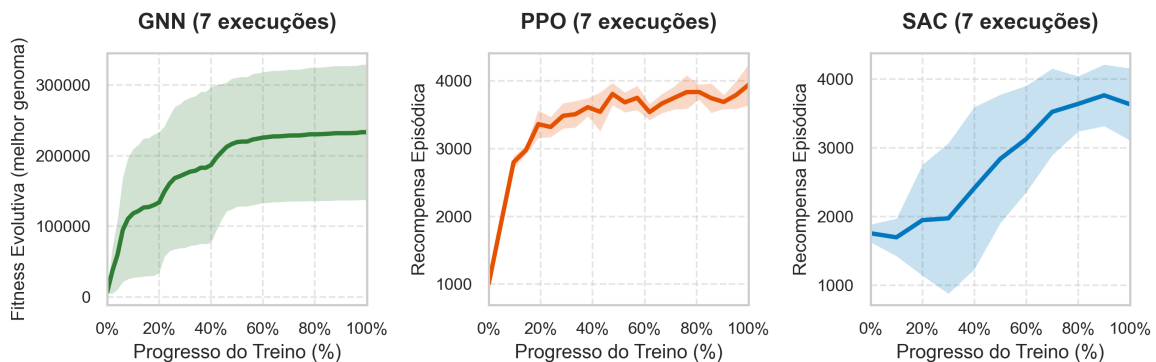
Curvas de Aprendizagem — Porta Cooperativa (3 Robôs)



Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.9. Curvas de aprendizagem no cenário Porta Cooperativa.

Curvas de Aprendizagem — Percepção Cooperativa (Alvo Móvel)

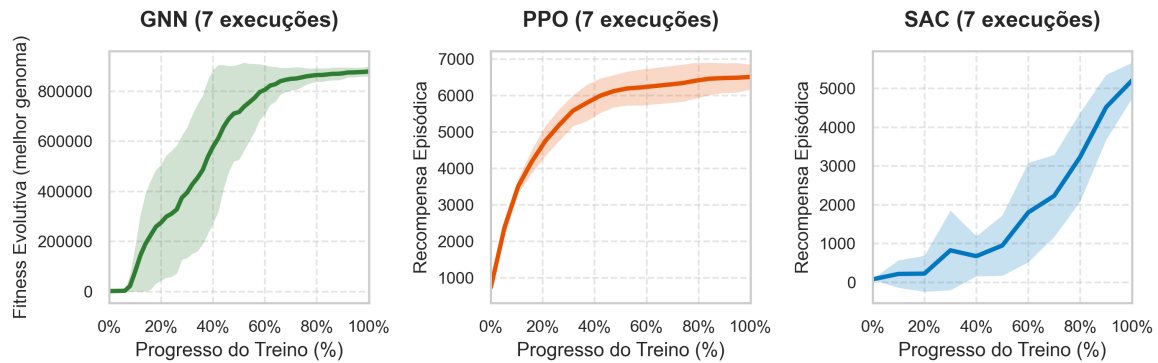


Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.10. Curvas de aprendizagem no cenário Percepção Cooperativa (Alvo Móvel).

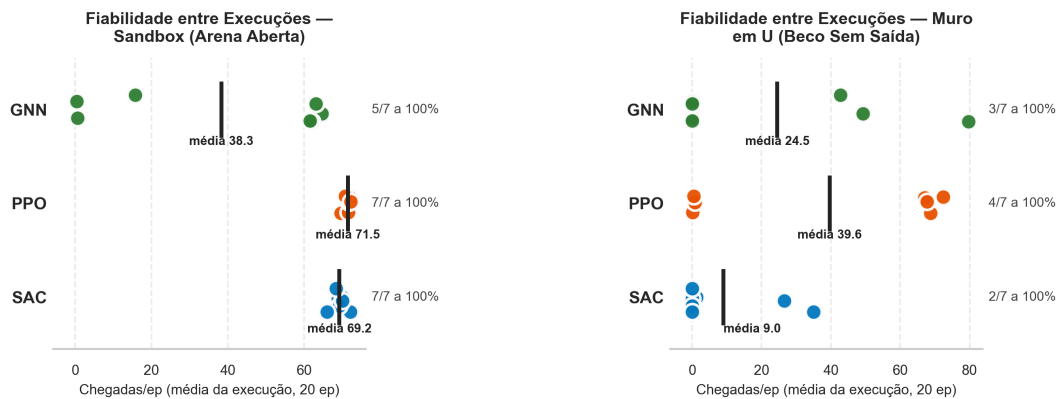
dos três algoritmos ficam agrupadas e afastadas do zero — treinar é *fiável* — e o GNN posiciona-se sistematicamente acima do SAC (e, no Quatro Salas, muito acima de ambos os métodos de gradiente). No *Gargalo*, PPO e GNN são fiáveis (7/7 execuções), mas as execuções do SAC espalham-se de 0 a 88 chegadas/ep — o treino é uma lotaria. No *Sandbox* inverte-se: PPO/SAC agrupados, GNN com duas execuções degeneradas. No *Muro em U*, os três algoritmos têm execuções a zero: nenhum garante a convergência, e só o PPO tem a maioria das execuções funcionais (4/7). Esta análise por execução é a base dos testes de significância da Secção 6.15. Onde treinar é uma lotaria, a mitigação prática é repetir o treino e selecionar — o que converte falta de fiabilidade em custo de máquina, e torna o preço de cada execução parte da comparação: está medido, por paradigma, no Quadro 6.7 (Secção 6.14.1).

Curvas de Aprendizagem — Porta com Alternativa (3 Robôs ou desvio)



Linha = média entre as 7 execuções; banda = ± 1 desvio padrão. Eixos verticais não comparáveis entre paradigmas.

FIGURA 6.11. Curvas de aprendizagem no cenário Porta Cooperativa com Alternativa (*deceptive*).



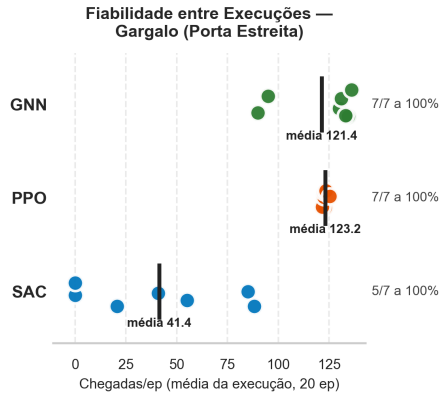
Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

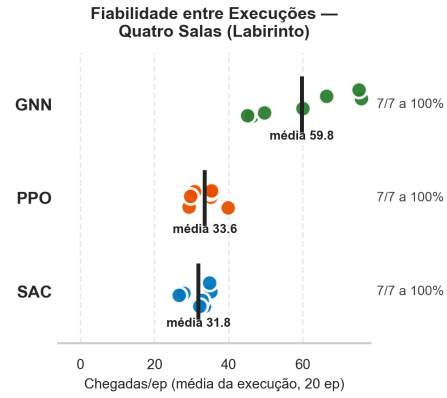
FIGURA 6.12. Chegadas/ep por execução (7 execuções, 20 ep cada): Sandbox (esq.) e Muro em U (dir.). Cada ponto é uma execução independente e a barra vertical é a média reportada na tese; note-se que os dois painéis têm escalas horizontais diferentes.

6.7. Visão Global por Algoritmo

Consolidando o desempenho ao longo dos sete cenários (Figuras 6.16–6.18), emergem três perfis bem definidos. O *GNN evolutivo com fitness de homing* é o especialista em *navegação estruturada*: converge em *todas* as 28 execuções dos quatro cenários de gargalo (Gargalo, Quatro Salas e as duas Portas Cooperativas — incluindo, portanto, o cenário *deceptive*, que o *homing* resolve sem necessidade de pressão por novidade), lidera a magnitude de chegadas no Quatro Salas e na Porta Cooperativa e empata estatisticamente com o PPO no Gargalo e na Porta com Alternativa; o seu ponto fraco são os cenários abertos, onde ocasionalmente degenera. O *PPO* é o *generalista fiável*: converge em todas as execuções de seis cenários com variância mínima, e é o menos afetado pelo Muro em U (4/7). O *SAC* é fiável em espaço aberto e nos cooperativos,

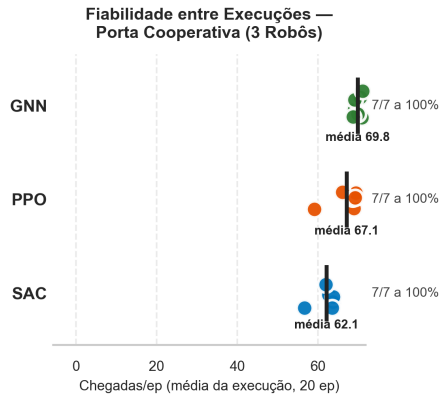


Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

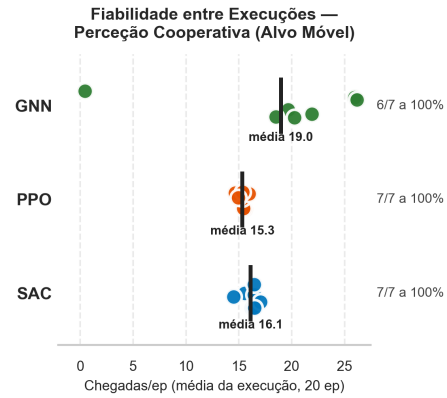


Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

FIGURA 6.13. Chegadas/ep por execução: Gargalo (esq.) e Quatro Salas (dir.).

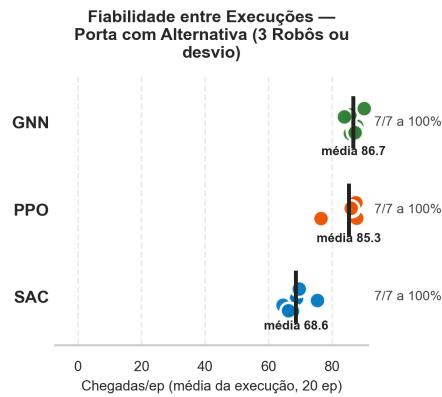


Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.



Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

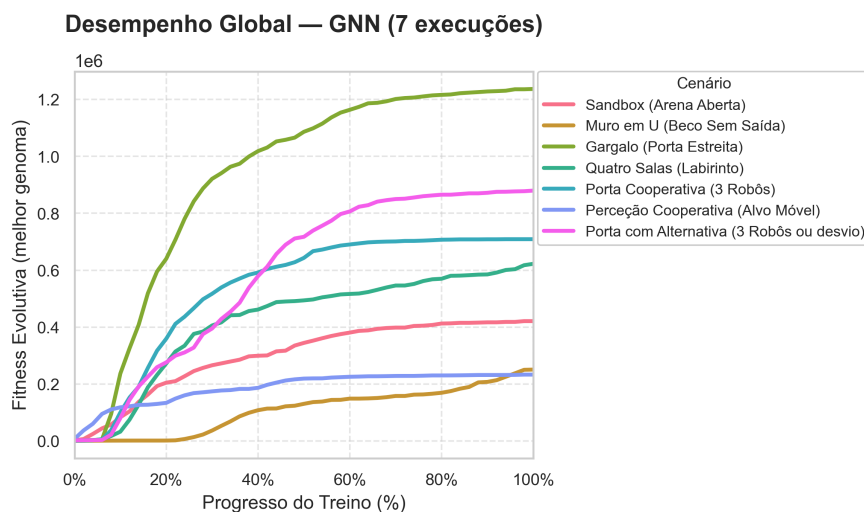
FIGURA 6.14. Chegadas/ep por execução: Porta Cooperativa (esq.) e Percepção Cooperativa (dir.).



Cada ponto = 1 execução independente (7 por algoritmo). A barra vertical é a média reportada na tese.

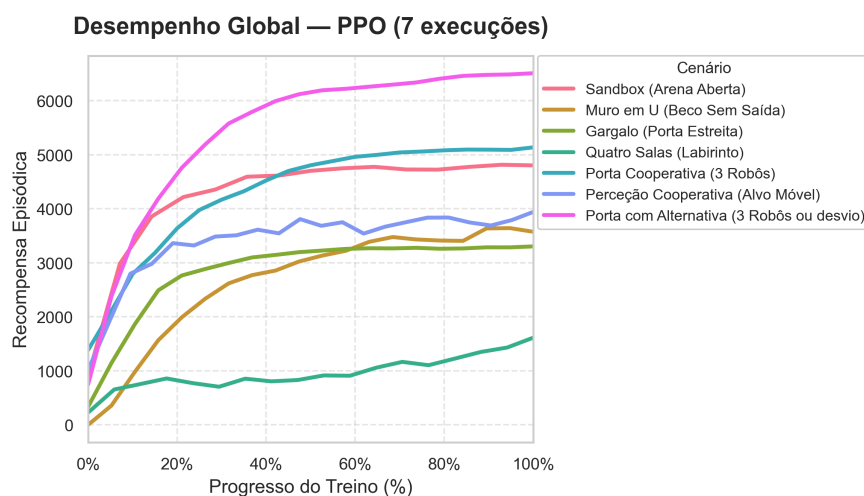
FIGURA 6.15. Chegadas/ep por execução: Porta Cooperativa com Alternativa (*deceptive*).

mas é o mais *sensível aos gargalos físicos*: é o único que falha execuções no Gargalo e tem a pior taxa no Muro em U (2/7).



Linha = média entre as 7 execuções, cada uma interpolada numa grelha comum de 51 pontos de progresso. A dispersão por execução está nos painéis por cenário (curvas de aprendizagem).

FIGURA 6.16. Desempenho do GNN (evolutivo) nos sete cenários (linha = média entre as 7 execuções por progresso de treino).

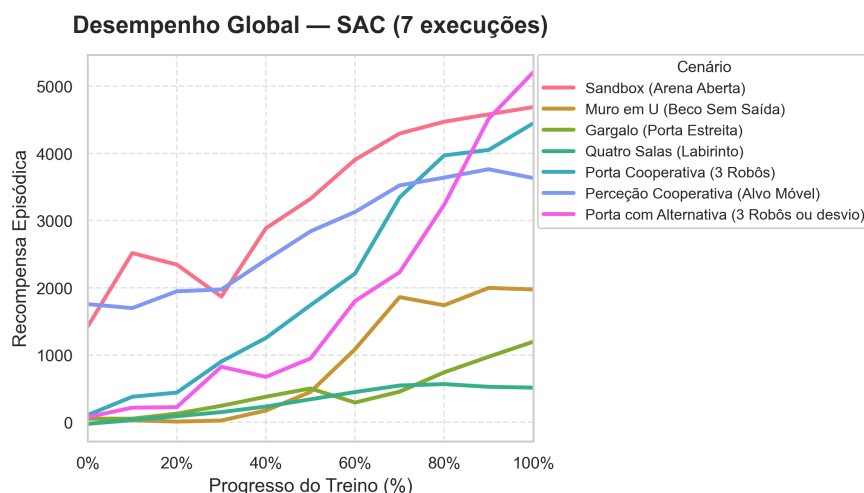


Linha = média entre as 7 execuções, cada uma interpolada numa grelha comum de 22 pontos de progresso. A dispersão por execução está nos painéis por cenário (curvas de aprendizagem).

FIGURA 6.17. Desempenho do PPO nos sete cenários (linha = média entre as 7 execuções por progresso de treino).

6.8. Avaliação Determinística (Tarefa Pura)

A medida mais direta do desempenho na missão é a avaliação em modo determinístico — sem ruído de exploração, com 7 execuções $\times$ 20 episódios emparelhados por configuração —, e está no Quadro 6.1, apresentado na Secção 6.2 porque é dele que parte a leitura por cenário. Recolhido aqui o que as secções intermédias acrescentaram, os seus valores sustentam três leituras: a competitividade do controlador evolutivo nos cenários de navegação, a consistência do PPO e a fragilidade localizada do SAC nos gargalos.



Linha = média entre as 7 execuções, cada uma interpolada numa grelha comum de 11 pontos de progresso. A dispersão por execução está nos painéis por cenário (curvas de aprendizagem).

FIGURA 6.18. Desempenho do SAC nos sete cenários (linha = média entre as 7 execuções por progresso de treino).

6.9. Análise de Cenários Complexos e Comportamento Emergente

Para além das métricas agregadas, a inspeção qualitativa das trajetórias (Figuras 6.19–6.20) revela os comportamentos coletivos que sustentam — ou comprometem — o desempenho de tarefa. As figuras traçam o percurso completo dos 20 agentes num episódio do modelo campeão da campanha final, com a cor a indicar o instante do episódio; são os mesmos episódios que o painel tridimensional reproduz. No *Muro em U* e no *Quatro Salas* (Figura 6.19), as políticas bem-sucedidas exibem contorno fluido das paredes, com os agentes a acompanharem as paredes em vez de colidirem frontalmente com os obstáculos — o efeito direto da substituição do potencial euclidiano discutida na Secção 6.3. A dispersão inicial do *spawn* pelas salas opostas ao ninho mitiga o congestionamento nas passagens estreitas, que era a causa do colapso anterior no *Quatro Salas*.

Nos cenários cooperativos (Figura 6.20) emergem comportamentos coordenados não programados explicitamente: na *Porta Cooperativa*, observa-se a sincronização de pelo menos três agentes na zona de pressão para abrir a passagem, condição imposta pela mecânica do cenário; na *Percepção Cooperativa*, o enxame organiza-se em torno do alvo móvel, mantendo cobertura à medida que este se desloca. Estes padrões mostram que o controlador evolutivo aprende não só a navegar, mas a explorar a estrutura cooperativa da tarefa. Cada figura corresponde a um episódio, não a uma montagem de vários: as chegadas indicadas no título são as desse episódio, e o comportamento é o típico do modelo campeão — a dispersão entre execuções está quantificada na Secção 6.6.

6.10. Deceção e Procura por Novidade (*Novelty Search*)

Dois cenários da bateria expõem a otimização orientada ao objetivo a paisagens de recompensa enganadoras, por vias distintas. A *Porta Cooperativa com Alternativa* é *deceptive* por desenho: o gradiente de *homing* aponta para a porta bloqueada, e o caminho que dispensa a cooperação — o corredor alternativo — exige afastamento temporário do ninho. O *Muro em U* exhibe

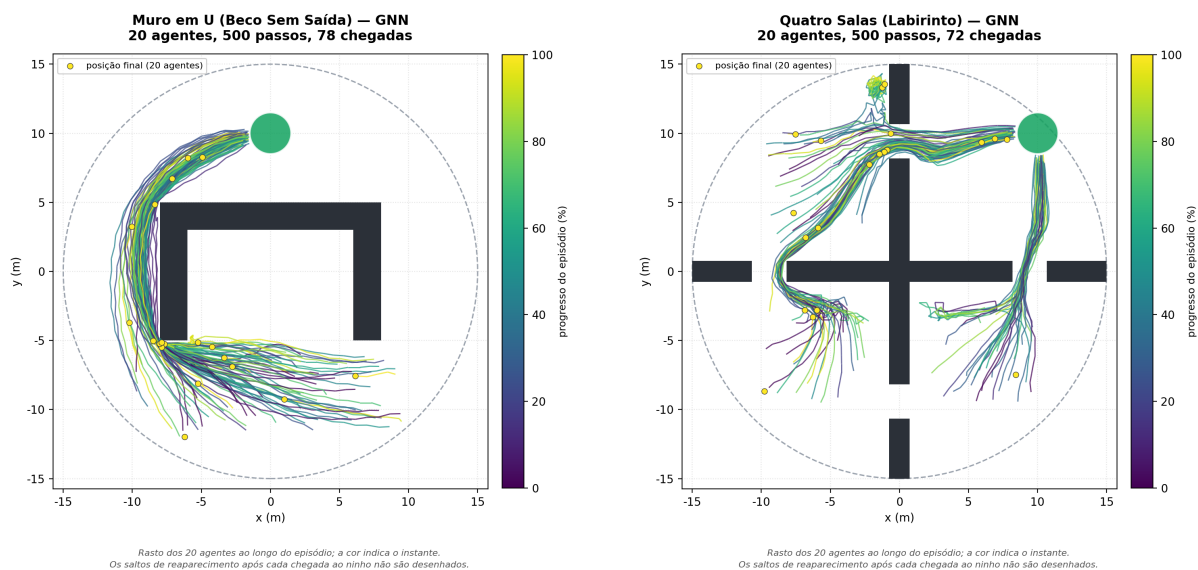


FIGURA 6.19. Trajetórias completas de um episódio do controlador evolutivo (20 agentes; cor = instante do episódio): contorno do obstáculo em U (esq., 78 chegadas) e navegação no labirinto de Quatro Salas (dir., 72 chegadas). Nenhuma trajetória atravessa uma parede: os saltos de reaparecimento após cada chegada ao ninho estão excluídos do traçado.

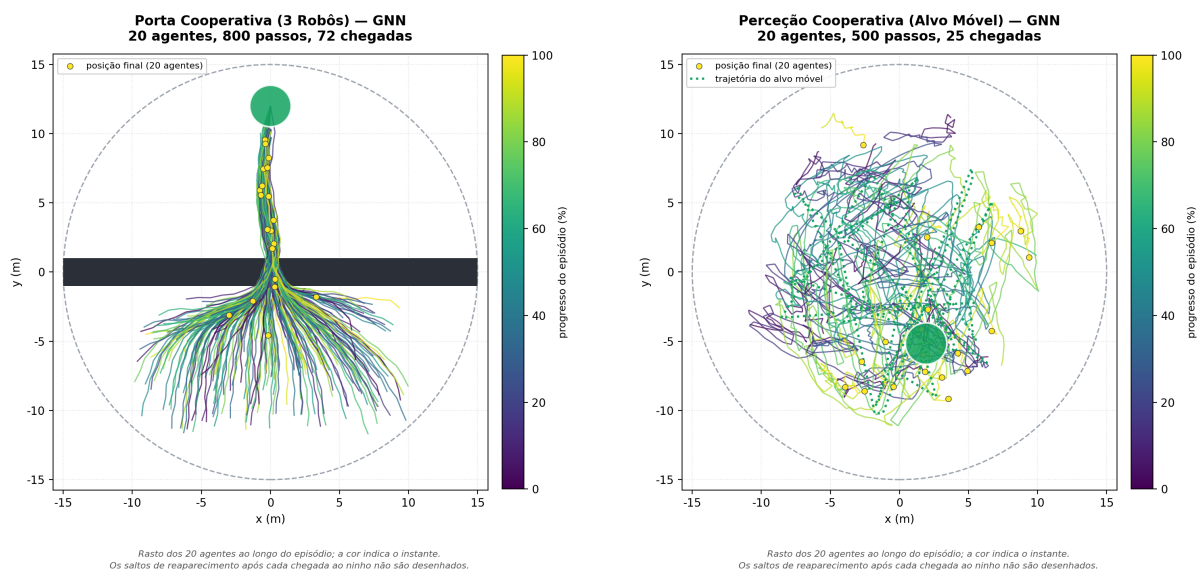


FIGURA 6.20. Comportamento cooperativo nas trajetórias de um episódio: o funil de percursos que converge para a única passagem da Porta Cooperativa e a atravessa até ao ninho (esq., 72 chegadas) e a cobertura do alvo móvel, cuja rota está marcada a tracejado verde (dir., 25 chegadas).

deceção *espacial*: o potencial geodésico aponta corretamente para o contorno, mas o agente só observa a bússola euclidiana e o LiDAR local, pelo que a *descoberta* do desvio comprido depende de exploração — e é precisamente aí que a campanha final mostrou comportamento bimodal transversal aos três algoritmos (Secção 6.6). A QI6 pergunta se a hibridização com *Novelty Search* (Secção 5.2.3) ajuda a escapar a estes ótimos locais. Realizaram-se quatro

experiências: uma comparação emparelhada preliminar, duas campanhas multi-execução com orçamento igualado (peso fixo) e, por fim, uma campanha *pré-registada* de dosagem *adaptativa* do peso de novidade nos sete cenários.

Comparação emparelhada preliminar (novidade vs. objetivo puro). Numa experiência controlada anterior à campanha final, treinou-se um controlador com *blend* de novidade ($w = 0,5$, 600 minutos) na Porta com Alternativa e comparou-se com o melhor controlador puramente objetivo então disponível (*fitness* de *homing*, 195 minutos), sob o mesmo protocolo de avaliação emparelhado (20 episódios, *seeds* comuns). O controlador com novidade obteve $81,3 \pm 1,9$ chegadas/ep (100% de sucesso) contra 64,5 do objetivo puro — +26%, Wilcoxon $p = 8,7 \times 10^{-5}$, $\delta = +1,00$. Duas ressalvas metodológicas são, contudo, incontornáveis, e ambas motivaram as campanhas seguintes. Primeira, a execução com novidade dispôs de um orçamento $\approx 3\times$ superior, pelo que o resultado media o efeito *conjunto* (novidade + orçamento). Segunda, este teste é o único do capítulo cuja unidade estatística é o *episódio* ($n = 20$, um só modelo por braço) e não a execução: os episódios de um mesmo modelo não são realizações independentes do processo de treino, pelo que este p mede a consistência do par de modelos comparados e *não é comparável* com os valores reportados nas restantes secções, todos calculados sobre médias por execução. É por esta razão que a comparação preliminar se apresenta aqui como indício, e não como resultado. Acresce uma limitação de proveniência: os artefactos brutos desta execução preliminar (avaliação por episódio e modelo) não foram retidos — foram entretanto sobrescritos no servidor de treino por campanhas posteriores —, pelo que os valores citados constituem o registo da altura e não são reproduzíveis a partir dos dados arquivados. As campanhas com orçamento igualado que se seguem, essas, têm todos os dados de avaliação versionados e são integralmente reproduzíveis.

Campanhas com orçamento igualado. Para isolar o efeito da novidade, treinaram-se 7 execuções independentes com *blend* de novidade ($w = 0,5$) em cada um dos dois cenários, com o *mesmo* orçamento (195 minutos/execução), protocolo de avaliação (20 episódios determinísticos) e *seeds* da campanha final — o braço de controlo é, assim, o próprio GNN objetivo do Quadro 6.1. A significância é avaliada ao nível da execução (Mann–Whitney exato sobre as médias por execução, $n = 7$ por braço), como no resto do capítulo.

No *Muro em U*, o resultado é o mais expressivo da linha de novidade: a hibridização elevou a taxa de convergência de 3/7 para 7/7 *execuções a 100% de sucesso*, com $69,8 \pm 5,9$ chegadas/ep contra $24,5 \pm 32,6$ do objetivo puro ($p = 0,026$, $\delta = +0,71$). A pressão por novidade não só venceu a bimodalidade que nenhum dos três algoritmos base superou, como o fez com a variância mais baixa registada neste cenário — a configuração com novidade é a única, de todas as estudadas nesta dissertação, que resolve o Muro em U de forma fiável.

Na *Porta com Alternativa*, o panorama inverte-se: ambos os braços convergem em 7/7 execuções, mas o objetivo puro é significativamente superior em magnitude — $86,7 \pm 2,0$ contra $63,0 \pm 21,9$ chegadas/ep com novidade ($p = 0,0006$, $\delta = -1,00$: todas as execuções objetivas superam todas as execuções com novidade). O ganho de +26% da comparação preliminar fica assim desmascarado como artefacto do orçamento triplo; com orçamento igualado, desviar

metade da pressão de seleção para a diversidade comportamental *custa* desempenho num cenário cujo gradiente, apesar de enganador à partida, é vencível pela otimização objetiva com a profundidade de treino da campanha final.

Leitura conjunta. A pressão por novidade comporta-se como um instrumento *direcionado*, não como um melhoramento universal: onde a paisagem de *fitness* deixa a otimização objetiva inconsistente (Muro em U, bimodal), a novidade compra a descoberta do desvio e paga-se largamente; onde o gradiente da tarefa já basta (Porta com Alternativa, 7/7), o mesmo mecanismo desperdiça metade da pressão seletiva em diversidade redundante e degrada o resultado. O peso fixo $w = 0,5$ é, aliás, a explicação provável de ambos os efeitos — o que levanta a hipótese de uma *dosagem adaptativa* do peso, testada de seguida.

Campanha confirmatória: dosagem adaptativa da novidade. A tensão anterior sugere que o peso w deve ser pago apenas *enquanto compra descoberta*. Testou-se por isso uma temperagem orientada pela própria descoberta: w parte de 0,5 e, assim que o melhor genoma por objetivo faz pelo menos uma chegada durante 10 gerações consecutivas, decai $\times 0,98$ por geração até 0 exato (regressando a seleção, a partir daí, ao objetivo puro). A campanha foi *pré-registada*: hipótese, testes confirmatórios, regra de decisão e compromissos de reporte foram fixados no repositório *antes* de existirem dados de avaliação, e a análise seguiu esse protocolo sem desvios. A condição primária replica o protocolo do capítulo — os *sete* cenários, 7 execuções de 195 minutos, *seeds* 1–7, avaliação determinística emparelhada, inferência sobre médias por execução — com o GNN objetivo do Quadro 6.1 como controlo.

Os quatro testes pré-registados confirmaram a hipótese. **(T1) Não-degradação:** nos cinco cenários que a otimização objetiva já resolve, nenhuma diferença é significativa (o menor dos cinco é $p = 0,21$); reportam-se os desvios descritivos — Porta Cooperativa $65,9 \pm 8,0$ vs. $69,8 \pm 0,9$ ($\delta = -0,45$), Perceção $13,1 \pm 10,0$ vs. $19,0 \pm 8,7$ ($\delta = -0,39$) e uma execução degenerada no Gargalo — nenhum dos quais atinge significância; no Sandbox o adaptativo até sobe descritivamente ($47,5 \pm 28,1$, 6/7, vs. $38,3 \pm 31,0$, 5/7). **(T2) O ganho no Muro em U mantém-se:** 7/7 execuções a 100% e $68,5 \pm 13,1$ chegadas/ep contra $24,5 \pm 32,6$ do objetivo ($p = 0,009$ unilateral, $\delta = +0,76$) — a fiabilidade do peso fixo, preservada. **(T3) O custo na Porta com Alternativa não se reproduz:** $77,2 \pm 16,7$ vs. $86,7 \pm 2,0$ ($p = 0,32$, $\delta = -0,35$, sem significância), com 7/7 execuções a 100% — em contraste direto com o $\delta = -1,00$ do peso fixo. **(T4) Face ao peso fixo,** o adaptativo é indistinguível no Muro em U ($p = 0,80$) e superior em magnitude na Porta com Alternativa ($77,2$ vs. $63,0$; $p = 0,073$, $\delta = +0,59$ — com $n = 7$, o peso da leitura recai no tamanho de efeito, não em cruzar 0,05).

Duas análises exploratórias, assim rotuladas no pré-registo, completam o retrato. Primeira, o *braço de controlo de orçamento*: o objetivo puro no Muro em U com o *dobro* do tempo (390 min/execução) continua bimodal — 4/7 execuções, $31,5 \pm 35,0$ — pelo que o ganho da novidade *não é comprável com orçamento*: é do mecanismo. Segunda, o adaptativo com 390 minutos mantém 7/7 no Muro em U ($77,8 \pm 11,9$) e atinge na Porta com Alternativa *o melhor resultado desta dissertação* ($88,7 \pm 0,6$, 7/7), superando o próprio objetivo puro ($86,7 \pm 2,0$) com a menor variância registada no cenário. A dosagem adaptativa fecha, assim, a tensão que o peso fixo

abrir: retém a exploração onde ela compra descoberta e devolve a seleção ao objetivo onde ele basta — sem custo mensurável em nenhum dos sete cenários.

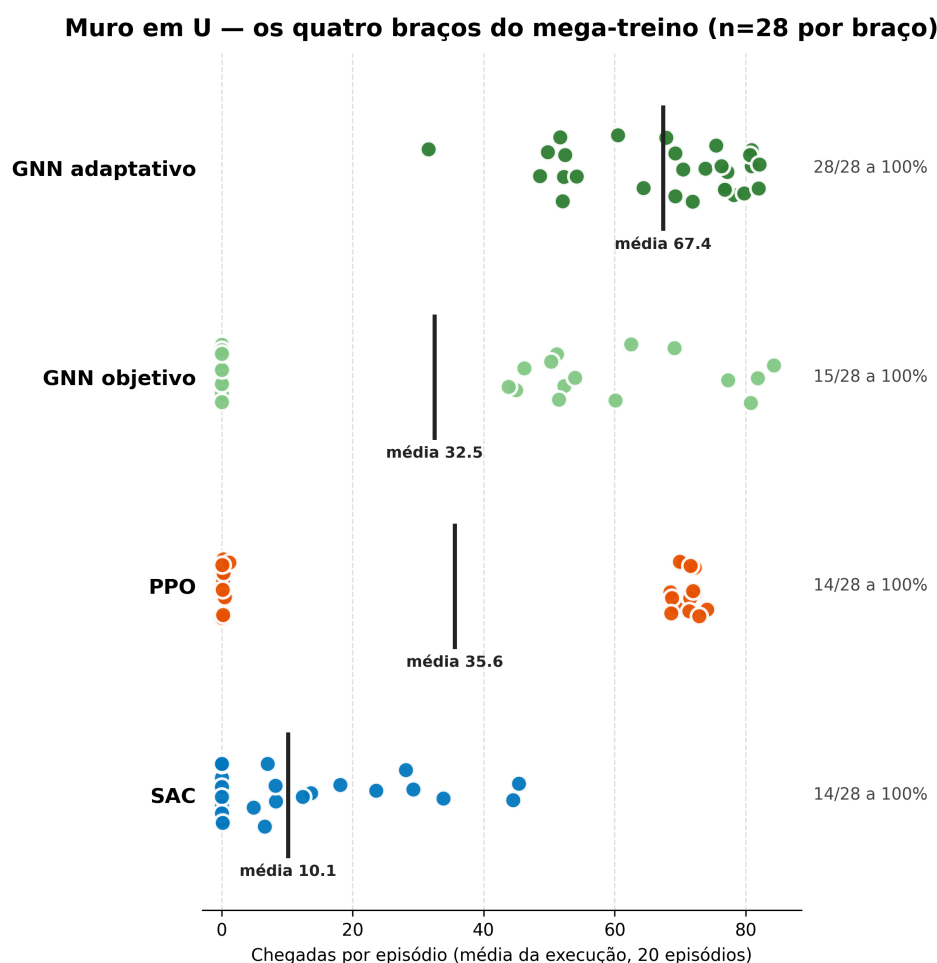
Replicação com poder estatístico: o mega-treino a $n = 28$. Os quatro testes acima assentam em 7 execuções por braço — suficientes para a inferência sobre médias, mas curtas onde a leitura depende de *contagens*: com $n = 7$, um teste exato de Fisher sobre a convergência não é reportável, e o $\delta = +0,59$ de (T4) ficou à espera de poder estatístico. Uma campanha final de um mês de máquina, com *pré-registo próprio* fixado antes de existirem dados de avaliação, repetiu por isso o cenário crítico com *quatro vezes* as execuções. As células a $n = 28$ são *autocontidas* — execuções novas, que o pré-registo proíbe expressamente somar às sete anteriores — e cobrem os quatro braços no Muro em U (GNN adaptativo, GNN objetivo, PPO e SAC), todos com o orçamento por execução da campanha final.

O resultado principal (M1) confirma-se e reforça-se: o adaptativo faz $67,4 \pm 13,4$ chegadas/ep contra $32,5 \pm 32,5$ do objetivo puro ($p < 0,0001$ unilateral, $\delta = +0,61$) e, sobretudo, a convergência — agora reportável — separa os braços sem ambiguidade: *28/28 execuções a 100% de sucesso contra 15/28* (Fisher exato, $p < 0,0001$). Confirma-se, com quatro vezes mais execuções, que a dosagem adaptativa resolve o Muro em U em todas elas. A comparação com os métodos de gradiente ao mesmo n (M2) coloca o adaptativo acima de ambos — PPO $35,6 \pm 36,0$ ($14/28$; $p = 0,0010$, $\delta = +0,51$) e SAC $10,1 \pm 14,2$ ($14/28$; $p < 0,0001$, $\delta = +0,99$) —, ao passo que o GNN *objetivo* e o PPO permanecem indistinguíveis ($p = 0,088$): a vantagem não é da neuroevolução enquanto tal, mas da *dosagem adaptativa da exploração*. A Figura 6.21 mostra porque é que a média sozinha engana neste cenário — o GNN objetivo e o PPO repartem-se entre execuções que resolvem e execuções que ficam a zero, praticamente sem nada pelo meio, o SAC é uniformemente fraco, e o adaptativo não tem uma única execução falhada. Assinala-se que os seis pares de M2 são *p* brutos: o pré-registo manda declarar a multiplicidade e ancorar a leitura no tamanho de efeito, não corrigir os valores.

Também (M3) o custo do peso fixo na Porta com Alternativa fica resolvido com significância: o adaptativo faz $80,9 \pm 15,6$ chegadas/ep em *21/21 execuções* contra $63,0 \pm 21,9$ do peso fixo ($p = 0,0016$, $\delta = +0,77$), pelo que o $\delta = +0,59$ de (T4) — que a $n = 7$ não cruzava o limiar convencional — se confirma. Como o braço de peso fixo provém da campanha anterior, a comparação é declarada como *entre campanhas*, tal como o pré-registo obriga.

Por fim, uma ablação exploratória da própria temperagem — duas durações do critério de descoberta sustentada (5 e 20 gerações) e dois fatores de decaimento (0,95 e 0,995) — mostra que o mecanismo *não é sensível à afinação*: as quatro variantes convergem em *7/7 execuções* nos dois cenários, com médias entre 59,7 e 69,1 chegadas/ep no Muro em U e entre 74,8 e 82,4 na Porta com Alternativa. O ganho é, pois, do princípio — pagar novidade apenas enquanto ela compra descoberta — e não de um *schedule* particular, o que reforça a sua transponibilidade para outros domínios com decação espacial.

As restantes três células exploratórias do pré-registo respondem no mesmo sentido, e reportam-se aqui por inteiro — o pré-registo compromete-se a divulgar *todas* as fases, e não apenas as que confirmam a hipótese. No *Sandbox*, o cenário em que o controlador evolutivo



Cada ponto = 1 execução independente (28 por algoritmo). A barra vertical é a média reportada na tese. O braço objetivo é bimodal: as execuções que resolvem fazem-no bem, e as que não resolvem ficam a zero — a média sozinha não descreve execução nenhuma.

FIGURA 6.21. Muro em U — as 28 execuções independentes de cada braço do mega-treino. Cada ponto é uma execução (média de 20 episódios determinísticos) e a barra vertical é a média reportada no texto. A leitura decisiva não é a diferença de médias, mas a *forma* das distribuições, e ela não é a mesma nos três braços de comparação: o GNN objetivo e o PPO *repartem-se* entre execuções que resolvem o cenário com magnitude plena e execuções que ficam a zero — o PPO com 14 das 28 acima de 60 chegadas/ep —, ao passo que o SAC não é bimodal mas sim *uniformemente fraco*: nenhuma das suas 28 execuções passa de 45,4 chegadas/ep, e as restantes distribuem-se de forma contínua até zero. A dosagem adaptativa da novidade, essa, não deixa uma única execução por resolver (28/28).

mais degenera, a dosagem adaptativa em 21 execuções eleva a convergência de 5/7 (71%) do objetivo puro para **20/21** (95%), com $55,7 \pm 20,1$ contra $38,3 \pm 31,0$ chegadas/ep ($p = 0,14$, $\delta = +0,39$): a assimetria das dimensões amostrais e a variância do braço objetivo mantêm o teste longe do limiar convencional, mas a direção do efeito e a queda da dispersão são as previstas — a novidade compra descoberta também onde não há paredes que canalizem o *homing*. Na *Percepção Cooperativa*, onde a pergunta era se o 5/7 do adaptativo na condição primária (T1) representava um custo real da pressão por novidade, 21 execuções mostram que não representava:

17/21 (81%) e $17,5 \pm 9,3$ contra 6/7 (86%) e $19,0 \pm 8,7$ do objetivo ($p = 1,00$, $\delta = +0,01$) — o mecanismo é neutro onde a decepção não morde, que é exatamente o que a temperagem promete. Por fim, a célula do SAC no Gargalo repetiu com 21 execuções a combinação que a campanha final resolvera em apenas 5/7: com os mesmos 48 minutos por execução da campanha final, converge em 7/21 (33%), com $21,3 \pm 37,5$ chegadas/ep, contra 5/7 e $41,4 \pm 36,8$ na campanha final. Igual em minutos, o orçamento não o foi em passos: as execuções 1–6 fizeram 8,3–9,4 milhões de passos e convergem em 5/6 (68,1 chegadas/ep), ao passo que as 15 seguintes, com o servidor partilhado, fizeram só 2,2–4,0 milhões e convergem em 2/15 (2,6) — a campanha final fizera 5,8–6,4 milhões. Com passos suficientes, a réplica reproduz a campanha final; com um terço deles, falha quase sempre. É a hipótese de sub-treino da Secção 6.1 que sai reforçada, e não uma lotaria entre execuções.

6.11. Composição de Dificuldades: o Mapa Composto

6.11.1. Motivação e Questão

Os sete cenários analisados até aqui têm uma propriedade comum: cada um isola *uma* dificuldade — um gargalo, um beco decetivo, quatro salas, uma porta que exige cooperação. Essa escolha é o que torna a comparação interpretável, porque atribui cada diferença de desempenho a uma causa identificável. Mas levanta a objeção mais natural a qualquer *benchmark* por cenários isolados: saber resolver cada exercício não é o mesmo que resolver o problema composto.

Formula-se então a Q17: as conclusões obtidas em sete cenários de dificuldade isolada transferem-se para um ambiente que *combina* essas dificuldades a uma escala substancialmente maior, ou a composição faz emergir modos de falha que os cenários isolados não revelam?

A pergunta é a extensão natural da contribuição mais forte desta dissertação. A transferência *Zero-Shot* para dimensões de enxame não vistas (Secção 6.12) mostra invariância à *cardinalidade* do enxame; o mapa composto estende a mesma pergunta à *topologia* não vista. Se o controlador de grafo transferir, o argumento da representação passa de “invariante a N ” a “invariante a N e robusto a composição”.

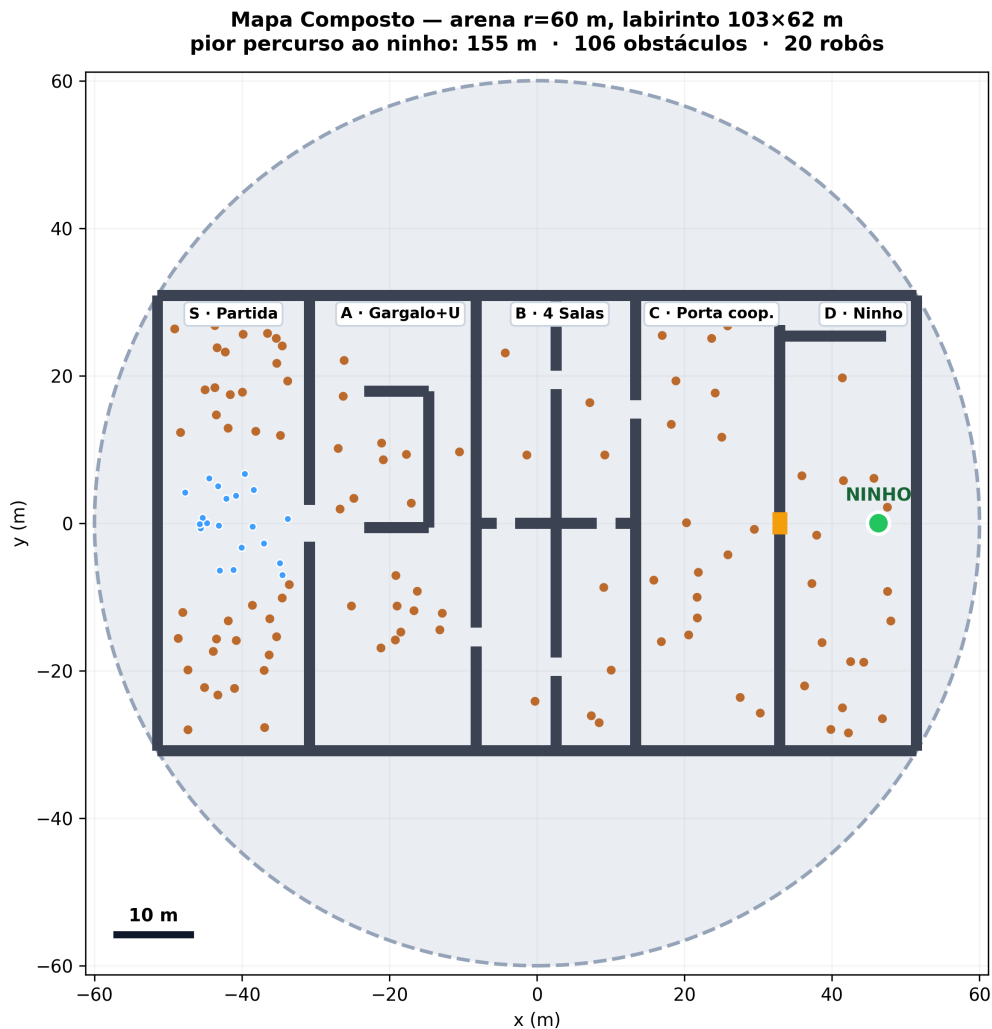
6.11.2. O Ambiente

O oitavo cenário (mapa_grande) é um labirinto composto de 103×62 m inscrito numa arena de raio 60 m — contra os 15 m de raio dos sete cenários anteriores. Está organizado em cinco zonas, de oeste para este, que o enxame tem de atravessar por ordem:

- (1) **S — sala de partida:** espaço aberto com obstáculos dispersos, onde os agentes nascem. É a única zona com analogia direta ao Sandbox.
- (2) **A — gargalo e beco em U:** uma passagem estreita seguida de um bolso em U com a boca virada a *oeste*, isto é, para o lado de onde o enxame chega. A orientação é deliberada: como a bússola de *homing* é euclidiana, um bolso só constitui armadilha quando a linha reta entre agente e ninho lhe entra pela boca (ver a nota metodológica no fim desta secção).

- (3) **B — quatro salas:** uma cruz completa (eixos horizontal e vertical) com quatro aberturas que ligam os quadrantes em ciclo, sem atalho pelo centro, replicando a topologia do cenário *Quatro Salas* à escala do mapa.
- (4) **C — porta cooperativa com alternativa:** o painel que só abre com três agentes em simultâneo na zona de empurrão, e um corredor alternativo mais longo. Com o painel fechado, o percurso dos agentes ao ninho encarece 20% (1,205×, medido sobre as posições de partida): cooperar é vantajoso, não obrigatório.
- (5) **D — câmara do ninho:** o objetivo.

Do centro da zona de partida ao ninho vão 128,8 m — os agentes nascem espalhados por uma caixa, pelo que o percurso varia entre ≈ 121 m para o mais próximo e ≈ 139 m para o mais afastado — e o ponto mais distante do ninho em todo o mapa fica a 155,4 m, contra 34 m do *Quatro Salas*. A Figura 6.22 mostra a planta, gerada a partir da geometria do próprio simulador.



Geometria lida do simulador (classic_scenario: mapa_grande) — é o mapa que os robôs treinam, não um esquema à parte.

FIGURA 6.22. Planta do mapa composto, lida da geometria do próprio simulador: cinco zonas de oeste para este, 106 obstáculos, ninho na câmara a este e caixa de *spawn* a oeste. O beco em U da zona A tem a boca virada a oeste, para o lado de onde o enxame chega.

Três parâmetros foram fixados antes de qualquer treino, e a justificação de cada um consta do pré-registo da campanha:

- **$N = 20$ agentes**, como nos sete cenários. Mantém $\text{obs_dim} = 111$ e, com isso, a comparação emparelhada e a possibilidade de avaliar os campeões existentes sem qualquer alteração arquitetural. Com $N = 40$ a dimensão passaria a 211 e as políticas MLP do PPO e do SAC exigiriam uma rede nova.
- **$\text{max_steps} = 2000$** , contra os 500 dos cenários abertos. A 0,2 m por passo (o limite por eixo), o pior ponto do mapa fica a 777 passos apenas de ida; 2000 passos deixam uma folga de $2,6\times$ sobre essa ida ($2,9\times$ a partir do pior *spawn*), acima do mínimo de $2,5\times$ fixado a priori.
- **$\text{required_to_eat} = 1$** . A cooperação que este mapa mede está na *porta*; exigir também três agentes simultâneos no ninho empilharia uma segunda tarefa cooperativa e a métrica deixaria de isolar a navegação.

Tudo o resto — recompensas, física, LiDAR, população, σ — é idêntico ao das campanhas anteriores. Sem isso, a comparação com os sete cenários não seria limpa.

6.11.3. Protocolo

A campanha decompõe-se em duas fases, de propósito por esta ordem, porque a leitura da primeira não depende da segunda e o contraste entre ambas é, em si, um resultado:

F1 — Zero-Shot de topologia: Os campeões dos sete cenários (Secção 6.2) são avaliados neste mapa *sem qualquer retreino*, 20 episódios por célula, com sementes emparelhadas. Responde a “o que já sabíamos transfere?”.

F2 — Treino nativo: Os três algoritmos treinam de raiz no mapa, 21 execuções independentes com sementes 1–21, e são avaliados no mesmo protocolo determinístico das restantes campanhas.

O número de execuções do F2 foi elevado de 7 para 21 antes de existir qualquer dado de avaliação (emenda 19 do pré-registo). A razão é de recursos e não de resultado: o desenho original ocupava menos de um quarto da janela de servidor disponível até ao limite de integração. O acréscimo foi todo para *execuções* e nenhum para minutos por execução — os 780 minutos do controlador evolutivo foram calculados para igualar *gerações* às campanhas fechadas dos sete cenários (o que, como se mostra adiante, não se verificou). As métricas M1–M3, a regra de decisão e os compromissos de reporte ficaram como estavam; em particular, *M2 permanece descritiva*. Com $n = 21$ um teste exato sobre proporções de convergência passaria a ter poder, e é precisamente por isso que se declara que não se acrescenta: converter um descritivo em teste depois de subir o n é escolher o teste com o n na mão.

O orçamento de F2 não iguala minutos, mas *gerações de evolução*, que é a unidade de otimização do controlador evolutivo. Uma geração custa $\text{pop_size} \times \text{eval_episodes} \times \text{max_steps}$ passos de simulação; com episódios quatro vezes mais longos, igualar o número de minutos daria ao controlador evolutivo um quarto da otimização e chamar-lhe-ia comparação justa. O orçamento foi, por isso, fixado em 780 minutos por execução do controlador evolutivo e 192

minutos por execução de cada método de gradiente, ambos registados antes de qualquer análise. O cálculo que o fixou partia, porém, de uma estimativa do custo por passo de simulação que as medições posteriores desmentiram, e a igualdade de gerações que pretendia garantir não se verificou: as 21 execuções fecharam com 165,3 gerações em média (amplitude de 132 a 191), contra cerca de 550 por execução, com o mesmo orçamento de 195 minutos, nas execuções do controlador evolutivo no Muro em U. Neste mapa, o controlador evolutivo dispôs, portanto, de menos de um terço das gerações que teve nos sete cenários — o que reforça a leitura de que o orçamento não chegou ao planalto (Secção 7.5).

6.11.4. Transferência sem Retreino (F1)

O resultado do F1 dispensa quadro: das 21 células da condição natural — sete cenários de origem $\times$ três algoritmos, 20 episódios cada, 420 episódios — *todas* as 21 registam 0,00 chegadas por episódio e 0% de sucesso. Nenhum dos 21 campeões faz uma única chegada ao ninho no mapa composto sem retreino, e o mesmo se verifica nas três condições de controlo, o que perfaz 84 células a zero em 1680 episódios.

A leitura desta grelha exige cuidado, e o pré-registo fixa-a antes dos dados: um zero no F1 admite *quatro* causas e apenas uma é a pergunta da QI7. Além da topologia composta, a mesma célula a zero pode resultar de (i) a escala da observação — as distâncias são normalizadas pelo raio da arena, que aqui é quatro vezes maior, pelo que o mesmo modelo recebe todas as distâncias comprimidas 4 $\times$; (ii) os *obstáculos* — dos oito cenários, apenas o Sandbox e este têm obstáculos dispersos, pelo que um campeão de labirinto nunca encontrou um em toda a sua otimização; e (iii) as quatro dimensões da observação associadas à porta, identicamente nulas no treino de quem nunca teve porta e ativas neste mapa.

Nenhuma das três causas alternativas explica o resultado. Cada condição de controlo repete as mesmas 21 células com *uma* alteração e o resto bit-a-bit igual — normalizar as distâncias pelo raio da arena de treino em vez do raio deste mapa, remover os obstáculos dispersos, e zerar as quatro dimensões da observação associadas à porta — e as três devolvem, célula a célula, exatamente o mesmo: 0,00 chegadas por episódio, sem uma única célula “ressuscitada”. Com as 21 células a zero nas duas condições de cada comparação emparelhada, nem o teste de Wilcoxon nem o δ de Cliff têm conteúdo (não há sequer um par discordante); o que sustenta a exclusão é o *número de células ressuscitadas*, que é zero nas três. As três condições não se detalham, por isso, célula a célula, e a leitura reporta-se sobre a condição natural, como o pré-registo fixou antes de haver dados.

O que resta é a causa que a QI7 pergunta: a topologia composta à escala do mapa.

Duas causas adicionais, que a geometria tridimensional do simulador torna plausíveis, foram excluídas por medição direta e não figuram entre as quatro do pré-registo por não terem sido antecipadas. A primeira é o *limite vertical*: os sete cenários de origem não o têm, e os controladores usam a dimensão vertical de forma substancial durante o treino — nos episódios registados para inspeção, a altitude média dos agentes vai de 0,13 a 5,67 m e chega a máximos de 14,6 m —, pelo que a restrição a ± 2 m imposta neste mapa altera a dinâmica em que essas políticas foram otimizadas. Repetindo a avaliação com o limite vertical removido (isto é, com

a casca esférica da arena como único limite), quatro campeões evolutivos — os do Muro em U, Gargalo, Quatro Salas e Sandbox — sobre três sementes cada mantêm 0,00 chegadas por episódio, exatamente como na condição com limite. O limite vertical não é, portanto, o que explica o zero. A segunda é a *impossibilidade geométrica de chegar ao ninho* discutida nas limitações: também essa fica excluída, porque a distância mínima ao ninho *na horizontal*, tomada sobre os vinte agentes e todos os passos do episódio, fica entre 37 e 79 m num percurso mínimo de $\approx 121\text{--}139$ m — os agentes não chegam sequer à vizinhança do ninho, quanto mais a precisar de descer para lá entrar. Os mesmos modelos, nos seus cenários de origem, aproximam-se até 1,4 m do ninho e fazem entre 67 e 137 chegadas por episódio. O que falha na transferência é a navegação, e falha cedo.

6.11.5. Treino Nativo (F2)

QUADRO 6.3. Treino nativo no mapa composto (F2): 21 execuções independentes por algoritmo, avaliação determinística de 20 episódios com sementes emparelhadas.

Algoritmo	Chegadas/ep	Desvio-padrão	Sucesso	Convergentes
GNN	1,7	3,6	17%	4/21
PPO	0,0	0,0	0%	0/21
SAC	0,0	0,0	0%	0/21

O Quadro 6.3 reúne as três métricas pré-registradas sobre as 21 execuções de cada controlador; os parágrafos seguintes leem-nas uma a uma.

M1 (magnitude). Mann-Whitney U bilateral sobre as médias por execução ($n = 21$): GNN vs. PPO: $p = 0,3058$, $\delta = +0,19$; GNN vs. SAC: $p = 0,3058$, $\delta = +0,19$; PPO vs. SAC: $p = 1,0000$, $\delta = +0,00$. O peso da evidência está no tamanho de efeito, como a limitação desta campanha declara.

M2 (convergência, descritivo). Execuções que atingem pelo menos uma chegada na avaliação determinística: GNN em 4/21, das quais 2 a 100% de sucesso; PPO em 0/21, das quais 0 a 100% de sucesso; SAC em 0/21, das quais 0 a 100% de sucesso. Pelo compromisso pré-registrado (emenda 19), não se infere sobre proporções: a contagem é reportada como descritivo.

M3 (porta cooperativa). Fração de episódios em que a porta é aberta: GNN 43%, PPO 0%, SAC 0%.

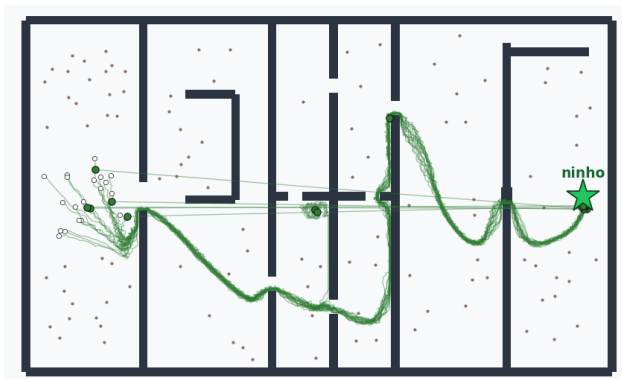
Um zero em chegadas é um número pobre: não distingue “os agentes não se movem” de “os agentes percorrem quase todo o mapa e ficam-se a dez metros do fim”. A Figura 6.23 desfaz essa ambiguidade mostrando, para um episódio de cada controlador, o percurso de todos os agentes sobre a planta. Os três episódios partilham as sementes e o horizonte; para o PPO e o SAC escolheu-se deliberadamente a *melhor* das 21 execuções — a que mais se aproxima do ninho —, para que a comparação não dependa de uma execução particularmente má.

6.11.6. Discussão

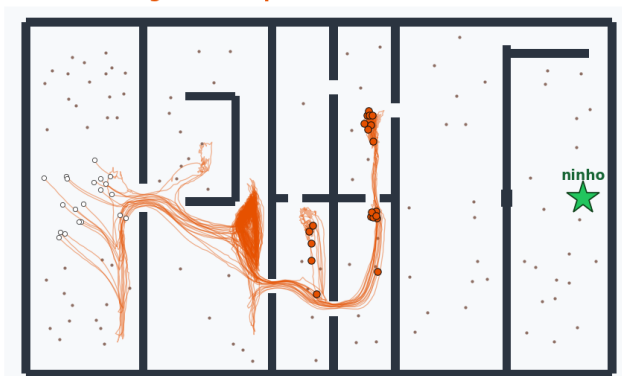
O mapa composto é resolúvel pelo GNN, mas não de forma fiável: 4 das 21 execuções atingem pelo menos uma chegada e 2 chegam aos 100% de sucesso, abaixo do limiar de 15 que o pré-registo fixou. Pela regra de decisão pré-comprometida, a *QI7 não sobe a resultado afirmativo*,

Mapa composto: por onde andam os 20 agentes num episódio (2000 passos)

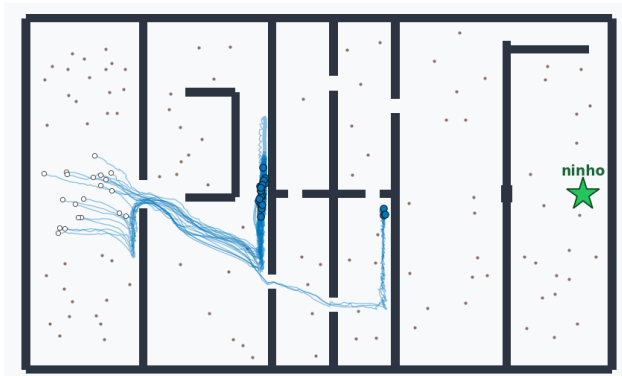
GNN (Evolutivo) — 8 chegadas no episódio



PPO — 0 chegadas no episódio



SAC — 0 chegadas no episódio



Rasto de cada agente ao longo do episódio inteiro. Pontos brancos: onde nasceram. Pontos cheios: onde acabaram.
Mesma planta, mesma escala, mesmas sementes. Os episódios são os mesmos que a vista «Episódio 3D» do dashboard desenha.

FIGURA 6.23. Percurso de cada um dos 20 agentes ao longo de um episódio completo (2000 passos), sobre a planta do mapa. Pontos brancos: posições iniciais; pontos cheios: posições finais. O controlador de grafo atravessa as cinco zonas e regressa ao ninho — os segmentos retos são reposições após cada chegada —, ao passo que o PPO se concentra na zona das quatro salas e o SAC não transpõe consistentemente o gargalo da zona A. Os episódios são os mesmos que a instrumentação por distância geodésica quantifica: sobre 21 execuções $\times$ 3 episódios, a fração que alcança a passagem para a zona da porta é de 49% (GNN), 3% (PPO) e 0% (SAC).

e o número de execuções convergentes é declarado aqui para que quem prefira outro critério o tenha à frente. A distribuição entre execuções é o resultado, e não a média: média de 8,9 chegadas por episódio entre as que chegam ao ninho (amplitude de 7,6 a 10,6), contra 0,00 nas restantes — a distribuição é de tudo-ou-nada: as execuções que resolvem o mapa resolvem-no com magnitude comparável entre si.

Esta é a assinatura que a dissertação já documentou no Muro em U (Secção 6.6), onde a mesma configuração ora resolve o cenário por completo ora fica a zero, e onde a média entre execuções não descreve nenhuma execução real. O que o mapa composto acrescenta é que a bimodalidade *sobrevive à escala*: multiplicar por quatro o raio da arena e compor quatro dificuldades não a substitui por um colapso uniforme — o que seria a hipótese natural — mas amplia a fração de execuções que ficam do lado errado. A leitura da QI7 é, portanto, intermédia: as conclusões dos cenários isolados transferem-se *parcialmente* e ao preço da fiabilidade, e é sobretudo a fiabilidade entre execuções que a composição degrada.

Resta responder à pergunta nos termos em que ela foi posta na abertura desta secção, porque é aí que o resultado negativo pode ser lido a mais do que vale: ele *não retira* ao argumento da representação o que a Secção 6.12 lhe deu. A invariância à *cardinalidade* do enxame continua medida e continua total — 100% de sucesso nas 28 combinações cenário $\times$ dimensão, sem retreino —, e nada neste mapa a contradiz: o que aqui falha não é transferir para mais agentes, é aprender uma topologia composta. O que este cenário mostra é que a invariância a *N não se estende* à composição topológica — uma arquitetura invariante ao número de agentes não é, por isso, invariante à estrutura do ambiente. São duas invariâncias diferentes, e a distinção entre a QI2 e a QI7 passa a ser a forma de as medir em separado.

Sob esta leitura, a resposta à QI7 que transita para as Conclusões é uma condição, e não um veredicto: um enxame treinado de raiz resolve o mapa composto em 4/21 tentativas de treino independentes, pelo que qualquer aplicação teria de treinar várias vezes e seleccionar — que é exatamente a prática que a Secção 6.10 mostra ser dispensável nos dois cenários de decepção quando a exploração é doseada adaptativamente. Fica em aberto, e como trabalho futuro declarado, se a mesma dosagem adaptativa recupera a fiabilidade neste mapa. A objecção vizinha — a de que *faltou treino* — é uma pergunta diferente desta, e o braço exploratório de orçamento longo que o pré-registo previa para lhe responder *não chegou a ser lançado* (emenda 24, 13 de agosto): com o limiar de decisão já fora de alcance, três execuções adicionais não alterariam o veredicto e competiriam pelo tempo de máquina que faltava ao braço principal. Ela responde-se, por isso, por medição sobre as 21 execuções que existem — e a medição *sustenta* a objecção em vez de a afastar: em 19 das 21 o melhor *fitness* surge nos últimos 20% das gerações, o que indica que o orçamento não chegou ao planalto (Secção 7.5).

6.11.7. Limitações desta Campanha

Seis limitações ficam declaradas. As quatro primeiras foram fixadas antes dos dados; as duas últimas resultaram de medições feitas ao mapa construído, e declaram-se aqui porque condicionam a leitura dos resultados acima.

Primeiro, as duas condições de normalização da observação estão *ambas* fora da distribuição de treino, de maneiras opostas: na condição natural as distâncias chegam comprimidas 4×; na condição de controlo passam a valer mais de 1 (até $\approx 2,5$), valor que nenhum modelo viu em treino. Não há terceira hipótese sem retrainar — e retrainar é, precisamente, a fase F2.

Segundo, o peso da evidência está no tamanho de efeito (δ de Cliff) e não em cruzar o limiar de 0,05. Com $n = 21$ o teste sobre médias por execução tem poder suficiente, mas a métrica de convergência (M2) mantém-se *descritiva* por compromisso declarado antes dos dados: com $n = 7$ um teste exato de Fisher não distinguiria 7/7 de 3/7, e passar a testá-lo só porque o n subiu seria escolher o teste depois de conhecer a amostra.

Terceiro, o simulador é tridimensional e este mapa é, por construção, planar: o labirinto vive num plano, o *LiDAR* varre na horizontal, e tanto o bónus de exploração como o campo geodésico que suporta a recompensa de progresso são calculados na projeção horizontal. A dimensão vertical não contribui, portanto, para nada do que a QI7 mede — e por isso o movimento dos agentes é limitado a ± 2 m em z neste cenário, uma folga de treze raios de robô que permite manobra local e não permite abandonar o plano do labirinto. A nota metodológica adiante explica porque é que essa restrição foi necessária e não apenas conveniente.

Quarto, este cenário *não entra* nos quadros comparativos dos sete cenários (Quadros 6.1 e 6.8). As campanhas daqueles têm orçamento, número de passos por episódio e protocolo próprios; juntá-los na mesma linha compararia coisas diferentes. É também a razão pela qual o código mantém a lista dos sete cenários da dissertação separada da lista de cenários ativos do simulador.

Quinto, o *incentivo a cooperar na zona C é modesto, e tem de ser reportado ao lado da métrica que o mede*. A métrica M3 — fração de episódios em que a porta é aberta — foi pré-registada para testar se a cooperação emerge *quando é vantajosa*. O quanto ela é vantajosa está medido desde 28 de julho (emenda 15 do pré-registo) e vale a pena trazê-lo para junto da métrica: sobre os *spawns* reais e com o mesmo campo geodésico que alimenta a recompensa de progresso, fechar a porta faz o percurso do agente médio passar de 130–132 m para 157–159 m, uma razão de 1,205. Não cooperar custa, portanto, cerca de 20% a mais, o que equivale a poupar 17% cooperando — e cooperar exige que três agentes ocupem a zona de empurrão *ao mesmo tempo*. Com esta razão de custos, uma M3 baixa não distingue “a cooperação não emergiu” de “não compensava emergir”, e a segunda leitura é a mais parcimoniosa. A alternativa existe de facto: com o painel da porta a contar como parede, o potencial no *spawn* continua finito — não abrir a porta atrasa, não impede.

Sexto, a chegada ao ninho é avaliada em *três* dimensões, enquanto tudo o resto do que este mapa mede é planar. A condição de chegada compara a distância euclidiana tridimensional do agente ao centro do ninho, que está no plano $z = 0$, com o raio do ninho: um agente acima de 1,6 m não conta como chegada, esteja onde estiver na horizontal, nem que esteja exatamente por cima do alvo. Como o movimento vertical está limitado a ± 2 m neste cenário, sobram 0,4 m de altitude em que a navegação é possível e a chegada não. A restrição é pequena, e é muito menor do que a dos sete cenários — onde, sem limite vertical, a mesma faixa improdutiva se estende

até ao raio da arena, 13,4 m —, mas fica declarada porque a assimetria entre uma tarefa planar e um critério de sucesso tridimensional não é evidente a partir da descrição do cenário. Uma versão futura deste mapa deve alinhar o limite vertical com o raio do ninho, e não com uma folga de manobra escolhida à parte.

Nota metodológica sobre o desenho do mapa. Quatro correções feitas antes de existir qualquer resultado válido neste cenário merecem registo, porque ilustram um cuidado que se aplica a qualquer ambiente composto. A primeira: um bolso em U só funciona como armadilha se a bússola de *homing* apontar para dentro dele — com a boca virada para o lado oposto ao da chegada do enxame, medindo sobre sessenta pontos de entrada na zona, *nenhum* agente era atraído para o interior, e o bolso era espaço morto. A segunda: uma zona anunciada como “quatro salas” que tenha apenas uma parede e uma abertura são, de facto, duas salas; a cruz completa foi verificada por decomposição do espaço livre em quatro componentes de área equivalente.

A terceira é a mais instrutiva. O simulador é tridimensional e a arena é uma esfera, pelo que uma parede só veda o que a sua altura cobre. A altura das paredes era uma constante herdada dos sete cenários, onde coincide exatamente com o diâmetro da arena; neste mapa, cuja arena tem quatro vezes o raio, sobravam quarenta e cinco metros de espaço livre acima de todas elas. Um agente que subisse — o que custa setenta e cinco dos dois mil passos do episódio — percorria o mapa inteiro em linha reta, sem gargalo, sem salas e sem porta. Verificou-se, ao medir a altitude dos agentes, que os controladores o faziam de facto, e que a primeira avaliação conduzida neste cenário media isso e não a topologia; foi descartada. A altura das paredes passou a derivar do raio da arena.

A quarta é a consequência da terceira, e a mais subtil. Vedar as paredes em toda a altura impede o atalho, mas não retira ao agente a possibilidade de subir — e subir era *lucrativo*. Um agente encostado ao limite superior da arena esférica passa a ser tratado pelo simulador como estando fora dos limites e é reposicionado para dentro; nesse passo não lhe é cobrado o custo energético, e a variação de potencial resultante do reposicionamento não lhe é imputada. Como o reposicionamento aponta para o centro da arena, e o potencial se mede na projeção horizontal, a aproximação ao ninho era creditada sem custo. O resultado é um estado em que a pressão temporal desaparece: medido no controlador do ensaio de integração, os agentes estabilizavam entre quarenta e cinquenta e sete metros de altitude, com 10,7% dos passos sem qualquer recompensa e uma aproximação acumulada nunca creditada. A correção foi restringir o movimento vertical ao plano do labirinto, como descrito nas limitações.

A lição é comum às quatro, e as duas últimas são o seu caso extremo: um mapa composto não herda as dificuldades dos cenários que diz compor, e cada uma tem de ser verificada no ambiente construído — incluindo as que parecem garantidas pela geometria. A conectividade do labirinto tinha sido validada por decomposição do espaço livre, mas num plano, e o espaço em que os agentes se moviam não era planar. Vale a pena registar que ambas as falhas foram encontradas porque o controlador as explorou: uma otimização suficientemente livre trata

qualquer imprecisão do ambiente como parte do espaço de soluções, e é essa a razão pela qual o comportamento aprendido é, em si, um instrumento de auditoria do simulador.

6.12. Análise de Escalabilidade (Zero-Shot Transfer)

Para além do desempenho de tarefa, é na *escalabilidade* que reside o contributo distintivo do controlador evolutivo — e a justificação arquitetural da abordagem por atenção sobre grafo. Avaliou-se a transferência, *sem qualquer retreino*, da política aprendida com $N = 20$ agentes para enxames de $N \in \{10, 50, 100\}$ — e, numa literatura em que a transferência para dimensões não vistas raramente é medida (Secção 3.6), a bateria cobre *os sete cenários*, incluindo os labirintos e as tarefas cooperativas. Em cada célula (cenário $\times$ dimensão) correram-se 20 episódios determinísticos com o modelo campeão da campanha final.

O resultado é inequívoco: o controlador de grafo mantém *100% de sucesso nas 28 combinações*, de metade a cinco vezes o exame de treino, em todos os cenários (Quadro 6.4). As políticas MLP do PPO e do SAC, com dimensão de entrada fixa ($\mathbb{R}^{111}$, calibrada para $N = 20$), são por construção incompatíveis com outras dimensões de exame (Quadro 6.5); no tamanho de treino, os três algoritmos são comparáveis (Sandbox, $N = 20$: GNN 65,8, PPO 71,7, SAC 71,3 chegadas/ep). Este contraste corrobora diretamente a hipótese de que o mecanismo de atenção sobre a vizinhança confere invariância à cardinalidade do exame, sendo a propriedade que habilita o controlo escalável.

QUADRO 6.4. Escalabilidade Zero-Shot do controlador de grafo nos sete cenários (20 episódios determinísticos por célula; modelo campeão da campanha final, treinado com $N = 20$): chegadas *por agente* e retenção da eficiência per capita em $N = 100$ face a $N = 20$. Taxa de sucesso: 100% em todas as células.

Cenário	$N = 10$	$N = 20$	$N = 50$	$N = 100$	Retenção
Sandbox	3,74	3,29	1,95	1,27	39%
Perceção Cooperativa	1,64	1,30	0,83	0,59	45%
Gargalo	6,92	6,93	5,96	4,04	58%
Quatro Salas	3,60	3,86	3,26	2,54	66%
Muro em U	3,84	4,01	3,81	3,14	78%
Porta Cooperativa	3,40	3,54	3,41	3,12	88%
Porta com Alternativa	4,31	4,50	4,44	4,06	90%

O padrão da coluna de retenção merece leitura atenta, porque desarma a objeção mais natural a este resultado — a de que um exame cinco vezes maior colapsaria por congestionamento nas passagens estreitas. Observa-se, em larga medida, o *contrário*: a retenção mais alta pertence aos cenários com estrutura física (Porta com Alternativa 90%, Porta Cooperativa 88%, Muro em U 78%) e a mais baixa aos cenários abertos (Sandbox 39%, Perceção Cooperativa 45%), onde $N = 100$ agentes partilham um recurso finito cuja densidade por agente se reduz a um quinto — diluição de recurso, não falha de coordenação. Nesses cenários estruturados, o fluxo canalizado pelas paredes e a natureza repetível da chegada no ninho mantêm o rendimento individual: com 100 agentes, a porta cooperativa e o desvio do Muro em U continuam a ser atravessados de forma ordeira.

É preciso, porém, registrar os dois casos intermédios: o Quatro Salas retém 66% e o *Gargalo* 58% — a retenção mais baixa de todos os cenários com paredes. É precisamente o que se esperaria de uma saturação parcial do fluxo: o Gargalo tem uma *única* passagem de 2,5 m, e é o cenário em que o congestionamento tem mais margem para morder. Mas mesmo aí não há colapso: a taxa de sucesso mantém-se em 100% e as chegadas *totais* continuam a crescer com o enxame (138,6 em $N = 20$ para 403,6 em $N = 100$). A conclusão que a coluna suporta é, pois, mais matizada do que a ausência total de congestionamento: a estrutura física *atenua* a diluição em vez de a agravar, e onde a geometria impõe um estrangulamento único observa-se um custo de fluxo mensurável — mas nunca a rutura da coordenação que a objeção antecipava. A Figura 6.24 mostra graficamente esse contraste: a linha do controlador de grafo estende-se por toda a gama de dimensões, enquanto as dos métodos de gradiente se interrompem em $N = 20$ — a dimensão de entrada fixa da política MLP impede-os sequer de ser avaliados nas restantes.

QUADRO 6.5. O contraste arquitetural no cenário Sandbox (20 episódios/dimensão): taxa de sucesso e chegadas totais por episódio ao transferir a política treinada com $N = 20$, sem retreino.

Algoritmo	$N = 10$	$N = 20$ (treino)	$N = 50$	$N = 100$
GNN (Evolutivo)	100% / 37,4	100% / 65,8	100% / 97,5	100% / 127,3
PPO	N/A [†]	100% / 71,7	N/A [†]	N/A [†]
SAC	N/A [†]	100% / 71,3	N/A [†]	N/A [†]

Cada célula: taxa de sucesso / chegadas por episódio. [†] A política MLP do PPO/SAC tem dimensão de entrada fixa ($\mathbb{R}^{111}$, para $N = 20$), sendo incompatível com outras dimensões de enxame. Só a arquitetura de atenção sobre grafo (controlador evolutivo) suporta a transferência *Zero-Shot*. As chegadas totais do GNN crescem monotonicamente com N (37,4 $\rightarrow$ 127,3) sem qualquer colapso.

6.13. Resiliência e Robustez a Falhas de Agentes

A métrica R_{robust} avalia a resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio (os agentes afetados ficam inertes, deixando de agir e de contar para os requisitos cooperativos). A Figura 6.25 compara as chegadas com e sem falha, em avaliação emparelhada (20 episódios por célula, 3 algoritmos $\times$ 7 cenários) sobre os modelos da campanha final.

O resultado evidencia uma resiliência elevada e *transversal aos paradigmas*: a retenção de chegadas situa-se entre 92% e 106% em todas as 21 combinações algoritmo–cenário com desempenho de base, incluindo a Porta Cooperativa — notável por exigir três agentes simultâneos para abrir a passagem: o enxame redistribui-se e mantém a função apesar da baixa. O controlador evolutivo retém 92–97%. (Os valores pontualmente acima de 100% correspondem a episódios em que a perda de agentes reduziu o congestionamento nas passagens estreitas.) A ausência de degradação catastrófica sugere que o *parameter sharing* confere ao enxame uma redundância funcional implícita: nenhum agente é insubstituível, pelo que a falha parcial é absorvida pela reorganização do coletivo.

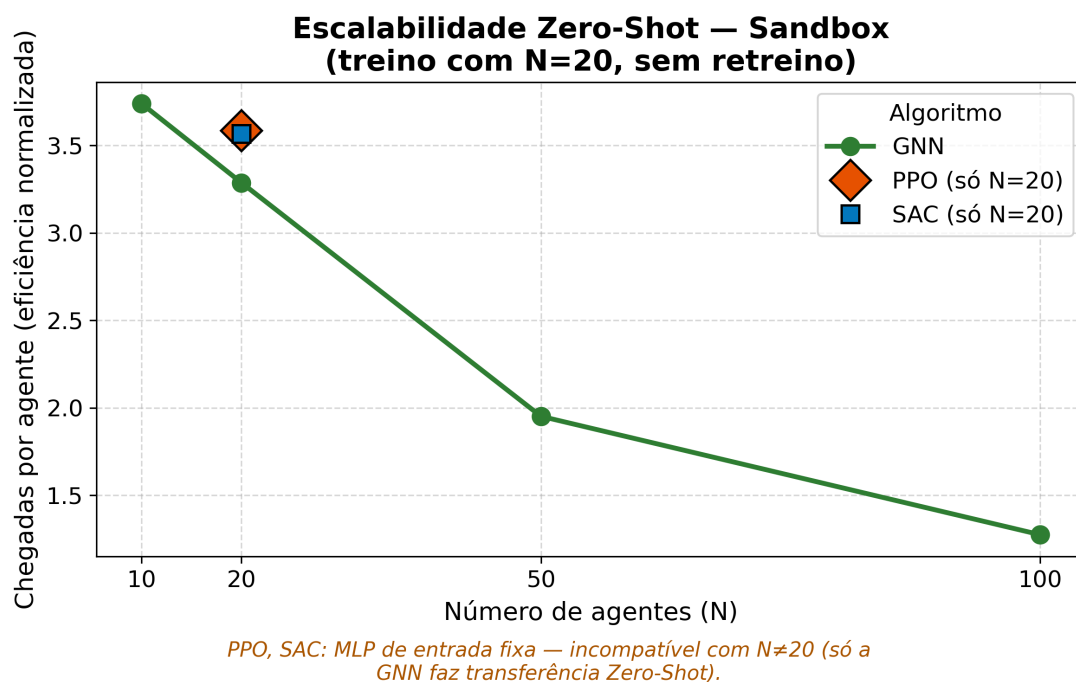


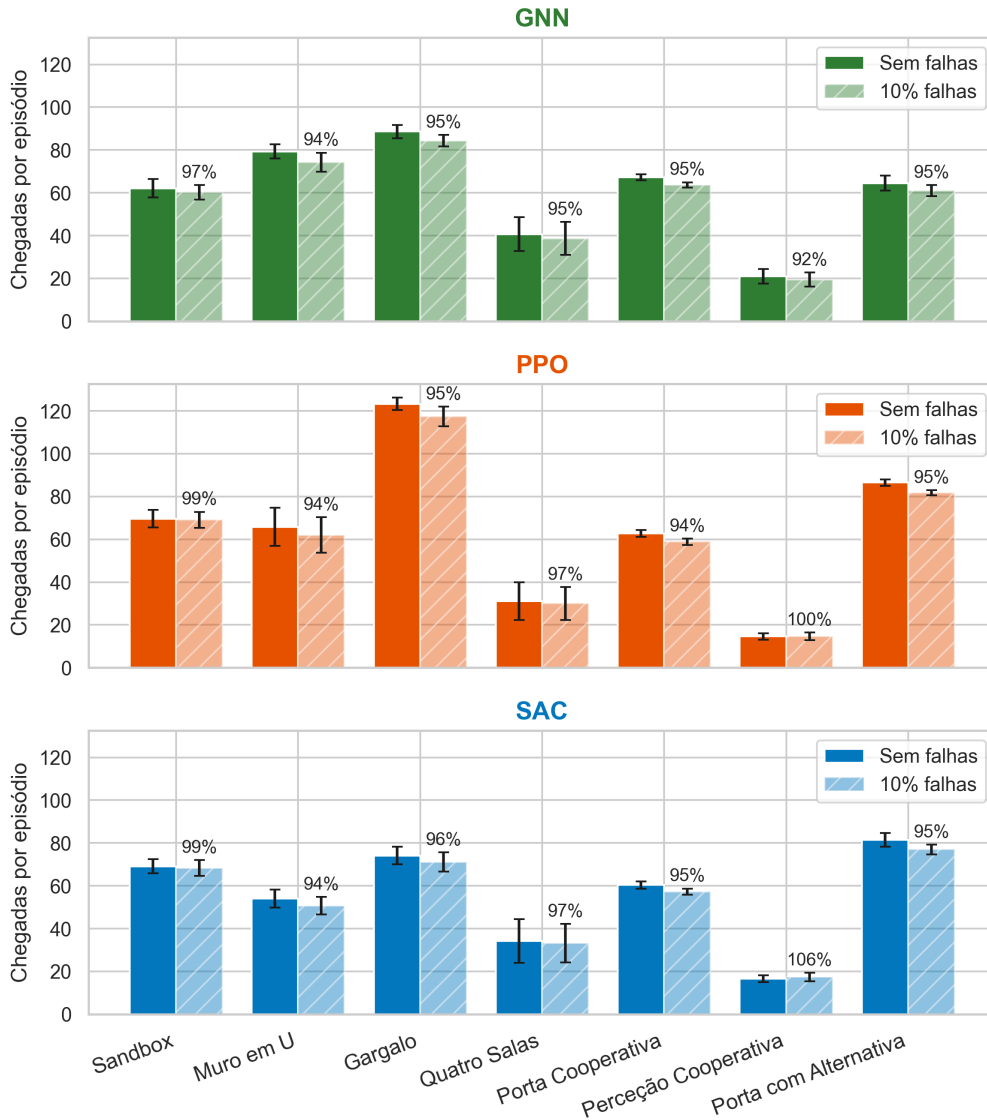
FIGURA 6.24. Eficiência *por agente* (chegadas por episódio a dividir pelo número de agentes) com o aumento do enxame, sem retreino, no cenário Sandbox. Só a linha do controlador de grafo atravessa toda a gama: o PPO e o SAC têm um único ponto, em $N = 20$ (o losango e o quadrado de contorno preto, sobrepostos porque distam 0,02 chegadas por agente), porque a política MLP de entrada fixa não admite outras dimensões de enxame.

6.14. Desempenho Computacional do Simulador

A viabilidade da campanha experimental depende do *throughput* do simulador, medido em execução de referência *single-thread* (uma arena, $N = 20$ agentes, 3 000 passos com ações aleatórias na máquina de desenvolvimento; protocolo automatizado em `scripts/benchmark_sim.py`). A implementação original sustentava cerca de *139 passos de simulação por segundo* ($\approx 2\,770$ atualizações de agente por segundo; $\approx 3,6$ s por episódio de 500 passos); após a vetorização descrita abaixo, a mesma medição sustenta *≈ 420 passos/s* ($\approx 8\,400$ agente-passos/s; $\approx 1,2$ s por episódio) — um ganho de $\approx 3,0\times$, consistente com a aceleração de $2,58\times$ medida isoladamente no custo do passo (Quadro 6.6).

O treino dos métodos de gradiente escapa a este limite por *paralelização*: PPO e SAC executam 16 clones independentes da arena em processos paralelos (`SubprocVecEnv`), recolhendo experiência concorrentemente nos vários núcleos dos servidores de cálculo; o controlador evolutivo, por sua vez, avalia os 30 genomas da população em paralelo. Adicionalmente, antes da campanha final o próprio simulador foi *vetorizado* com NumPy: o varrimento LiDAR em lote (todos os agentes $\times$ todos os raios numa única operação matricial, aceleração de $19,5\times$ nessa componente) e o cálculo em lote das observações egocêntricas ($2,58\times$ no custo total do passo de simulação). Ambas as otimizações foram validadas por testes de equivalência *bit-exata* contra a implementação de referência (zero discrepâncias em dezenas de milhares de comparações),

Robustez a Falhas de Agentes (Rrobust)
10% dos agentes falham a meio do episódio



Avaliação determinística emparelhada (20 episódios, mesmas seeds);
rótulo = % de chegadas retidas face à avaliação sem falhas.

FIGURA 6.25. Resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio (R_{robust}). Os três painéis partilham a escala vertical, pelo que as alturas são comparáveis entre algoritmos; o rótulo é a percentagem de chegadas retida face à avaliação sem falhas, nas mesmas *seeds*. A base de cada barra é o modelo campeão de cada célula, e não a média das sete execuções do Quadro 6.1: nos cenários de convergência bimodal os dois valores afastam-se bastante (no Muro em U, 79 contra 24,5 chegadas/ep), porque a média entre execuções inclui as que ficam a zero. A retenção não é afetada por essa escolha — é um rácio entre duas avaliações da mesma fonte, com e sem falhas. Os 106% do SAC na Percepção Cooperativa não são um erro de medida: nesse cenário o alvo é móvel e um enxame ligeiramente menor congestionava-o menos — a diferença (16,5 → 17,5 chegadas/ep) está dentro do desvio entre episódios.

garantindo que aceleram a experimentação sem alterar a tarefa — no caso do controlador evolutivo, o ganho no custo do passo de simulação ($2,58\times$) traduz-se num aumento aproximadamente proporcional do número de gerações avaliadas no mesmo orçamento de treino.

QUADRO 6.6. Desempenho computacional do simulador (*throughput* de referência, $N = 20$).

Configuração	Throughput
Single-thread, pré-vetorização (1 arena)	≈ 139 passos/s ($\approx 2\,770$ ag.-passo/s)
Single-thread, pós-vetorização (1 arena)	≈ 420 passos/s ($\approx 8\,400$ ag.-passo/s)
Episódio completo (500 passos), pós-vetorização	$\approx 1,2$ s
Treino vetorizado (SubprocVecEnv, 16 arenas)	paralelização $\approx n.^{\circ}$ de núcleos

6.14.1. Custo de Treino e de Execução por Algoritmo

O *throughput* acima é do simulador e é comum aos três algoritmos; o que cada um *faz* com esse orçamento é distinto, e o Quadro 6.7 mede-o. Os valores provêm da campanha de replicação ($n = 28$ execuções por braço, Muro em U, Secção 6.10), que é a única em que os quatro braços correram no *mesmo servidor, no mesmo cenário e com a mesma dimensão amostral* — condição sem a qual uma comparação de tempos não significa nada. A duração e o número de passos de simulação são lidos dos registos de treino de cada execução (contador do treinador evolutivo; `total_timesteps` do *Stable-Baselines3* convertido de transições de agente para passos de ambiente, dividindo por $N = 20$), e reporta-se a mediana entre as 28 execuções. O tempo de inferência foi medido à parte, numa máquina de secretária e com um único fio de execução — o cenário embarcado —, cronometrando a propagação direta de uma decisão para os 20 agentes.

QUADRO 6.7. Custo de treino e de execução por algoritmo (mediana de 28 execuções no Muro em U; inferência medida em CPU, um fio, $N = 20$).

Braço	Núcleos	Duração (min)	Núcleos-hora	Passos sim. (10^6)	Passos/s por núcleo	Inferência (ms)
GNN adaptativo	30	195,2	97,6	24,1	68,4	0,12
GNN objetivo	30	195,2	97,6	23,8	67,7	0,12
PPO	16	48,0	12,8	0,65	14,2	0,06
SAC	16	47,6	12,7	0,46	10,0	0,06

O quadro responde à pergunta “qual é o mais eficiente?” com três respostas que apontam em sentidos diferentes, e é essa divergência que interessa reter.

- **Por orçamento de máquina**, os métodos de gradiente são claramente mais baratos: 12,8 contra 97,6 núcleos-hora por execução, uma razão de $7,6\times$ — o $\approx 8\times$ referido ao longo do capítulo.
- **Por eficiência amostral**, a distância é ainda maior e no mesmo sentido: o controlador evolutivo consome $\approx 37\times$ mais passos de simulação (24,1 contra 0,65 milhões) para atacar o mesmo cenário, porque uma busca sem gradiente tem de reavaliar a população inteira a cada geração em vez de aproveitar cada transição para um passo de gradiente.

- **Por aproveitamento de cada núcleo**, a relação *inverte-se*: o evolutivo sustenta 68,4 passos de simulação por segundo e por núcleo, contra 14,2 do PPO e 10,0 do SAC. A razão é estrutural — a evolução só faz propagação direta, ao passo que os métodos de gradiente pagam, além da simulação, a retropropagação; o SAC é o mais lento dos três por executar uma atualização de gradiente por passo do ambiente. Dito de outro modo: o evolutivo é caro porque *precisa* de muita simulação, não porque a faça mal.

Em *execução* — o regime que interessa a um robô real — a diferença é praticamente irrelevante: ambas as políticas decidem para os 20 agentes em menos de um sexto de milissegundo, cerca de vinte vezes menos do que dura um passo de simulação. O contraste é antes de *dimensão*: a rede de grafos com atenção usa $\approx 8,2$ mil parâmetros contra os ≈ 95 mil do ator MLP, é cerca de duas vezes mais lenta por decisão, e é a única das duas que aceita um número de agentes diferente daquele com que foi treinada (Secção 6.12). O custo do paradigma evolutivo está, portanto, todo do lado do treino — que é *offline* e se faz uma vez —, e não do lado do controlador que vai a bordo.

6.15. Discussão Global e Validação da Hipótese

A hipótese central desta investigação postula que a *inteligência adaptativa* (MARL) supera a *robustez estática* bio-inspirada em cenários de *stress* dinâmico. Os resultados da campanha final exigem uma resposta diferenciada, e não um veredicto único.

Desempenho de tarefa. Para quantificar as diferenças entre algoritmos aplicou-se o teste não-paramétrico de Mann-Whitney U sobre as *médias de chegadas por execução* ($n = 7$ por grupo — a unidade estatística independente é a execução de treino, não o episódio), com o δ de Cliff como tamanho de efeito; o Quadro 6.8 reporta os resultados.

A leitura do quadro desfaz a narrativa simples de um paradigma dominante. O controlador *evolutivo* é *significativamente superior a ambos os métodos de gradiente* em três cenários (Quatro Salas, Porta Cooperativa e Percepção Cooperativa, com $\delta \geq +0,71$) e superior ao SAC em mais dois (Gargalo e Porta com Alternativa, $\delta = +1,00$), empatando com o PPO nesses dois. Os métodos de gradiente são superiores no *Sandbox* (onde as execuções degeneradas do GNN pesam, $\delta = \pm 1,00$). No *Muro em U*, nenhuma comparação atinge significância: com desempenho bimodal nos três algoritmos, sete execuções não bastam para distinguir taxas de convergência de 2/7 a 4/7 — a conclusão é que o cenário permanece por resolver de forma fiável por qualquer dos três algoritmos base. É este impasse que a hibridização com procura por novidade veio a quebrar (7/7 execuções, Secção 6.10).

Anatomia do (antigo) colapso evolutivo — e da sua cura. Nas campanhas exploratórias, o controlador evolutivo colapsava na maioria das execuções dos cenários de gargalo, e a mecânica desse colapso merece registo porque fundamenta a QI5. A neuroevolução otimiza os pesos a partir de um único escalar de *fitness* por episódio, sem atribuição de crédito temporal: não dispõe de informação sobre *qual* ação, em *que* instante, contribuiu para o resultado. Com recompensa de tarefa esparsa, a quase totalidade dos genomas não faz uma única chegada, a *fitness* reduz-se ao termo de *shaping* e — na formulação inicial, saturada por uma *tanh* do retorno acumulado — converge para um valor comum a toda a população: um *planalto de fitness* onde a pressão

QUADRO 6.8. Significância das diferenças em chegadas por episódio (Mann-Whitney U sobre as médias por execução, $n = 7$ por algoritmo; δ de Cliff; $\alpha = 0,05$).

Cenário	Par	média A	média B	p	δ	Signif.
Sandbox	GNN vs PPO	38,31	71,46	0,0006	-1,00	Sim
Sandbox	GNN vs SAC	38,31	69,24	0,0021	-1,00	Sim
Sandbox	PPO vs SAC	71,46	69,24	0,0297	+0,71	Sim
Muro em U	GNN vs PPO	24,54	39,62	0,1552	-0,47	Não
Muro em U	GNN vs SAC	24,54	8,99	0,5714	+0,18	Não
Muro em U	PPO vs SAC	39,62	8,99	0,0526	+0,63	Não
Gargalo	GNN vs PPO	121,39	123,17	0,2086	+0,43	Não
Gargalo	GNN vs SAC	121,39	41,44	0,0006	+1,00	Sim
Gargalo	PPO vs SAC	123,17	41,44	0,0006	+1,00	Sim
Quatro Salas	GNN vs PPO	59,76	33,58	0,0006	+1,00	Sim
Quatro Salas	GNN vs SAC	59,76	31,82	0,0006	+1,00	Sim
Quatro Salas	PPO vs SAC	33,58	31,82	0,3374	+0,33	Não
Porta Cooperativa	GNN vs PPO	69,81	67,06	0,0262	+0,71	Sim
Porta Cooperativa	GNN vs SAC	69,81	62,09	0,0006	+1,00	Sim
Porta Cooperativa	PPO vs SAC	67,06	62,09	0,0175	+0,76	Sim
Perceção Cooperativa	GNN vs PPO	18,98	15,33	0,0297	+0,71	Sim
Perceção Cooperativa	GNN vs SAC	18,98	16,11	0,0297	+0,71	Sim
Perceção Cooperativa	PPO vs SAC	15,33	16,11	0,0547	-0,63	Não
Porta com Alternativa	GNN vs PPO	86,70	85,25	0,8477	+0,08	Não
Porta com Alternativa	GNN vs SAC	86,70	68,61	0,0006	+1,00	Sim
Porta com Alternativa	PPO vs SAC	85,25	68,61	0,0021	+1,00	Sim

seletiva se anula e a evolução deambula (*fitness exploitation*). A cura não foi mais orçamento, mas *melhor desenho da fitness*: a substituição do retorno acumulado (farmável por vaguear) pelo *homing* terminal — que só é maximizável *terminando* junto ao ninho (Eq. 5.2) — restaurou o gradiente de seleção nos labirintos. O efeito é visível na campanha final: convergem as 28 execuções que perfazem os quatro cenários de gargalo (sete por cenário), com magnitudes que igualam ou superam os métodos de gradiente. O “colapso do evolutivo” era, portanto, um artefacto do desenho da *fitness*, e não uma limitação intrínseca do paradigma — com a ressalva de que esta sensibilidade ao desenho é, em si mesma, um custo de engenharia do paradigma evolutivo que os métodos de gradiente, com atribuição de crédito passo-a-passo, não pagam na mesma medida. Esta centralidade do desenho do sinal de treino encontra, aliás, convergência independente no grupo responsável pelo único estudo comparativo da revisão sistemática: no seu trabalho dedicado às estruturas de recompensa, Iskandar conclui igualmente que o desenho do sinal — esparsos, modelado ou aprendido por demonstração — é o fator que mais condiciona o comportamento e a aprendizagem do enxame [47].

Variância entre execuções. A distribuição da variância pelos cenários é, ela própria, informativa. O controlador evolutivo é *quase determinístico* nos labirintos com gradiente geodésico informativo (desvios de 1–2 chegadas/ep na Porta Cooperativa e na Porta com Alternativa), mas bimodal nos cenários abertos (Sandbox 5/7): sem paredes que canalizem o *homing*, a paisagem de *fitness* oferece ótimos locais de vaguear dos quais algumas execuções não escapam. O

SAC exibe o padrão inverso — estável em espaço aberto, lotaria nos gargalos físicos (Gargalo $41,4 \pm 36,8$, 5/7) —, sugerindo que a exploração por entropia máxima, eficaz em espaço aberto, não gera com fiabilidade as trajetórias longas e coordenadas que atravessam passagens estreitas. O PPO é o mais estável dos três, com um único cenário bimodal (Muro em U, 4/7). O Muro em U, transversalmente bimodal, isola o ingrediente que falta a todos: nenhum dos mecanismos de exploração estudados (entropia, *clipped surrogate*, mutação gaussiana) garante a descoberta do desvio comprido. Note-se que o obstáculo não está no sinal de treino — o potencial geodésico aponta corretamente para o contorno (Secção 4.3) —, mas na *descoberta*: o agente observa apenas a bússola de *homing* euclidiana e o LiDAR local, pelo que percorrer o desvio exige uma trajetória longa que se afasta, durante dezenas de passos, da única direção que o agente vê como o ninho. É precisamente esta a motivação da linha de *Novelty Search* (Secção 6.10).

Custo computacional e eficiência. A competitividade do evolutivo na tarefa tem um preço de cómputo, medido em detalhe na Secção 6.14.1 (Quadro 6.7): cada execução do GNN consumiu 195 minutos com ≈ 30 núcleos (avaliação paralela da população), contra 48 minutos com 16 núcleos do PPO/SAC — uma razão de $\approx 8\times$ em núcleos-hora, e de $\approx 37\times$ em passos de simulação consumidos. Esta assimetria é também o limite da comparação: como a verificação da Secção 6.1 mostra, o orçamento não foi suficiente para atingir o planalto em sete das 21 células — quatro delas do SAC, incluindo as duas subidas mais marcadas —, pelo que a leitura correta é a de *desempenho atingível dentro do orçamento concedido*, com os valores dessas células a funcionarem como limite inferior. A conclusão que resiste é a inversa da que o cómputo sugeriria: mesmo com $8\times$ menos núcleos-hora, os métodos de gradiente igualam ou superam o evolutivo em quatro dos sete cenários — em *eficiência amostral e computacional* a sua vantagem é clara, e deve pesar na escolha de paradigma quando o orçamento de treino é o recurso escasso.

Escalabilidade. No eixo da transferência *Zero-Shot* (Secção 6.12), o controlador evolutivo com atenção sobre grafo é o *único* viável, mantendo 100% de sucesso de $N = 10$ a $N = 100$ sem retreino *nos sete cenários* — com a eficiência per capita em $N = 100$ a reter 58–90% nos cenários com paredes (contra 39–45% nos abertos), onde um colapso por congestionamento seria a expectativa natural — enquanto as políticas MLP do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$. A adaptabilidade que a hipótese atribuíra ao MARL manifesta-se, assim, não no algoritmo de otimização, mas na *representação* (grafo com atenção vs. vetor de dimensão fixa).

Robustez. Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada (retenção de 92–106%, Secção 6.13), sem o colapso que a hipótese poderia antecipar para o controlador estático — o *parameter sharing* confere redundância funcional transversal.

Composição de dificuldades. Cada um dos sete cenários isola uma dificuldade; o oitavo compõe quatro delas num só labirinto, quatro vezes maior (Secção 6.11), e é aí que o panorama acima encontra o seu limite. Sem retreino, nada transfere: as 84 células do estudo de transferência ficam todas a 0,00 chegadas por episódio, em 1680 episódios, e três condições de controlo excluem que a causa seja a escala da observação, os obstáculos dispersos ou as dimensões

associadas à porta. Com treino nativo no mapa, o controlador evolutivo é o único que o resolve — em 4 das 21 execuções, contra 0 de 21 do PPO e 0 de 21 do SAC —, abaixo do limiar de 15 execuções convergentes que o pré-registo fixou antes dos dados. Duas leituras se somam ao que precede. A primeira é sobre a *forma* do resultado e não sobre a sua média: as execuções que resolvem o mapa fazem-no com magnitude comparável entre si e as restantes ficam a zero, ou seja, a composição de dificuldades degrada sobretudo a *fiabilidade* com que se aprende — é a assinatura bimodal do Muro em U, à escala de um mapa com quatro dificuldades encadeadas. A segunda é sobre o alcance dos eixos de escolha mapeados nesta secção: eles valem no regime em que uma dificuldade domina de cada vez; composta a dificuldade, a diferença entre paradigmas ($p = 0,3058$ e $\delta = +0,19$ entre o evolutivo e cada um dos métodos de gradiente) torna-se menor do que a diferença entre execuções do *mesmo* paradigma. É a fiabilidade do treino, e não a escolha do paradigma, que passa a ser o problema dominante — e é também a razão pela qual a dosagem adaptativa da novidade, que é o mecanismo que nesta dissertação demonstrou comprar fiabilidade, é o candidato mais imediato a trabalho futuro neste mapa.

Veredicto. A hipótese confirma-se apenas *parcialmente*, e o retrato final é mais rico do que o antecipado. Com o desenho adequado da *fitness*, o controlador evolutivo com atenção sobre grafo deixou de ser o elo fraco na tarefa: é estatisticamente superior aos métodos de gradiente em três dos sete cenários, indistinguível do melhor deles em mais dois, e é o único a escalar para dimensões de enxame não vistas. Os métodos de gradiente preservam duas vantagens sólidas: a *fiabilidade em espaço aberto* (onde o evolutivo ocasionalmente degenera) e a *eficiência computacional* ($\approx 8\times$ menos núcleos-hora por execução). O resultado mais valioso para a engenharia de exames não é eleger um vencedor, mas mapear estes eixos de escolha — estrutura do cenário, variabilidade da dimensão do enxame e orçamento de cómputo — e assinalar o resultado que fecha o balanço: a exploração fiável sob decepção espacial (Muro em U), que nenhum dos três algoritmos base resolveu de forma consistente, só cedeu à hibridização com pressão por novidade (7/7 execuções, Secção 6.10) — um mecanismo que, com peso fixo, degrada o desempenho onde o gradiente da tarefa já basta, mas cuja *dosagem adaptativa* (campanha pré-registada, mesma secção) elimina esse custo mantendo o ganho, reforçando a tese central de que a escolha e a dosagem dos ingredientes de treino devem responder à estrutura do cenário.

CAPÍTULO 7

Conclusões e Trabalhos Futuros

7.1. Conclusões

Esta dissertação comparou dois paradigmas de controlo descentralizado de enxames. A revisão sistemática mostrou que as comunidades de *Multi-Agent Reinforcement Learning* (MARL) e de otimização bio-inspirada quase não se comparam entre si (um estudo em 58), e o trabalho respondeu a essa lacuna com uma comparação sob protocolo comum: desenvolveu-se um simulador próprio, implementou-se o *framework* RS2C (com PPO e SAC) e um controlador neuroevolutivo assente numa rede de grafos com atenção, e compararam-se os três algoritmos nos mesmos sete cenários, com 7 execuções independentes por combinação, sob avaliação determinística emparelhada e tratamento estatístico não-paramétrico.

Os resultados (Capítulo 6) não elegem um vencedor universal, mas desenharam um mapa de escolha claro. No cumprimento efetivo da tarefa, e ao contrário do que as campanhas exploratórias sugeriam, o controlador evolutivo com *fitness* de *homing* é estatisticamente superior aos métodos de gradiente em três cenários (Quatro Salas, Porta Cooperativa e Perceção Cooperativa) e indistinguível do PPO nos restantes cenários de gargalo, convergindo em todas as execuções dos quatro cenários de gargalo; o antigo “colapso do evolutivo” revelou-se um artefacto do desenho da *fitness* (*fitness exploitation*), curado pela substituição do retorno acumulado pelo *homing* terminal. Os métodos de gradiente preservam a fiabilidade em espaço aberto (onde o evolutivo ocasionalmente degenera) e uma vantagem de $\approx 8\times$ em núcleos-hora por execução. Na transferência *Zero-Shot*, só a arquitetura de atenção sobre grafo escala de $N = 10$ a $N = 100$ sem retreino, mantendo 100% de sucesso, enquanto as políticas MLP de dimensão fixa do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$ — a adaptabilidade que a hipótese atribuída ao MARL manifesta-se, afinal, na *representação*, não no algoritmo de otimização. Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada (retenção de 92–106%), conferida pela partilha de parâmetros. O cenário de decepção espacial (Muro em U), bimodal para os três algoritmos base, só foi resolvido de forma consistente pela hibridização da neuroevolução com procura por novidade (7/7 execuções com orçamento igualado) — mecanismo que, com peso fixo, degradou o desempenho no cenário *deceptive* que o objetivo puro já resolvia, mostrando que a exploração por novidade é um instrumento direcionado e não um melhoramento universal. Uma campanha final *pré-registada* de *dosagem adaptativa* do peso de novidade (decaimento após a descoberta) fechou esta tensão: manteve os 7/7 no Muro em U ($p = 0,009$ face ao objetivo), não pagou custo significativo em nenhum dos restantes seis cenários, e — com o dobro do orçamento, no braço exploratório — atingiu no cenário *deceptive* o melhor resultado da dissertação ($88,7 \pm 0,6$ chegadas/ep), enquanto o objetivo puro, mesmo com esse dobro do tempo, continuou bimodal no Muro em U (4/7): o ganho é do mecanismo,

não do orçamento. Uma replicação final com 28 execuções por braço elevou este resultado de indicação a evidência robusta — 28/28 execuções resolvidas contra 15/28 do objetivo puro e 14/28 de cada método de gradiente —, mostrando ainda que o GNN *objetivo* e o PPO são indistinguíveis entre si neste cenário: o que separa os braços não é o paradigma de otimização, mas a dosagem da exploração.

Um oitavo cenário, construído para *compor* as dificuldades que os sete isolam — gargalo, beco em U, quatro salas e porta cooperativa, num labirinto de 103×62 m —, mostra onde estas conclusões se degradam primeiro. Nenhum dos vinte e um campeões faz uma única chegada ao ninho sem retreino; com treino nativo, o GNN resolve o mapa em 4 das 21 execuções, abaixo do limiar fixado antes dos dados. O que a composição degrada primeiro é a *fiabilidade* com que se consegue: as execuções que resolvem o mapa fazem-no com magnitudes parecidas entre si e as restantes ficam a zero — a mesma bimodalidade que o Muro em U já expunha, agora num mapa quatro vezes maior. Para uma aplicação, isto significa treinar várias vezes e seleccionar, que é precisamente a prática que a dosagem adaptativa de novidade tornou dispensável nos dois cenários de decepção, o Muro em U e a Porta com Alternativa.

Confirma-se, assim, apenas *parcialmente* a hipótese: a escolha do paradigma deve ser ditada pelos eixos de exigência dominantes da aplicação — estrutura do cenário e orçamento de cómputo (favoráveis ao gradiente em espaço aberto e a orçamentos curtos) versus navegação estruturada e escalabilidade a dimensões não vistas (favoráveis ao controlador de grafo com *fitness* bem desenhada). Estas conclusões assentam em 7 execuções independentes por configuração, com significância avaliada ao nível da execução — com exceção do único cenário em que a leitura depende de contagens e não de médias, o Muro em U, que a replicação final elevou a 28 execuções por braço. Onde o alargamento não chegou (Secção 7.5), as estimativas de variância dos cenários bimodais continuam por refinar.

7.2. Resposta às Questões de Investigação

Retomam-se as questões de investigação enunciadas na Secção 1.4, pelos dois níveis em que foram formuladas: primeiro as quatro questões gerais de comparação entre paradigmas, depois as três questões específicas sobre os mecanismos que produzem esse desempenho.

Nível 1 — a comparação entre paradigmas.

QI1 — Desempenho de tarefa: Não há domínio de paradigma, nem na navegação cooperativa até ao ninho nem na percepção cooperativa: com a *fitness* de *homing*, o controlador neuroevolutivo é estatisticamente superior aos métodos de gradiente em três dos sete cenários (incluindo os dois cooperativos com magnitude comparável) e indistinguível do PPO nos restantes cenários de gargalo; o PPO é o mais consistente entre execuções e os métodos de gradiente mantêm vantagem clara nos cenários abertos, onde o evolutivo ocasionalmente degenera. O único cenário por resolver de forma fiável — por qualquer dos três algoritmos base — é o de decepção espacial (Muro em U) (Capítulo 6).

QI2 — Escalabilidade: Sim, mas apenas para a arquitetura de grafo: o controlador com atenção transfere de $N = 10$ a $N = 100$ agentes sem retreino, com *100% de sucesso nas 28 combinações* cenário $\times$ dimensão — incluindo os labirintos e as tarefas cooperativas,

onde a objeção do congestionamento seria mais plausível. A eficiência per capita em $N = 100$ retém 58–90% nos cenários com paredes (máximo nas duas Portas Cooperativas, mínimo no Gargalo, onde a passagem única impõe algum custo de fluxo) e dilui-se nos abertos (39% no Sandbox, 45% na Percepção Cooperativa), por partilha de um recurso finito e não por falha de coordenação. As políticas MLP de dimensão fixa do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$. A capacidade de *Zero-Shot Transfer* revelou-se, portanto, uma propriedade da *representação* (invariância à permutação do grafo), e não do algoritmo de otimização.

QI3 — Robustez a falhas: Todos os paradigmas exibem resiliência elevada à falha súbita de 10% dos agentes, com retenção de desempenho próxima da integral. Esta robustez é conferida pela partilha de parâmetros, que torna o comportamento coletivo independente da identidade (e da sobrevivência) de agentes individuais.

QI4 — Critério de escolha: Não há um vencedor universal: em enxames de dimensão fixa e conhecida, o MARL por gradiente é preferível em espaço aberto e quando o orçamento de treino é escasso ($\approx 8\times$ menos núcleos-hora), ao passo que na navegação estruturada o controlador evolutivo com *fitness* de *homing* o iguala ou supera; quando a dimensão do enxame é variável ou desconhecida à partida, ou a missão exige escalar sem retreino, a arquitetura de grafo com atenção (aqui otimizada por neuroevolução) é a única opção viável entre as estudadas. A robustez a falhas, sendo transversal, não constitui critério discriminante.

Nível 2 — os mecanismos por detrás do desempenho.

QI5 — Desenho da função de *fitness*: Sim, de forma decisiva: o desempenho da neuroevolução revelou-se mais sensível ao desenho da *fitness* do que ao orçamento de treino. Com a *fitness* inicial — chegada mais o retorno acumulado do episódio, *farmável* por deambulação —, o controlador colapsava na maioria das execuções dos cenários de navegação complexa; a substituição desse termo pelo *homing* terminal (proximidade ao ninho no *fim* do episódio, medida no potencial geodésico, e por isso não-farmável) desbloqueou os quatro cenários de gargalo: na última campanha exploratória com a *fitness* inicial, só 3 de 12 execuções nesses cenários chegaram ao ninho durante o treino, e com o *homing* convergem as 28 execuções da campanha final, todas a 100% de sucesso. É uma comparação entre campanhas, e não uma ablação controlada: a campanha anterior não teve avaliação determinística e, com a *fitness*, mudou também o decaimento de σ (Secção 5.2.1). O resultado tornou o controlador evolutivo competitivo — e, em cenários selecionados, superior — face aos métodos de gradiente (Capítulo 6). O “colapso do evolutivo” é, portanto, um artefacto do desenho da *fitness*, e não uma limitação intrínseca do paradigma. A ressalva é o Muro em U: mesmo com o potencial geodésico a apontar corretamente para o desvio, o desempenho permanece bimodal (3/7 execuções), tal como sucede nos métodos de gradiente. O desenho da *fitness* é, pois, condição necessária mas não suficiente — resolve a atribuição de crédito,

não a descoberta do desvio comprido, que veio a exigir pressão explícita por novidade (QI6).

QI6 — Deceção e procura por novidade: Sim onde a deceção efetivamente morde — e com custo onde não morde. Nas campanhas com orçamento igualado (7 execuções de 195 minutos por braço), a hibridização com *Novelty Search* ($w = 0,5$) resolveu o Muro em U de forma consistente — 7/7 execuções a 100% de sucesso e $69,8 \pm 5,9$ chegadas/ep, contra 3/7 e $24,5 \pm 32,6$ do objetivo puro (Mann–Whitney $p = 0,026$, $\delta = +0,71$) —, tornando-se a única configuração estudada a dominar o cenário que resistia aos três algoritmos base. Simetricamente, na Porta com Alternativa, onde o *homing* já convergia em 7/7 execuções, a mesma pressão por novidade degradou a magnitude ($63,0 \pm 21,9$ vs. $86,7 \pm 2,0$; $p = 0,0006$, $\delta = -1,00$), desmascarando o +26% da comparação preliminar como artefacto do orçamento triplo. Com peso fixo, a resposta seria condicional: a procura por novidade melhora o resultado quando a otimização objetiva é inconsistente e prejudica-o quando o gradiente da tarefa basta — a exploração tem preço. A campanha final *pré-registada* de *dosagem adaptativa* (w decai após a descoberta sustentada) removeu, porém, a condição: manteve as 7/7 execuções no Muro em U ($68,5 \pm 13,1$; $p = 0,009$, $\delta = +0,76$ vs. objetivo), não degradou significativamente nenhum dos restantes seis cenários (T1/T3 do pré-registo) e superou o peso fixo em magnitude na Porta com Alternativa ($77,2$ vs. $63,0$; $\delta = +0,59$). O braço de controlo exploratório fecha o argumento: o objetivo puro com o *dobro* do orçamento continua bimodal no Muro em U (4/7), pelo que o ganho é do mecanismo e não do tempo de treino — e o adaptativo com esse orçamento atinge o melhor resultado da dissertação na Porta com Alternativa ($88,7 \pm 0,6$). A resposta final à QI6 é, pois, *afirmativa quando a novidade é doseada adaptativamente*: com temperagem após a descoberta, a pressão por novidade domina tanto o objetivo puro como o peso fixo, sem custo mensurável onde a deceção não morde (Secção 6.10). Uma campanha de replicação com $n = 28$ execuções por braço — quatro vezes a dimensão amostral dos testes acima — fecha a questão no plano da fiabilidade, que é o que o cenário exige: a dosagem adaptativa resolve o Muro em U em 28/28 execuções contra 15/28 do objetivo puro (Fisher exato, $p < 0,0001$), 14/28 do PPO e 14/28 do SAC, sem uma única execução falhada neste cenário; e supera o peso fixo na Porta com Alternativa com significância ($80,9$ vs. $63,0$; $p = 0,0016$, $\delta = +0,77$), confirmando o tamanho de efeito que a $n = 7$ ficara aquém do limiar convencional.

QI7 — Composição de dificuldades: Parcialmente, e ao preço da fiabilidade: o mapa composto é resolúvel pelo GNN, mas só em 4 das 21 execuções — abaixo do limiar de 15 fixado antes dos dados, pelo que a resposta à QI7 *não é afirmativa*. O que a composição degrada é sobretudo a *fiabilidade entre execuções* — as que resolvem o mapa fazem-no com magnitudes parecidas entre si e as restantes ficam a zero: é a assinatura bimodal que a dissertação já documenta no Muro em U (Secção 6.6), agora à escala de um mapa quatro vezes maior e com quatro dificuldades encadeadas (Secção 6.11).

A resposta é, por construção, condicionada ao orçamento: em 19 das 21 execuções o melhor *fitness* ainda subia no último quinto do treino, pelo que o que aqui se reporta é o que se alcança *dentro* deste orçamento, neste mapa e com este sinal de treino — e não um limite do paradigma (Secção 7.3).

7.3. Limitações do Estudo

Em prol da transparência científica, identificam-se as seguintes limitações da implementação atual:

- **Arquitetura assimétrica entre controladores:** as *baselines* MARL (PPO e SAC) empregam uma política totalmente ligada (MLP) sobre a observação de vizinhança densa, enquanto o mecanismo de atenção sobre grafo é instanciado apenas no controlador neuroevolutivo. Esta opção — ditada pelo uso das implementações de referência da *Stable-Baselines3* — implica que a comparação entre paradigmas combina dois fatores (algoritmo de otimização e arquitetura da rede). A integração do módulo de atenção numa política treinável por gradiente (*custom policy*) é a extensão prioritária de trabalho futuro, permitindo isolar o efeito do otimizador.
- **Escalabilidade restrita ao controlador de grafo:** como consequência do ponto anterior, o *Zero-Shot Transfer* para $N \neq 20$ só é diretamente avaliável no controlador com atenção; a MLP de entrada fixa do PPO/SAC é, por construção, incompatível com outras dimensões de enxame.
- **Crítico não-centralizado:** a fase de treino utiliza um crítico que parte da mesma observação local do ator (*parameter sharing*), não explorando o estado global. Trata-se de execução e treino descentralizados, e não de uma arquitetura CTDE com crítico centralizado em sentido estrito.
- **Percepção global dos companheiros de enxame:** a observação de cada agente inclui a direção, a distância e o sinal de comunicação de *todos* os outros agentes, sem limite de alcance e sem oclusão pelas paredes (Secção 4.2.2). Local é a percepção dos obstáculos, pelo LiDAR; a do enxame é global e idealizada — em $N = 100$, cada agente conhece a posição relativa dos outros 99. A simplificação é comum aos três controladores e não favorece nenhum na comparação, mas afasta o simulador das plataformas físicas, em que a deteção de vizinhos tem alcance limitado, e significa que a escalabilidade da Secção 6.12 foi medida sem as restrições de comunicação que um enxame real imporia.
- **Sim-to-real não validado:** a totalidade dos resultados é obtida em simulação; a transferência para plataformas físicas (*Deployment Gap*) permanece por validar empiricamente.
- **Dimensão vertical não observada, mas usada:** o simulador é tridimensional — a ação tem componente em z e a arena é uma esfera — ao passo que tudo o que as tarefas medem é planar: os labirintos vivem num plano, o *LiDAR* varre na horizontal, e tanto a grelha de exploração como o campo geodésico que suporta a recompensa de progresso são calculados na projeção horizontal. Daqui resultam duas assimetrias

que não são visíveis na descrição dos cenários. A primeira: a altitude de um agente não entra em observação nenhuma — nem na sua, nem na dos vizinhos —, pelo que dois agentes a alturas muito diferentes são, para efeitos de percepção, indistinguíveis. A segunda: a condição de chegada compara a distância euclidiana *tridimensional* ao centro do ninho, que está no plano $z = 0$, com o raio deste, pelo que acima de 1,6 m não há chegada possível — e, sem limite vertical, essa faixa improdutiva estende-se até ao raio da arena (13,4 m). A medição da altitude nos episódios registados mostra que os controladores ocupam essa faixa de facto: a altitude média por episódio vai de 0,13 a 5,67 m, com máximos de 14,6 m. Nessas mesmas 21 células (três controladores $\times$ sete cenários, um episódio determinístico cada), a altitude média associa-se negativamente às chegadas — $\rho_{\text{Spearman}} = -0,74$ sobre valores centrados dentro de cada cenário, e em cinco dos sete cenários o controlador que voa mais alto é o que menos chegadas faz. Com um episódio por célula, a associação é indicativa e não um teste; a direção, porém, é a esperada, e o mecanismo é direto: cada passo gasto a subir é um passo não gasto a percorrer o plano onde a tarefa vive. O movimento vertical é, assim, um grau de liberdade que consome tempo de episódio sem ser observado nem premiado; não invalida nenhuma comparação — é comum aos três controladores e a todos os cenários —, mas limita o desempenho absoluto de todos eles e deve ser restringido ao plano da tarefa em trabalho futuro, como já foi feito no oitavo cenário.

- **Duas costuras na física das paredes, exploradas por uma minoria de modelos:** as paredes são caixas com a altura do diâmetro da arena, centradas em $z = 0$, e a colisão resolve a interpenetração pela face de *menor* penetração nos três eixos. Daqui resultam duas costuras que a descrição dos cenários não mostra. A primeira está no teto: o topo de cada parede coincide com o limite da esfera, e um agente que lhe encoste a menos de 1 m do teto tem menor penetração em z do que em x ou y — é empurrado para cima, devolvido ao teto pela esfera, e desliza por cima da parede. A segunda está nas junções em T das Quatro Salas: a saída de uma caixa pela face mais próxima pode empurrar o agente para dentro da caixa vizinha, cuja resolução o expulsa pelo lado oposto. Nenhuma das duas é visível na avaliação, que só regista chegadas. Para as medir reproduziram-se 3 episódios determinísticos com cada um dos 343 modelos arquivados das dezasseis campanhas (campeões e modelos de execução; os 771 ficheiros contam uma vez os que várias fases copiaram), classificando como *travessia* a entrada numa parede por um lado e a saída pelo lado oposto, e como *preso* o agente que esteve dentro de uma parede sem a atravessar. O resultado: 11 modelos atravessam paredes (3 deles pelo teto) e 12 apenas ficam presos numa junção; nenhum é PPO ou SAC, e nenhum está no Sandbox, no Gargalo, na Porta Cooperativa, na Percepção Cooperativa ou no mapa composto. As travessias concentram-se nas Quatro Salas do controlador evolutivo — 8 dos 16 modelos únicos do GNN nesse cenário —, com dois casos residuais no Muro em U (2 modelos, 7 travessias) e um na Porta com Alternativa (1 modelo, 1 travessia), todos do mega-treino. Na campanha final, que

sustenta os quadros desta dissertação, 3 dos 70 modelos atravessam — as execuções 1, 4 e 6 do GNN nas Quatro Salas, a primeira e a última pelas junções e a quarta pelo teto — e as travessias não as tornam melhores: nos episódios reproduzidos, essas três execuções fazem em média 55,7 chegadas/ep contra 60,3 das quatro execuções limpas, e a distância a que qualquer destes valores fica dos métodos de gradiente nesse cenário (33,6 e 31,8 na avaliação oficial) é muito maior do que o efeito medido, pelo que a superioridade aí reportada não depende das travessias. Onde a costura pesa é na campanha de dosagem adaptativa: o seu campeão das Quatro Salas atravessa pelo teto em todos os episódios (67 travessias em 3), pelo que a célula Quatro Salas da comparação «sem custo nos restantes seis cenários» (Secção 6.10) está contaminada e não deve ler-se como evidência de desempenho. Com 3 episódios por modelo, as contagens são um limite inferior: um modelo que atravessasse raramente pode ter escapado. Como no ponto anterior, corrigir a colisão — tratar as paredes como infinitas em altura e resolver as junções em conjunto — alteraria a física de cenários com campanhas fechadas; fica declarado, com os modelos afetados identificados, para a repetição da campanha.

- **Sobreposição entre a zona de nascimento e a barreira, no Gargalo:** neste cenário a posição inicial de cada agente é sorteada na faixa $y \in [-12, -2]$, ao passo que a barreira ocupa $y \in [-4, 4]$ — os dois intervalos interseitam-se em dois metros. Em consequência, 17,9% das posições iniciais caem dentro do volume da barreira (179 de 1000: 50 episódios $\times$ 20 agentes). A origem é documental: o comentário que descreve o sorteio no código afirma que a barreira cobre $y \in [-1, 1]$, o que deixou de ser verdade quando a sua espessura passou a 8 m. O efeito prático, porém, é pequeno e não favorece ninguém. A separação física resolve a interpenetração *ao primeiro passo* — nenhum dos 179 permanece dentro da barreira, nem ao passo 1 nem ao passo 200 — e a resolução empurra-os para o lado de onde vêm: 159 para sul, 20 para a abertura central, *nenhum* para o lado oposto. Longe de encurtar o percurso, atrasa-o: medida no campo geodésico que a própria recompensa usa, a distância ao ninho após o primeiro passo é de 45,3 m (mediana) para quem nasceu dentro da barreira, contra 21,2 m para os restantes, e nenhum agente de qualquer dos grupos aparece junto ao ninho. Trata-se, pois, de uma penalização ligeira, idêntica para os três controladores e para todas as execuções, que não afeta nenhuma comparação; corrigir o sorteio implicaria alterar a física de um dos sete cenários e invalidar a comparabilidade das campanhas já fechadas, pelo que se declara em vez de se emendar.
- **Orçamento de treino insuficiente em sete das vinte e uma células:** a comparação asenta em orçamentos calibrados por paradigma para atingir o planalto de convergência, e a verificação *a posteriori* (Secção 6.1) mostra que em 7 das 21 combinações a curva ainda subia significativamente no último quinto do orçamento — destacadamente o SAC no Gargalo (+112%) e na Porta com Alternativa (+105%). A consequência mais séria recai sobre a leitura do SAC: a fragilidade que lhe é atribuída nos gargalos físicos

não é separável, com estes dados, de simples sub-treino. Acresce que os históricos consolidados da campanha guardaram a recompensa episódica mas não a métrica de tarefa por passo, pelo que nem sequer é possível decidir se esse crescimento final corresponde a mais chegadas ou a mais *shaping*. Uma repetição da campanha com orçamento estendido para o SAC — e com a métrica de tarefa registada ao longo do treino, o que o código já suporta — é a forma direta de fechar esta questão, e fica como trabalho imediato. Registar essa métrica em avaliações determinísticas periódicas, com as mesmas sementes para os três algoritmos, daria ainda a medida externa de progresso que as curvas atuais não oferecem, e permitiria comparar a *velocidade* de aprendizagem dos paradigmas, e não apenas o ponto a que cada um chega.

- **Alcance do estudo de composição:** a QI7 é respondida sobre *um* mapa composto — uma combinação particular de quatro dificuldades, numa geometria particular —, pelo que o que dele se conclui delimita o alcance dos três controladores nesta composição e não em qualquer outra. Acrescem três restrições declaradas antes dos dados e reportadas por inteiro na Secção 6.11: a contagem de execuções que resolvem o mapa mantém-se *descritiva* (o pré-registo fixou que não se inferiria sobre proporções, e mudar de teste depois de o n subir seria escolhê-lo à vista da amostra); o orçamento de 780 minutos por execução não chegou ao planalto, com o melhor *fitness* a surgir nos últimos 20% das gerações em 19 das 21 execuções; e a comparação com os sete cenários é de *alcance*, nunca célula a célula, por os protocolos e orçamentos serem diferentes. A resposta negativa à QI7 deve, por isso, ler-se como «não dentro deste orçamento, neste mapa e com este sinal de treino» — que é também o que a instrumentação por distância geodésica permite decompor (Secção 7.5).

7.4. Contributos

Do trabalho realizado resultam os seguintes contributos para a área de Sistemas Multiagente e Robótica de Enxame:

- **Benchmarking quantitativo entre paradigmas:** estabeleceu-se uma metodologia de comparação padronizada entre algoritmos de naturezas distintas (aprendizagem por gradiente vs. otimização evolutiva), aplicada a 3 controladores $\times$ 7 cenários $\times$ 7 execuções independentes, sob avaliação determinística emparelhada (20 episódios com sementes fixas) e inferência não-paramétrica ao nível da execução (Mann-Whitney U com δ de Cliff). A unidade estatística é a execução de treino — e não o episódio —, o que evita a inflação de n que é comum na literatura comparada.
- **O desenho da *fitness* como causa (e cura) do colapso evolutivo:** demonstrou-se que o colapso da neuroevolução nos cenários de navegação complexa — reportado como limitação do paradigma — era, neste sistema, um *artefacto do sinal de treino*: com uma *fitness* baseada no retorno acumulado, a população satura num planalto onde a pressão seletiva se anula (*fitness exploitation*). A substituição pelo *homing* terminal, que só é maximizável terminando junto ao ninho, restaurou o gradiente de seleção e levou o controlador a convergir nas 28 execuções que perfazem os quatro cenários de

gargalo (Gargalo, Quatro Salas e as duas Portas Cooperativas, sete execuções cada), contra 3 de 12 execuções que chegavam ao ninho, em treino, na última campanha com a *fitness* inicial. Este é o principal contributo metodológico do trabalho.

- **Escalabilidade *Zero-Shot* como propriedade da representação:** mostrou-se empiricamente que o controlador com atenção sobre grafo transfere de $N = 10$ para $N = 100$ agentes sem retreino e sem perda de sucesso (100% nas 28 combinações cenário $\times$ dimensão), retendo entre 58% e 90% da eficiência per capita nos cenários estruturados, enquanto as políticas MLP de dimensão fixa são estruturalmente incompatíveis com $N \neq 20$ — respondendo ao desafio de escalabilidade colocado por Chen et al. (2024) [16] e localizando a adaptabilidade na *arquitetura*, não no algoritmo de otimização.
- **Caracterização condicional da procura por novidade:** quantificou-se, com orçamento de treino igualado, que a pressão por novidade resolve o cenário de decepção espacial que resistia aos três algoritmos base (7/7 execuções, contra 3/7 do objetivo puro) mas *degrada* o desempenho onde o gradiente da tarefa já é informativo — custo que a dosagem adaptativa do peso, testada numa campanha pré-registada, elimina. A exploração é, assim, um instrumento direcionado e não um melhoramento universal — um resultado que contraria a leitura otimista que uma comparação com orçamentos desiguais sugeriria.
- **Mapa de escolha para engenharia de exames:** em vez de eleger um vencedor universal, o trabalho mapeia os eixos que devem ditar a escolha do paradigma — estrutura do cenário, variabilidade da dimensão do exame e orçamento de cómputo ($\approx 8\times$ mais núcleos-hora para o evolutivo) — sintetizados na discussão do Capítulo 6.
- **Um mapa composto como instrumento, e um resultado negativo reportado por compromisso:** construiu-se um oitavo cenário que *compõe* num único labirinto quatro das dificuldades que os sete isolam, com o desenho auditado antes de existirem dados (existe caminho do nascimento ao ninho para todos os agentes mesmo com a porta fechada, e um navegador geodésico resolve-o), e mediram-se nele os três controladores em duas fases pré-registadas. O contributo é duplo. Em substância: a transferência sem retreino é nula e, com treino nativo, só o controlador evolutivo resolve o mapa, em 4 de 21 execuções — pelo que a composição de dificuldades degrada primeiro a *fiabilidade* do treino: as execuções que resolvem o mapa fazem-no com magnitudes parecidas entre si, e as restantes ficam a zero. Em método: a questão foi respondida com hipótese, métricas, regra de decisão e compromissos de reporte fixados *antes* dos dados, e o resultado, tendo ficado abaixo do limiar declarado, é reportado como negativo com o número de execuções convergentes à vista — incluindo as emendas ao protocolo, datadas, e um braço pré-registado que acabou por não correr. É a prática que a Secção 6.11 documenta, e não apenas o seu desfecho, que se propõe como contributo.
- **Implementação de referência extensível:** disponibiliza-se um repositório aberto — <https://github.com/Pombo2001/swarm-robotics-tese> (Anexo E) — com o

simulador, os sete cenários, o *framework* RS2C (PPO/SAC) e o controlador neuroevolutivo com atenção. A arquitetura foi organizada para que a extensão seja barata onde importa (um algoritmo novo exige uma classe, um módulo de treino e uma entrada num registo, passando automaticamente a integrar campanhas, avaliação, estatística e figuras), com as garantias de não-regressão e a dívida técnica remanescente declaradas explicitamente na Secção 4.5 — de modo que quem continue este trabalho saiba, à partida, o que herda.

7.5. Trabalhos Futuros

A partir das limitações identificadas (Secção 7.3) e do estado atual da experimentação, delineiam-se as seguintes linhas de trabalho prioritárias:

- (1) **Alargamento da bateria estatística onde ainda é $n = 7$:** a replicação a 28 execuções por braço fechou a questão no cenário que mais a exigia — o Muro em U, onde a leitura depende de contagens e não de médias (Secção 6.10) —, mas o alargamento foi *assimétrico* e deixa três frentes por cobrir. Primeiro, nas células exploratórias do Sandbox e da Percepção Cooperativa só o braço *adaptativo* correu a 21 execuções, mantendo-se os comparadores a 7; é essa assimetria de dimensões amostrais, e não a direção do efeito, que mantém o $\delta = +0,39$ do Sandbox longe do limiar convencional, pelo que alargar o braço de controlo é o que falta para o decidir. Segundo, as 21 execuções do SAC no Gargalo tiveram os 48 minutos da campanha final, mas 15 delas fizeram só um terço dos passos: fixar o orçamento em passos, e não em minutos, é o que falta para replicar o 5/7 dessa combinação sem depender da carga do servidor. Terceiro, as vinte e uma combinações da campanha final permanecem a 7 execuções — dimensão suficiente para a inferência sobre médias, mas curta para contrastar taxas de convergência, que é a leitura que os cenários bimodais exigem.
- (2) **Política de gradiente com atenção (*custom policy*):** Integrar o mecanismo de atenção sobre grafo numa política treinável por gradiente (PPO/SAC com extrator de *features* personalizado na *Stable-Baselines3*), de modo a *isolar* o efeito do algoritmo de otimização do efeito da arquitetura — removendo a assimetria MLP vs. grafo que atualmente confunde a comparação (Secção 7.3).
- (3) **Novos cenários e tarefas:** Implementar tarefas de cobertura/vigilância de área (para além da navegação cooperativa até ao ninho e do cerco de alvos móveis), incluindo *foraging* propriamente dito, com recolha e transporte de objetos até ao ninho, e cenários com percursos alternativos, ampliando o espaço de avaliação de comportamentos coletivos.
- (4) **Validação *Sim-to-Real*:** Transferir as políticas para plataformas físicas (e.g. e-puck, Khepera IV) ou para um simulador de física de maior fidelidade, quantificando empiricamente o *Deployment Gap*.
- (5) **Hibridização de paradigmas:** Explorar arquiteturas que combinem a escalabilidade do controlador evolutivo com a eficiência de tarefa dos métodos de gradiente (e.g. inicialização evolutiva seguida de afinação por RL).

- (6) **Refinamento da dosagem adaptativa da novidade:** A campanha pré-registada da Secção 6.10 validou a temperagem do peso w após a descoberta; ficam por explorar variantes do *schedule* — o re-armamento de w quando a população estagna (a temperagem atual nunca re-arma), caracterizações comportamentais mais ricas do que o centroide final, e a transposição do mesmo princípio de dosagem para os mecanismos de exploração dos métodos de gradiente (*e.g.*, coeficiente de entropia do SAC dependente do progresso).
- (7) **O mapa composto, com o orçamento separado em três causas:** a QI7 fecha com 4 das 21 execuções a resolver o mapa (Secção 6.11), e a instrumentação por distância geodésica permite dizer *o que falta* — que não é uma causa só. Primeira: o *orçamento de treino* não chegou ao planalto. Em 19 das 21 execuções o melhor *fitness* surge nos últimos 20% das gerações, com a geração do máximo a cair, em mediana, a 95% do orçamento; nas quatro execuções que resolvem o mapa, a descoberta traduz-se num salto de *fitness* de mais de uma ordem de grandeza nessa fase final. Alargar os 780 minutos por execução é, portanto, a via com mais indicação empírica, e a que o *schedule* da novidade (item anterior) mais beneficiaria. Segunda: o *horizonte do episódio* aproxima quase todas as execuções do ninho, mas converte poucas em chegada. Duplicar `max_steps` de 2000 para 4000 *sem qualquer retreino* faz a distância mínima mediana ao ninho cair de 27,6 para 12,4m e quase duplica a magnitude da melhor execução (de 12 para 22 chegadas por episódio), mas leva o número de execuções com pelo menos uma chegada apenas de 4 para 6 em 21. É uma distinção com consequência: mais tempo de episódio traduz-se em mais percurso feito, e não em mais execuções a fechar a tarefa. Terceira, e é o que essa assimetria revela: há 5 execuções que param a menos de 13m do ninho — 3 delas a menos de 5m — e não entram nele mesmo com o dobro do tempo, o que aponta para a fase final de aproximação — e não para o orçamento — como o alvo do próximo refinamento do sinal de treino ou da observação. As medições estão em `results/mapa_grande/`, com três episódios determinísticos por execução contra os 20 da avaliação oficial: são diagnóstico para orientar trabalho futuro, não substituem o protocolo de avaliação — ainda que, ao horizonte pré-registado, reproduzam exatamente as 4 execuções convergentes que este reporta.
- (8) **Treino evolutivo não-geracional:** Implementar uma variante *steady-state* do treino evolutivo, com avaliação assíncrona e substituição de um genoma de cada vez, e compará-la com o ciclo geracional usado em todas as campanhas desta dissertação (Secção 5.2.2) — no aproveitamento dos núcleos, no tempo até à descoberta nos cenários enganadores e na fiabilidade entre execuções.
- (9) **Aprendizagem por currículo como via alternativa de descoberta:** O grupo responsável pelo único estudo da revisão sistemática que compara os dois paradigmas mitiga o sobreajuste do RL a condições não vistas com aprendizagem por currículo [47].

Aplicá-la aos cenários decetivos desta bateria — arenas progressivamente mais fechadas, culminando no Muro em U e na Porta com Alternativa — permitiria confrontar currículo e pressão por novidade, dois mecanismos distintos para o mesmo problema (a *descoberta*), sob o protocolo estatístico e o orçamento igualado aqui estabelecidos.

Referências Bibliográficas

- [1] E. Bonabeau, M. Dorigo e G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems* (Santa Fe Institute Studies in the Sciences of Complexity). Oxford University Press, 1999.
- [2] M. Tegmark, *Life 3.0: Being Human in the Age of Artificial Intelligence*. Alfred A. Knopf, 2017.
- [3] M. Dorigo, «Optimization, Learning and Natural Algorithms,» tese de doutoramento, Politecnico di Milano, 1992.
- [4] J. Kennedy e R. Eberhart, «Particle swarm optimization,» em *Proceedings of ICNN'95 - International Conference on Neural Networks*, IEEE, vol. 4, 1995, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968
- [5] C. C. Ekechi, T. Elfouly, A. Alouani e T. Khattab, «A Survey on UAV Control with Multi-Agent Reinforcement Learning,» *Drones*, vol. 9, n.º 7, p. 484, 2025. DOI: 10.3390/drones9070484
- [6] R. Pina, V. De Silva e C. Artaud, «Towards Self-Adaptive Resilient Swarms Using Multi-Agent Reinforcement Learning,» em *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods (ICPRAM)*, SciTePress, 2024, pp. 410–417. DOI: 10.5220/0012462800003654
- [7] A. Y. Majid, S. Saaybi, V. Francois-Lavet, R. V. Prasad e C. Verhoeven, «Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey,» *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, n.º 9, pp. 11 939–11 957, 2024. DOI: 10.1109/TNNLS.2023.3264540
- [8] M. Hüttenrauch, A. Šošić e G. Neumann, «Deep Reinforcement Learning for Swarm Systems,» *Journal of Machine Learning Research*, vol. 20, n.º 54, pp. 1–31, 2019.
- [9] W. M. Hassen e S. H. Amin, «Dynamic Window Approach for Mobile Robot Path Planning based on Hybrid Swarm Algorithms,» *Journal of Information Hiding and Multimedia Signal Processing*, vol. 16, n.º 2, pp. 721–736, 2025.
- [10] T. Salimans, J. Ho, X. Chen, S. Sidor e I. Sutskever, «Evolution Strategies as a Scalable Alternative to Reinforcement Learning,» *arXiv preprint arXiv:1703.03864*, 2017.
- [11] S. Somvanshi, M. M. Islam, S. A. Javed, G. Chhetri, K. S. Islam, T. I. Chowdhury, S. B. B. Pollock, A. Dutta e S. Das, «A Review on Influx of Bio-Inspired Algorithms: Critique and Improvement Needs,» *arXiv preprint arXiv:2506.04238*, 2025.
- [12] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2ª ed. MIT Press, 2018.
- [13] D. Huh e P. Mohapatra, «Multi-agent Reinforcement Learning: A Comprehensive Survey,» *arXiv preprint arXiv:2312.10256*, 2023.
- [14] M. Bettini, A. Prorok e V. Moens, «BenchMARL: Benchmarking Multi-Agent Reinforcement Learning,» *Journal of Machine Learning Research*, vol. 25, n.º 217, pp. 1–10, 2024.
- [15] M. Kegeleirs e M. Birattari, «Towards applied swarm robotics: current limitations and enablers,» *Frontiers in Robotics and AI*, vol. 12, p. 1 607 978, 2025. DOI: 10.3389/frobt.2025.1607978

- [16] X. Chen, N. NaderiAlizadeh, A. Ribeiro e S. Saeedi Bidokhti, «Transferable Graphical MARL for Real-Time Estimation in Dynamic Wireless Networks,» *arXiv preprint arXiv:2404.03227*, 2024, versão 1.
- [17] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley e J. Clune, «Deep Neuro-evolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning,» *arXiv preprint arXiv:1712.06567*, 2017.
- [18] B. Rivière, W. Hönig, Y. Yue e S.-J. Chung, «GLAS: Global-to-Local Safe Autonomy Synthesis for Multi-Robot Motion Planning with End-to-End Learning,» *IEEE Robotics and Automation Letters*, vol. 5, n.º 3, pp. 4249–4256, 2020. DOI: 10.1109/LRA.2020.2994035
- [19] N. Bredeche e N. Fontbonne, «Social learning in swarm robotics,» *Philosophical Transactions of the Royal Society B: Biological Sciences*, vol. 377, n.º 1843, p. 20200309, 2022. DOI: 10.1098/rstb.2020.0309
- [20] J. Lehman e K. O. Stanley, «Abandoning Objectives: Evolution Through the Search for Novelty Alone,» *Evolutionary Computation*, vol. 19, n.º 2, pp. 189–223, 2011. DOI: 10.1162/EVCO_a_00025
- [21] K. Hasselmann, A. Ligot e M. Birattari, «Automatic modular design of robot swarms based on repertoires of behaviors generated via novelty search,» *Swarm and Evolutionary Computation*, vol. 83, p. 101395, 2023. DOI: 10.1016/j.swevo.2023.101395
- [22] T. U. Zaman, P. Biteng e Q. Lu, «Evolving Adaptive Foraging Robot Swarms with NEAT in Environments with Obstacles,» em *8th International Conference on Intelligent Robotics and Control Engineering (IRCE)*, IEEE, 2025, pp. 33–38. DOI: 10.1109/IRCE66030.2025.11203043
- [23] P. Biteng, T. U. Zaman e Q. Lu, «Training Robot Swarms for Adaptive Foraging in Environments with Obstacles,» em *IEEE International Conference on Mechatronics and Automation (ICMA)*, IEEE, 2025, pp. 341–346. DOI: 10.1109/ICMA65362.2025.11120654
- [24] N. Fontbonne, O. Dauchot e N. Bredeche, «Distributed On-line Learning in Swarm Robotics with Limited Communication Bandwidth,» em *IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2020, pp. 1–8. DOI: 10.1109/CEC48606.2020.9185697
- [25] F. A. Oliehoek e C. Amato, *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, «Proximal Policy Optimization Algorithms,» *arXiv preprint arXiv:1707.06347*, 2017.
- [27] J. Schulman, P. Moritz, S. Levine, M. I. Jordan e P. Abbeel, «High-Dimensional Continuous Control Using Generalized Advantage Estimation,» em *International Conference on Learning Representations (ICLR)*, 2016.
- [28] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine, «Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,» em *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [29] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel e I. Mordatch, «Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [30] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò e Y. Bengio, «Graph Attention Networks,» em *International Conference on Learning Representations (ICLR)*, 2018.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin, «Attention is All You Need,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [32] H. Zhang, J. Cheng, L. Zhang, Y. Li e W. Zhang, «H2GNN: Hierarchical-Hops Graph Neural Networks for Multi-Robot Exploration in Unknown Environments,» *IEEE Robotics*

- and Automation Letters, vol. 7, n.º 2, pp. 3435–3442, 2022. DOI: 10.1109/LRA.2022.3146912
- [33] Y. Wang, T. Duhan, J. Li e G. Sartoretto, «LNS2+RL: Combining Multi-Agent Reinforcement Learning with Large Neighborhood Search in Multi-Agent Path Finding,» em *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 39, 2025, pp. 23 343–23 350. DOI: 10.1609/aaai.v39i22.34501
 - [34] P. Li, J. Hao, H. Tang, Y. Zheng e X. Fu, «RACE: Improve Multi-Agent Reinforcement Learning with Representation Asymmetry and Collaborative Evolution,» em *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19 490–19 503.
 - [35] O. Yigit, S. E. Ceylan, S. S. Palas, A. Isik, E. Bekar, B. Vatansever e Y. Oniz, «Multi Agent Reinforcement Learning Based Swarm Navigation,» em *2026 8th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*, IEEE, 2026, pp. 1–6. DOI: 10.1109/ICHORA69329.2026.11537199
 - [36] A. Iskandar, A. Hammoud e B. Kovács, «Swarm Robotics Navigation Task: A Comparative Study of Reinforcement Learning and Particle Swarm Optimization Methodologies,» *Mekhatronika, Avtomatizatsiya, Upravlenie*, vol. 25, n.º 9, pp. 471–478, 2024. DOI: 10.17587/mau.25.471-478
 - [37] M. Kegeleirs, D. G. Ramos, K. Hasselmann, L. Garattoni, G. Francesca e M. Birattari, «Transferability in the Automatic Off-Line Design of Robot Swarms: From Sim-to-Real to Embodiment and Design-Method Transfer Across Different Platforms,» *IEEE Robotics and Automation Letters*, vol. 9, n.º 3, pp. 2758–2765, 2024. DOI: 10.1109/LRA.2024.3360013
 - [38] J. Szpirer, D. G. Ramos e M. Birattari, «Automatic design of robot swarms that perform composite missions: an approach based on inverse reinforcement learning,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2024, pp. 5791–5798. DOI: 10.1109/IROS58592.2024.10801506
 - [39] E. Ordáz-Rivas, A. Rodríguez-Liñán e L. Torres-Treviño, «Multi-objective optimization in autonomous foraging using swarm robots,» *Swarm and Evolutionary Computation*, vol. 101, p. 102 294, 2026. DOI: 10.1016/j.swevo.2026.102294
 - [40] H. Tang, H. Zhang, Z. Shi, X. Chen, W. Ding e X.-P. Zhang, «Autonomous Swarm Robot Coordination via Mean-Field Control Embedding Multi-Agent Reinforcement Learning,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2023, pp. 8820–8826. DOI: 10.1109/IROS55552.2023.10341749
 - [41] X. Li, R. Zhou e G. Sun, «Adaptive Shape Formation Against Swarm-Scale Variants in Robot Swarms,» *IEEE Robotics and Automation Letters*, vol. 9, n.º 8, pp. 6776–6783, 2024. DOI: 10.1109/LRA.2024.3414210
 - [42] A. Iskandar, H. Rostum e B. Kovács, «Using Deep Reinforcement Learning to Solve a Navigation Problem for a Swarm Robotics System,» em *2023 24th International Carpathian Control Conference (ICCC)*, IEEE, 2023, pp. 185–189. DOI: 10.1109/ICCC57093.2023.10178888
 - [43] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba e P. Abbeel, «Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133
 - [44] M. Duarte, F. Silva, T. Rodrigues, S. Moura Oliveira e A. L. Christensen, «JBotEvolver: A Versatile Simulation Program for Evolutionary Robotics,» em *Artificial Life 14: Proceedings of the Fourteenth International Conference on the Synthesis and Simulation of Living Systems*, MIT Press, 2014, pp. 210–211. DOI: 10.7551/978-0-262-32621-6-ch035

- [45] A. Y. Ng, D. Harada e S. Russell, «Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,» em *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [46] W. Xiao, L. Yuan, T. Ran, L. He, J. Zhang e J. Cui, «CSPER: Curiosity-driven Safety-Prioritized Experience Replay for Mapless Navigation using Deep Reinforcement Learning,» *IEEE Transactions on Cognitive and Developmental Systems*, vol. 18, n.º 4, pp. 1028–1042, 2026. DOI: 10.1109/TCDS.2026.3658125
- [47] A. Iskandar, «Deep Reinforcement Learning for Swarm Robotic Systems,» tese de doutoramento, University of Miskolc, István Sályi Doctoral School of Mechanical Engineering Sciences, 2026. DOI: 10.14750/ME.2026.007

ANEXO A

Declaração de Utilização de Ferramentas de Inteligência Artificial

Declara-se, para efeitos de transparência académica, que foram utilizadas ferramentas de inteligência artificial generativa — assistentes conversacionais assentes em modelos de linguagem de grande dimensão — durante a realização deste trabalho, em dois domínios distintos e com graus de intervenção diferentes. Na fase inicial, de arranque e estruturação do código, foi usado o Gemini Pro (Google); a partir daí, foram usados exclusivamente o Claude Opus e o Claude Fable (Anthropic), nas versões Claude Opus 5, Claude Opus 5.5 e Claude Fable 5.1 na fase final do trabalho.

No **desenvolvimento do software**, essas ferramentas serviram de apoio à programação: geração e reescrita de excertos de código, depuração, escrita de testes automáticos e produção dos *scripts* auxiliares de avaliação e de geração de figuras. As decisões de conceção mantiveram-se do autor — a arquitetura do simulador, o modelo físico, a definição dos sete cenários, a formulação da função de recompensa e da função de aptidão, o desenho do plano experimental e a escolha dos testes estatísticos. Todo o código incorporado no repositório foi lido, executado e validado pelo autor, com as garantias de não-regressão descritas na Secção 4.5; os resultados reportados no Capítulo 6 provêm da execução desse código, não de qualquer saída de um assistente.

Na **redação do documento**, as ferramentas foram usadas para revisão linguística e estilística, reformulação e redação de parágrafos a partir de indicações do autor, verificação da coerência terminológica e factual entre o texto, o código e os dados, e apoio à formatação em $\text{\LaTeX}$. A formulação das questões de investigação, a revisão sistemática e a triagem dos estudos, a interpretação dos resultados e as conclusões que deles se tiram são do autor, e todo o texto redigido com apoio foi revisto criticamente por ele. Nenhuma referência bibliográfica foi incluída sem verificação direta na fonte original.

Estas ferramentas têm limitações conhecidas: podem produzir código com erros subtis, afirmações factualmente incorretas ou referências inexistentes, e reproduzir enviesamentos dos dados com que foram treinadas. Por isso, nenhuma sugestão foi aceite sem verificação — o código pelos testes e pela execução, o texto por leitura crítica do autor e as referências por consulta da fonte original.

O autor assume a inteira responsabilidade pelo conteúdo desta dissertação, incluindo pelas partes em que houve apoio das ferramentas aqui declaradas.

[Página intencionalmente deixada em branco.]

ANEXO B

Configurações e Hiperparâmetros

Para garantir a reprodutibilidade integral das experiências, este anexo reúne a configuração completa do sistema, tal como definida no ficheiro `configs/foraging.yaml` do repositório (Anexo E). Os valores não resultaram de uma pesquisa automática de hiperparâmetros (*grid/Optuna*), mas sim de *calibração manual* informada por testes de fumo (*smoke tests*) curtos e pelas escolhas de referência da literatura e das implementações da *Stable-Baselines3*; documentam-se aqui os valores efetivamente utilizados nos treinos reportados no Capítulo 6. O Quadro B.1 reúne os parâmetros do ambiente e da física, comuns aos três controladores; o Quadro B.2 reúne os hiperparâmetros de treino, específicos de cada um.

QUADRO B.1. Configuração do ambiente de simulação e da física.

Parâmetro	Valor	Descrição
arena_radius	15,0 m	Raio da arena (domínio esférico)
num_agents	20	Número de agentes (N)
num_obstacles	100	Obstáculos dinâmicos
obstacle_radius	0,2 m	Raio dos obstáculos
obstacle_velocity	0,02 m/s	Velocidade dos obstáculos
nest_radius	1,5 m	Raio do ninho
nest_velocity	0,015 m/s	Velocidade do ninho móvel
required_to_eat	3	k_{min} nos cenários cooperativos (forçado a 1 nos labirintos de navegação)
hunger_timer_max	250	Limite de fome, com <i>respawn</i> do agente (ativo apenas nos cenários <i>sem</i> campo geodésico: Sandbox e Percepção Cooperativa)
lidar_range	8,0 m	Alcance máximo do LiDAR
max_steps	500	Horizonte temporal base
max_steps_cooperative_door	800	Horizonte na Porta Cooperativa
max_steps_cooperative_door_bypass	1000	Horizonte na Porta Coop. c/ Alternativa
progress_reward_factor	10,0	Fator de Progresso (PBRs)
geodesic_cell_size	0,4 m	Resolução do campo geodésico
exploration_bonus	0,5	Bônus por célula nova
exploration_cell_size	2,0 m	Resolução da grelha de exploração
min_spread_distance	1,0 m	Limiar de dispersão (física; sem sinal de recompensa)
spreading_penalty_factor	0,0	Penalização <i>anti-clustering</i> (desativada)
agent_radius	0,15 m	Raio do robô
collision_penalty	0,0	Penalização de colisão entre agentes (desativada)
obstacle_penalty	0,0	Penalização de colisão com obstáculo/parede (desativada)
food_collected	+300,0	Recompensa de tarefa (chegada)
energy_cost	-0,05	Custo energético por passo
guillotine_threshold	-200	Limiar da guilhotina (aos 150 passos)

QUADRO B.2. Hiperparâmetros de treino dos três algoritmos.

Algoritmo	Hiperparâmetro	Valor
PPO	learning_rate	10^{-4}
	n_steps (por CPU)	64
	batch_size	512
	n_epochs	4
	net_arch	[256, 256]
	num_cpu (SubprocVecEnv)	16
	γ / λ_{GAE}	0,99 / 0,95
SAC	learning_rate	10^{-4}
	buffer_size	500 000
	ent_coef (α)	0,1 (fixo)
	net_arch	[256, 256]
	num_cpu	16
Evolutivo (AE)	pop_size (K)	30
	mutation_rate (p_{mut})	0,1
	sigma (σ)	0,1 ($\times 0,999$ /geração; $\sigma_{\min} = 0,03$)
	eval_episodes (E)	4 (seeds fixas)
	hidden_dim (h)	32 ($\approx$ 8k pesos)
	Elitismo	20% ($n_e = 6$ genomas)
	fitness_food_weight	10 000 (peso da chegada em J , Eq. 5.2)
	fitness_shaping_amplitude	5 000 (peso do <i>homing</i> terminal em J)
	novelty_weight (w)	0 (base) / 0,5 (campanhas de <i>Novelty Search</i>)
	novelty_k / arquivo	10 vizinhos / máx. 1000 (FIFO, +3 por geração)

[Página intencionalmente deixada em branco.]

ANEXO C

Estudos Incluídos na Revisão Sistemática

O Quadro C.1 dá a lista integral dos estudos que sobreviveram ao fluxo PRISMA descrito na Secção 3.2 (Figura 3.1). O quadro é gerado automaticamente a partir do registo de triagem (docs/slr/screening.csv), onde constam também os 622 registos excluídos, cada um com o respetivo critério — o que permite auditar qualquer decisão desta revisão, e não apenas as inclusões.

Quadro C.1: Estudos incluídos na revisão sistemática ($n = 58$), resultantes do fluxo PRISMA (Figura 3.1). A coluna *Par.* indica o paradigma dominante e, com *E*, se o estudo aborda escalabilidade ou generalização a dimensões não vistas; é derivada dos resumos por regras explícitas, registadas estudo a estudo em docs/slr/screening.csv com os termos que a determinaram. Por ser automática, é uma aproximação: difere da leitura manual do Capítulo 3 em alguns estudos (classifica, por exemplo, como MARL a aprendizagem social de Bredeche e Fontbonne), e por isso as suas contagens (24 estudos bio-inspirados, 31 com escalabilidade) diferem numa unidade das do texto (23 e 30).

#	Autores	Ano	Título	Publicação	Par.
1	Dai et al.	2020	Multi-Robot Dynamic Task Allocation for Exploration and Destruction	Journal of Intelligent and Robotic Systems: Theory and Applications	MARL
2	Das & Jena	2020	Multi-robot path planning using improved particle swarm optimization algorithm through novel evolutionary operators	Applied Soft Computing Journal	Bio
3	Fontbonne et al.	2020	Distributed On-line Learning in Swarm Robotics with Limited Communication Bandwidth	2020 IEEE Congress on Evolutionary Computation, CEC 2020 - Conference Proceedings	Bio
4	Mısırlı et al.	2020	Fuzzy-based self organizing aggregation method for swarm robots	BioSystems	Bio, E

#	Autores	Ano	Título	Publicação	Par.
5	Konda et al.	2020	Decentralized Function Approximated Q-Learning in Multi-Robot Systems For Predator Avoidance	IEEE Robotics and Automation Letters	MARL
6	Behjat et al.	2021	Learning Robot Swarm Tactics over Complex Adversarial Environments	2021 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)	MARL, E
7	Rivière et al.	2021	Neural Tree Expansion for Multi-Robot Planning in Non-Cooperative Environments	IEEE Robotics and Automation Letters	—
8	Pagnozzi & Birattari	2021	Off-Policy Evaluation of the Performance of a Robot Swarm: Importance Sampling to Assess Potential Modifications to the Finite-State Machine That Controls the Robots	Frontiers in Robotics and AI	Bio, E
9	Zheng et al.	2021	Optimizing Parameters of Self-Organizing Model for Swarm Robots via Evolutionary Algorithms	2021 IEEE Congress on Evolutionary Computation (CEC)	Bio
10	Bredecche & Fontbonne	2022	Social learning in swarm robotics	Philosophical Transactions of the Royal Society B: Biological Sciences	MARL
11	Baumann & Martinoli	2022	A Noise-Resistant Mixed-Discrete Particle Swarm Optimization Algorithm for the Automatic Design of Robotic Controllers	2022 IEEE Congress on Evolutionary Computation (CEC)	Bio
12	Ebert et al.	2022	A Hybrid PSO Algorithm for Multi-robot Target Search and Decision Awareness	2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	Bio
13	Tang et al.	2022	Learning to Coordinate for a Worker-Station Multi-Robot System in Planar Coverage Tasks	IEEE Robotics and Automation Letters	MARL, E

#	Autores	Ano	Título	Publicação	Par.
14	Zhang et al.	2022	Moving-Distance-Minimized PSO for Mobile Robot Swarm	IEEE Transactions on Cybernetics	Bio
15	Luo et al.	2022	Physarum Polycephalum Inspired Search Strategy for Multi-robot	Lecture Notes on Data Engineering and Communications Technologies	Bio
16	Stolfi & Danoy	2022	Optimising autonomous robot swarm parameters for stable formation design	GECCO 2022 - Proceedings of the 2022 Genetic and Evolutionary Computation Conference	Bio, E
17	Liu et al.	2022	Swarm Robots Decentralized Control using Reinforcement Learning Stepwise Training Method in the Encircling Task	2022 Global Conference on Robotics, Artificial Intelligence and Information Technology (GCRAIT)	MARL
18	Ayari & Bouamama	2023	Evolutionary Swarm Robotics: A Methodological Approach For Task and Path Planning	2023 IEEE Afro-Mediterranean Conference on Artificial Intelligence (AMCAI)	Bio
19	Tan et al.	2023	Deep Reinforcement Learning for Decentralized Multi-Robot Exploration With Macro Actions	IEEE Robotics and Automation Letters	MARL, E
20	Tang et al.	2023	Autonomous Swarm Robot Coordination via Mean-Field Control Embedding Multi-Agent Reinforcement Learning	2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	MARL, E
21	Hasselmann et al.	2023	Automatic modular design of robot swarms based on repertoires of behaviors generated via novelty search	Swarm and Evolutionary Computation	Bio

#	Autores	Ano	Título	Publicação	Par.
22	Cui et al.	2023	Scalable Task-Driven Robotic Swarm Control via Collision Avoidance and Learning Mean-Field Control	2023 IEEE International Conference on Robotics and Automation (ICRA)	MARL, E
23	Luo et al.	2023	Multi-robot Cooperative Search for Radioactive Sources Based on Improved Particle Swarm Optimization	2023 2nd International Conference on Robotics, Artificial Intelligence and Intelligent Control (RAIIC)	Bio
24	Sebastian et al.	2023	LEMURS: Learning Distributed Multi-Robot Interactions	Proceedings - IEEE International Conference on Robotics and Automation	—, E
25	Sharma et al.	2023	D2CoPlan: A Differentiable Decentralized Planner for Multi-Robot Coverage	2023 IEEE International Conference on Robotics and Automation (ICRA)	—, E
26	Chen et al.	2024	An Immune Algorithm with Dual-level Ranking Mutation for Large-scale Multi-objective Multi-robot Maritime Patrolling	ICNSC 2024 - 21st International Conference on Networking, Sensing and Control: Artificial Intelligence for the Next Industrial Revolution	Bio, E
27	Ramos & Birattari	2024	Automatically designing robot swarms in environments populated by other robots: an experiment in robot shepherding	2024 IEEE International Conference on Robotics and Automation (ICRA)	Bio

#	Autores	Ano	Título	Publicação	Par.
28	Liu et al.	2024	Language-Guided Pattern Formation for Swarm Robotics with Multi-Agent Reinforcement Learning	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	MARL, E
29	Iskandar et al.	2024	Swarm Robotics Navigation Task: A Comparative Study of Reinforcement Learning and Particle Swarm Optimization Methodologies	Mekhatronika, Avtomatizatsiya, Upravlenie	Ambos, E
30	Szpirer et al.	2024	Automatic design of robot swarms that perform composite missions: an approach based on inverse reinforcement learning	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	Ambos
31	Eguchi et al.	2024	Robot Swarm Control Based on Smoothed Particle Hydrodynamics for Obstacle-Unaware Navigation	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	—
32	Kegeleirs et al.	2024	Transferability in the Automatic Off-Line Design of Robot Swarms: From Sim-to-Real to Embodiment and Design-Method Transfer Across Different Platforms	IEEE Robotics and Automation Letters	Bio, E
33	Makarov et al.	2024	Particle swarm optimization-based method of source seeking by small robotic groups in the presence of noises and communication constraints	2024 9th International Conference on Robotics and Automation Engineering (ICRAE)	Bio
34	Peter et al.	2024	Dynamic Subgoal Based Path Formation and Task Allocation: A NeuroFleets Approach to Scalable Swarm Robotics	2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)	Bio, E

#	Autores	Ano	Título	Publicação	Par.
35	He et al.	2024	Multi-spherical robot formation control based on pigeon swarm optimization	2024 3rd International Conference on Robotics, Artificial Intelligence and Intelligent Control (RAIIC)	Bio
36	Li et al.	2024	Adaptive Shape Formation Against Swarm-Scale Variants in Robot Swarms	IEEE Robotics and Automation Letters	—, E
37	Zhang et al.	2024	Self-Reconfigurable Hierarchical Frameworks for Formation Control of Robot Swarms	IEEE Transactions on Cybernetics	—
38	Zhou et al.	2024	D3G: Learning Multi-robot Coordination from Demonstrations	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)	—, E
39	Wang et al.	2024	Expert System-Based Multiagent Deep Deterministic Policy Gradient for Swarm Robot Decision Making	IEEE Transactions on Cybernetics	MARL
40	Dutceac et al.	2025	Hybrid Ant Colony and Q-Learning Algorithm for Swarm Robots: Path Planning and Collision Avoidance in Unknown Environments	2025 17th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)	Ambos
41	Calzolari et al.	2025	Reinforcement Learning Driven Multi-Robot Exploration via Explicit Communication and Density-Based Frontier Search	2025 IEEE International Conference on Robotics and Automation (ICRA)	MARL, E
42	Xu et al.	2025	Efficient Multi-Robot Task and Path Planning in Large-Scale Cluttered Environments	IEEE Robotics and Automation Letters	—, E

#	Autores	Ano	Título	Publicação	Par.
43	Liu et al.	2025	Motion Coordination of Swarm Robots for Mobile Target Search	IEEE Transactions on Automation Science and Engineering	—
44	Hammoud et al.	2025	DYNAMIC FORAGING IN SWARM ROBOTICS: A HYBRID APPROACH WITH MODULAR DESIGN AND DEEP REINFORCEMENT LEARNING INTELLIGENCE	Informatics and Automation	Ambos
45	Chiun et al.	2025	MARVEL: Multi-Agent Reinforcement Learning for Constrained Field-of-View Multi-Robot Exploration in Large-Scale Environments	2025 IEEE International Conference on Robotics and Automation (ICRA)	MARL, E
46	Zhu et al.	2025	Autonomous and Adaptive Role Selection for Multi-Robot Collaborative Area Search Based on Deep Reinforcement Learning	IEEE Transactions on Automation Science and Engineering	MARL, E
47	Biteng et al.	2025	Training Robot Swarms for Adaptive Foraging in Environments with Obstacles	2025 IEEE International Conference on Mechatronics and Automation (ICMA)	Bio, E
48	Pagliuca & Vitanza	2025	A Comparative Study of Evolutionary Strategies for Aggregation Tasks in Robot Swarms: Macro- and Micro-Level Behavioral Analysis	IEEE Access	Bio
49	Zhu et al.	2025	TaskExp: Enhancing Generalization of Multi-Robot Exploration with Multi-Task Pre-Training	2025 IEEE International Conference on Robotics and Automation (ICRA)	MARL, E
50	An et al.	2025	Scalable Multi-Robot Cooperation for Multi-Goal Tasks Using Reinforcement Learning	IEEE Robotics and Automation Letters	MARL, E

#	Autores	Ano	Título	Publicação	Par.
51	Zaman et al.	2025	Evolving Adaptive Foraging Robot Swarms with NEAT in Environments with Obstacles	2025 8th International Conference on Intelligent Robotics and Control Engineering (IRCE)	Bio, E
52	Watson et al.	2025	Predator-Prey Q-Learning Based Collaborative Coverage Path Planning for Swarm Robotics	Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)	MARL, E
53	Xin & Dames	2025	Comparison of stochastic optimization strategies in multi-robot multi-target tracking scenarios	Swarm Intelligence	Bio
54	Huang et al.	2025	A Hierarchical Multi Robot Coverage Strategy for Large Maps With Reinforcement Learning and Dense Segmented Siamese Network	IEEE Robotics and Automation Letters	MARL, E
55	Zhao et al.	2025	Enhancing Stability of Multi-Underwater Robot Swarms Based on the Fusion of Social Force and Vicsek Models	International Journal of Pattern Recognition and Artificial Intelligence	MARL, E
56	Vashisth et al.	2026	Scalable Multi-Robot Informative Path Planning for Target Mapping via Deep Reinforcement Learning	IEEE Robotics and Automation Letters	MARL, E
57	Mokhtari et al.	2026	Energy harvesting aware path planning for ambiently-powered multi-robot systems	Robotics and Autonomous Systems	MARL, E
58	Ordáz-Rivas et al.	2026	Multi-objective optimization in autonomous foraging using swarm robots	Swarm and Evolutionary Computation	Bio, E

ANEXO D

Diagrama de Classes e Excertos de Código

Apresentam-se a organização das classes do sistema e os excertos que materializam os principais contributos técnicos descritos nos Capítulos 2 e seguintes. O código integral está disponível no repositório (Anexo E).

D.1. Diagrama de Classes

A Figura D.1 mostra as classes do sistema e as relações entre elas, a alto nível: cada caixa indica o nome da classe, a classe de que herda (entre aspas angulares) e os métodos principais, sem atributos. As caixas a tracejado são de bibliotecas externas ou módulos sem classes.

D.2. Mecanismo de Atenção sobre Vizinhos (QKV)

Implementação em PyTorch puro do *forward-pass* da GNNAgent3D, sem recurso a bibliotecas de grafos. Corresponde às Equações 2.17–2.19.

```
1 # h_env: features do ego; h_all: ego + vizinhos embebidos
2 Q = self.query_proj(h_env_expanded)          # consulta (1 x d)
3 K = self.key_proj(h_all)                     # chaves (N x d)
4 V = self.value_proj(h_all)                   # valores (N x d)
5
6 # Atencao escalada por sqrt(d) -> coeficientes alpha_ij
7 scores = torch.bmm(Q, K.transpose(1, 2)) / (self.hidden_dim ** 0.5)
8 alpha = F.softmax(scores, dim=-1)
9 msg_pool = torch.bmm(alpha, V).squeeze(1)    # agregacao ponderada
10
11 # Concatena ego + mensagem agregada e gera a acao continua
12 combined = torch.cat([h_env, msg_pool], dim=1)
13 action = self.actor(combined)                # Tanh -> [-1, 1]^3
```

EXCERTO D.1. Atenção de produto interno escalado sobre a vizinhança.

D.3. Física de Deslizamento em Muros (*Wall-Sliding*)

Em vez de imobilizar o agente, o motor projeta o movimento ortogonalmente à normal da face atingida, deixando o agente deslizar ao longo da parede.

D.4. Campo Geodésico para o *Reward Shaping*

Potencial de navegação Φ usado no termo de progresso nos cenários com paredes. Uma pesquisa de custo uniforme (Dijkstra, 8-conexa) a partir da célula do ninho calcula o comprimento do caminho mais curto que contorna os obstáculos, eliminando o mínimo local da distância

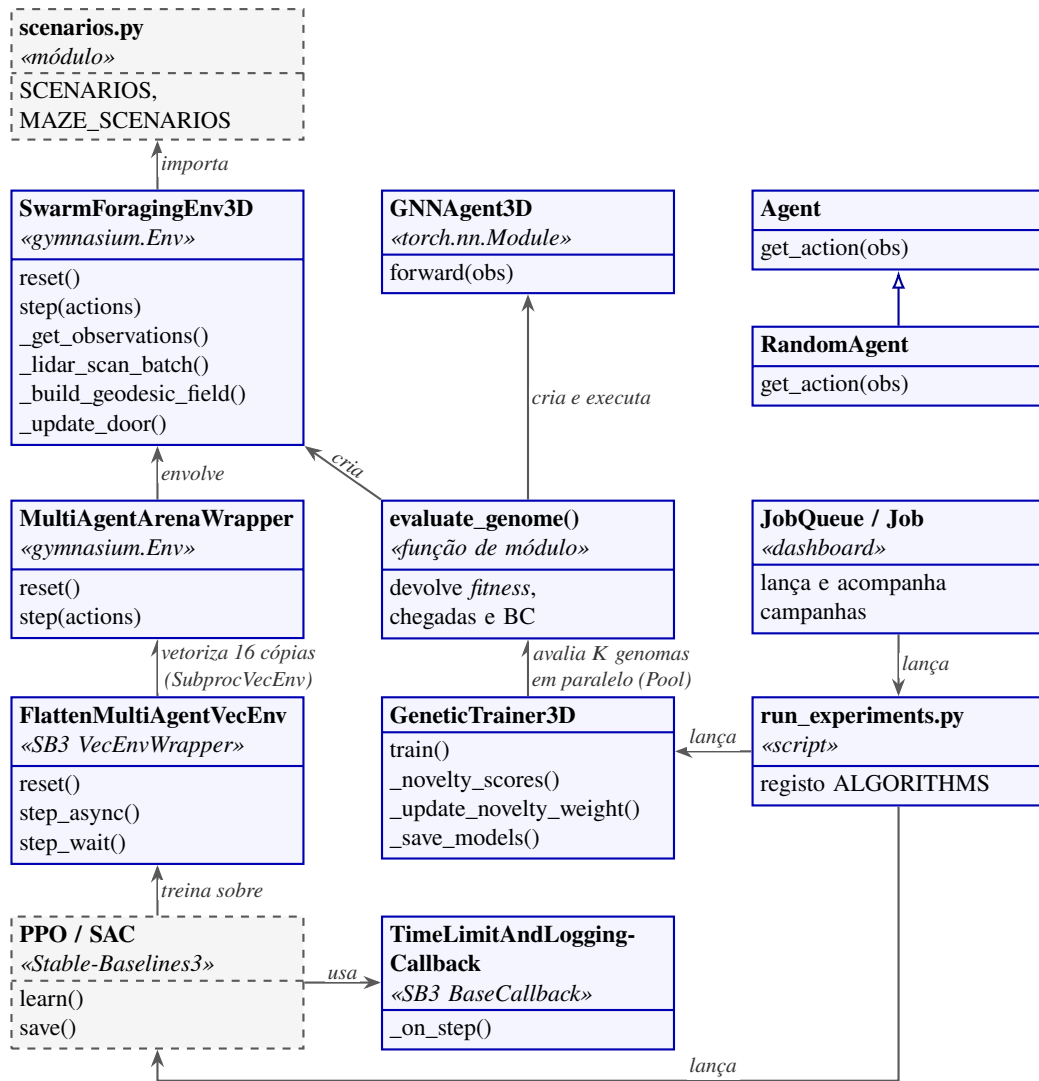


FIGURA D.1. Diagrama de classes do sistema, a alto nível (sem atributos). Setas a cheio: dependência (criação, envolvimento ou uso); seta de ponta aberta: herança. As duas classes de adaptação ao *Stable-Baselines3* (*MultiAgentArenaWrapper* e *FlattenMultiAgentVecEnv*) e a *callback* existem, com o mesmo código, nos módulos de treino do PPO e do SAC. A interface *Agent* não é implementada pelos controladores avaliados: o *GNNAgent3D* é chamado diretamente pela avaliação evolutiva e o PPO e o SAC são políticas da biblioteca. O nome *SwarmForagingEnv3D* é anterior à definição final da tarefa, que não inclui a recolha de objetos (Secção 4.2.4).

euclidiana. O campo é constante por cenário (paredes e ninho fixos), pelo que é calculado uma única vez e reutilizado.

D.5. Recompensa de Exploração *Count-Based*

Bónus atribuído na primeira visita a cada célula de uma grelha de discretização, complementando o termo de progresso geodésico ao premiar a cobertura de zonas novas onde o gradiente do potencial é pouco informativo.

```

1 for wall in self.walls:
2     next_pos = self.agent_positions[idx] + move_global
3     delta = next_pos - wall['pos']
4     half_size = wall['size'] / 2.0
5     penetration = (half_size + self.robot_radius) - np.abs(delta)
6     if np.all(penetration > 0):           # ha colisao
7         min_axis = np.argmin(penetration) # face atingida
8         normal = np.zeros(3)
9         normal[min_axis] = np.sign(delta[min_axis]) or 1.0
10        # remove a componente do movimento contra a normal (desliza)
11        move_global = move_global - np.dot(move_global, normal) *
            normal

```

EXCERTO D.2. Projeção ortogonal do movimento contra a normal do muro.

```

1 # paredes dilatadas por r_robot -> caminho fisicamente percorrivel
2 dist = np.full((n, n), np.inf); dist[ni, nj] = 0.0
3 heap = [(0.0, ni, nj)]
4 while heap:
5     d, i, j = heapq.heappop(heap)
6     if d > dist[i, j]:
7         continue
8     for di, dj, cost in neigh:           # 4 ortogonais (res) + 4
9         ii, jj = i + di, j + dj         diagonais (res*sqrt2)
10        if 0 <= ii < n and 0 <= jj < n and not blocked[ii, jj]:
11            nd = d + cost
12            if nd < dist[ii, jj]:
13                dist[ii, jj] = nd
14                heapq.heappush(heap, (nd, ii, jj))
15 # Phi(pos) = dist[celula(pos)]; progresso = 10.0 * (Phi_{t-1} - Phi_t)

```

EXCERTO D.3. Campo de distância geodésica ao ninho (Dijkstra sobre grelha).

```

1 cell = (int(pos[0] // self.exploration_cell_size),
2         int(pos[1] // self.exploration_cell_size))
3 if cell not in self.visited_cells[idx]:
4     self.visited_cells[idx].add(cell)
5     rewards[agent] += self.exploration_bonus # +0.5 por celula nova

```

EXCERTO D.4. Bónus de exploração por cobertura de células novas.

[Página intencionalmente deixada em branco.]

ANEXO E

Recursos Adicionais

A totalidade do código-fonte — simulador, implementações dos três algoritmos, *scripts* de avaliação e geração de figuras, e o *dashboard* de controlo experimental — encontra-se disponível, de forma aberta, no repositório do projeto:

<https://github.com/Pombo2001/swarm-robotics-tese>

O repositório inclui o ficheiro `configs/foraging.yaml` com a configuração reproduzida no Anexo B, um `README.md` com o procedimento de instalação e de reprodução dos resultados (treino, avaliação determinística, testes estatísticos e geração de gráficos), e os modelos treinados por cenário. As demonstrações qualitativas do comportamento emergente são reproduzíveis localmente através do visualizador 3D (motor Ursina), executado a partir do *dashboard* de controlo.